# Insights from Inside Neural Networks

Andrea Ferrario* Alexander Noll† Mario V. Wüthrich‡

Prepared for:
Fachgruppe "Data Science"
Swiss Association of Actuaries SAV

Version of April 23, 2020

**Abstract**

We provide a tutorial that illuminates different aspects that need to be considered when fitting neural network regression models to claim frequency insurance data. We discuss feature pre-processing, choice of loss function, choice of neural network architecture, class imbalance problem, as well as over-fitting and bias regularization. This discussion is based on a publicly available real car insurance data set.

**Keywords.** neural networks, architecture, over-fitting, loss function, dropout, regularization, LASSO, ridge, gradient descent, class imbalance, bias regularization, balance property, car insurance, claim frequency, Poisson regression model, machine learning, deep learning.

## 0 Introduction and overview

This data analytics tutorial has been written for the working group "Actuarial Data Science" of the Swiss Association of Actuaries SAV, see

`https://www.actuarialdatascience.org`

The main purpose of this tutorial is to illuminate the aspects that need to be considered when fitting neural network regression models to claim frequency data in insurance. This tutorial is based on our introductory tutorial Noll et al. [22]. We use the same French motor third-party liability (MTPL) insurance data set as in our introductory tutorial, and we provide an in-depth analysis of neural network calibration on that data set. As a result, we see that the models in Noll et al. [22] may be improved by a careful model choice.

We will present references while moving through this tutorial, for a general introduction to neural networks we recommend Goodfellow et al. [11].

*Mobiliar Lab for Analytics, ETH Zurich, aferrario@ethz.ch
†PartnerRe Holdings Europe Limited, alexander.noll.a@gmail.com
‡RiskLab, Department of Mathematics, ETH Zurich, mario.wuethrich@math.ethz.ch

# 1 The data and a warming-up exercise

## 1.1 French motor third-party liability insurance data

We revisit the data `freMTPL2freq` included in the R package `CASdatasets`, see Charpentier [4].[1] This data comprises a French motor third-party liability (MTPL) insurance portfolio with corresponding claim counts observed within one accounting year. This data has been illustrated and studied in our tutorial Noll et al. [22]. Listing 1 provides an excerpt of the data.

Listing 1: output of command `str(freMTPL2freq)`

```
1  > str(freMTPL2freq)
2  'data.frame':   678013 obs. of  12 variables:
3   $ IDpol     : num  1 3 5 10 11 13 15 17 18 21 ...
4   $ ClaimNb   : num [1:678013(1d)] 1 1 1 1 1 1 1 1 1 1 ...
5    ..- attr(*, "dimnames")=List of 1
6    .. ..$ : chr  "139" "414" "463" "975" ...
7   $ Exposure  : num  0.1 0.77 0.75 0.09 0.84 0.52 0.45 0.27 0.71 0.15 ...
8   $ Area      : Factor w/ 6 levels "A","B","C","D",..: 4 4 2 2 2 5 5 3 3 2 ...
9   $ VehPower  : int  5 5 6 7 7 6 6 7 7 7 ...
10  $ VehAge    : int  0 0 2 0 0 2 2 0 0 0 ...
11  $ DrivAge   : int  55 55 52 46 46 38 38 33 33 41 ...
12  $ BonusMalus: int  50 50 50 50 50 50 50 68 68 50 ...
13  $ VehBrand  : Factor w/ 11 levels "B1","B10","B11",..: 4 4 4 4 4 4 4 4 4 4 ...
14  $ VehGas    : Factor w/ 2 levels "Diesel","Regular": 2 2 1 1 1 2 2 1 1 1 ...
15  $ Density   : int  1217 1217 54 76 76 3003 3003 137 137 60 ...
16  $ Region    : Factor w/ 22 levels "R11","R21","R22",..: 18 18 3 15 15 8 8 20 20 12 ...
```

A detailed descriptive analysis of this data is provided in our tutorial Noll et al. [22]. The analysis in that reference also includes a (minor) data cleaning part on the original data, which is used but not further discussed in the present manuscript.

## 1.2 Descriptive statistics of the exposure measure

We start by providing descriptive and exploratory statistics about the exposure measure in this claim frequency example. The original model in Noll et al. [22] assumed that all insurance policies $i = 1, \ldots, 678'013$ have independent Poisson claim counts $N_i$ being described by

$$N_i \overset{\text{ind.}}{\sim} \text{Poi}\left(\lambda(\boldsymbol{x}_i)v_i\right), \tag{1.1}$$

for given volumes $v_i > 0$ (time `Exposure` in years on line 7 of Listing 1) and a given claim frequency function $\boldsymbol{x}_i \mapsto \lambda(\boldsymbol{x}_i)$, where $\boldsymbol{x}_i$ describes the feature information of policy $i$, see Assumptions 2.1 in Noll et al. [22] and lines 8-16 of Listing 1. Since all policies were active within the same accounting year, the volumes were considered pro-rata temporis $v_i \in (0, 1]$ for the corresponding exposures. A time exposure as a volume measure has been criticized in the literature, see e.g. Verbelen et al. [33], and other exposure measures have been proposed. In the descriptive analysis in this section we analyze the linearity of the expected number of claims in this time exposure, i.e. we analyze empirically the linearity $v_i \mapsto \mathbb{E}[N_i] = \lambda(\boldsymbol{x}_i)v_i$.[2] We

[1] CASdatasets website `http://cas.uqam.ca`; the data is described on page 55 of the reference manual [3]; we use version CASdatasets 1.0-8; note that the data has slightly changed in more recent versions.

[2] Linearity in the volume means that it is considered as a known offset in the regression function.

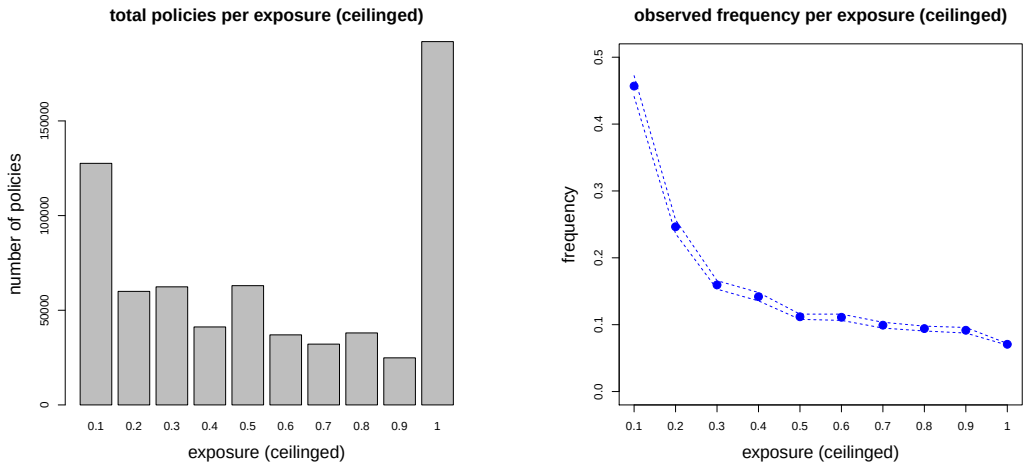


Figure 1: (lhs) histogram of the number of policies in each exposure group $I_k$, (rhs) claim frequency $\overline{f}_k$ per exposure group $I_k$, $k = 1, \ldots, 10$.

therefore build 10 exposure groups $I_k = \left(\frac{k-1}{10}, \frac{k}{10}\right]$, for $k = 1, \ldots, 10$, and we study the empirical frequencies on these exposure groups given by

$$\overline{f}_k = \frac{\sum_{i=1}^n N_i \, \mathbb{1}_{\{v_i \in I_k\}}}{\sum_{i=1}^n v_i \, \mathbb{1}_{\{v_i \in I_k\}}}.$$

In Figure 1 (lhs) and on the second line of Table 1 we provide the (relative) numbers of policies

| exposure group $I_k$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|
| rel. number of policies | 18.8% | 8.8% | 9.2% | 6.1% | 9.3% | 5.5% | 4.7% | 5.6% | 3.7% | 28.3% |
| empirical frequency $\overline{f}_k$ | 45.7% | 24.6% | 15.9% | 14.2% | 11.2% | 11.1% | 9.9% | 9.4% | 9.2% | 7.1% |

Table 1: relative number of policies in each exposure group and corresponding empirical frequencies $\overline{f}_k$, for $k = 1, \ldots, 10$.

in each exposure group $I_1, \ldots, I_{10}$. We observe that roughly 70% of all policies are exposed less than one accounting year. This is a rather high percentage of policies that are only partially exposed during the accounting year, and it shows that there is an issue in this data. Further analysis indicates that policy renewals during the year have been counted as two policies, for instance, if a policy expires at the end of March and is renewed for the remainder of the year, then the policy has two entries in the data, one with exposure 1/4 and one with 3/4. This seems unfortunate because it corresponds to the same driver, but unfortunately we do not have the sufficient information to merge such policies. The disclaimer at this stage is that we ignore these data problems and just continue, but in a real insurance situation this requires further investigation and data cleaning. Note that early termination during the year may also be caused by claims, which typically allows (both sides) to terminate an insurance contract. Therefore, the reason for early termination should be clarified.

In Figure 1 (rhs) and on the third line of Table 1 we provide the resulting empirical frequencies $\overline{f}_k$ on each exposure group $I_1, \ldots, I_{10}$. These empirical frequencies show a substantial decrease

in increasing exposure. This clearly indicates that the expected number of claims may not be proportional to the time exposure $v_i > 0$. Again, coming back to our disclaimer, one should explore whether the high frequencies on short exposures are caused by early termination of contracts after claims, or whether there are other reasons for these high frequencies on small exposure. Unfortunately, this information is not available, here.

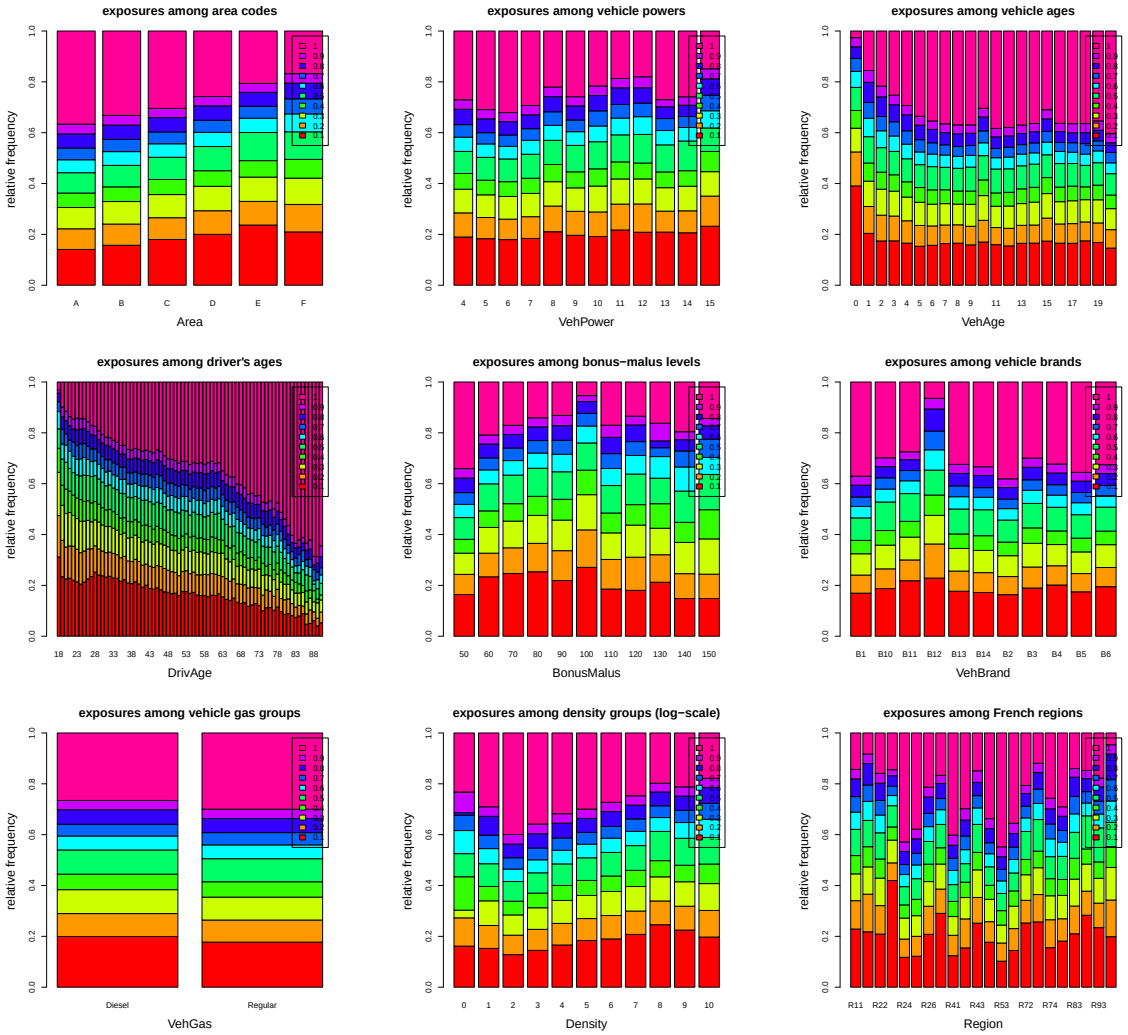


Figure 2: bar plots of the relative numbers of policies in each exposure group $I_1, \ldots, I_{10}$ (red, orange, yellow, $\ldots$, blue, violet, magenta) for all labels of all feature components of $\boldsymbol{x}_i$.

Of course, the latter conclusion is also not fully justified because $\overline{f}_k$ only considers a *marginal* frequency, which may depend on the underlying portfolio structure. For this reason we also analyze interactions between the exposures $v_i$ and the other feature components of $\boldsymbol{x}_i$. In Figure 2 we provide the bar plots of the relative numbers of policies in each exposure group $I_1, \ldots, I_{10}$ (red, orange, yellow, $\ldots$, blue, violet, magenta colors) for each label of each feature component of $\boldsymbol{x}_i$. We see non-homogeneity and rather strong interactions between the exposures $v_i$ and some feature components of $\boldsymbol{x}_i$. For instance, exposure is clearly increasing in driver's age, drivers with age 18 have hardly a full year of exposure, whereas 70% of the drivers with an age above 90

are exposed during the whole accounting year (this may illustrate changes in the renewal policy over the years, as the portfolio is aging). Most of the other feature components show a similar picture, for instance, vehicle brand B12 is quite different compared to other vehicle brands; a bonus-malus level of 100% has the shortest exposures (because this label typically contains new insurance contracts); vehicle ages 10 and 15 seem special (this may indicate newly bought second-hand cars where the age of the car is unknown). Such non-homogeneity may partially explain that $\overline{f}_k$ is non-constant in $k$. To get a more sophisticated answer to the question of (non-)linear exposure measures we extend model assumption (1.1). This is done in the next section.

### 1.3 Warming-up exercise in neural network modeling

As a warming-up exercise we resume the neural network modeling approach of Section 6 of Noll et al. [22]. But we replace model assumption (1.1) by the following Poisson regression model

$$N_i \overset{\text{ind.}}{\sim} \text{Poi}\left(\mu(\boldsymbol{x}_i, v_i)\right), \tag{1.2}$$

where we choose a regression function on the extended feature space $\mathcal{X}^+ = \mathcal{X} \times (0,1]$ given by

$$\mu : \mathcal{X}^+ \to \mathbb{R}_+, \qquad \text{with} \qquad (\boldsymbol{x}, v) \mapsto \mu(\boldsymbol{x}, v), \tag{1.3}$$

and where $\mathcal{X}$ is the feature space introduced in Section 6.1 of Noll et al. [22] (containing all potential insurance policies). Thus, model (1.1) is obtained as a special case of model (1.2) by choosing the regression function in (1.3) as $\mu(\boldsymbol{x}, v) = \lambda(\boldsymbol{x})v$. Next, we introduce a fully-connected single hidden layer feed-forward neural network with $q_1 = 20$ hidden neurons for modeling regression function (1.3). This is exactly the same model as in Section 6 of Noll et al. [22], except that we extend the input layer by the additional volume component $v \in (0,1]$. We introduce this neural network in a formal way because the corresponding notation will be used throughout this tutorial.

We start by defining a general feed-forward neural network, subsequently abbreviated as *network*. Choose $k \geq 1$ and hyperparameters $q_{k-1}, q_k \in \mathbb{N}$. A network *layer* is a mapping

$$\boldsymbol{z}^{(k)} : \mathbb{R}^{q_{k-1}} \to \mathbb{R}^{q_k}, \qquad \boldsymbol{z} \mapsto \boldsymbol{z}^{(k)}(\boldsymbol{z}) = \left(z_1^{(k)}(\boldsymbol{z}), \ldots, z_{q_k}^{(k)}(\boldsymbol{z})\right)', \tag{1.4}$$

with $q_k$ *hidden neurons* in the ($k$*-th hidden*) layer given by

$$z_j^{(k)}(\boldsymbol{z}) = \phi\left(w_{j,0}^{(k)} + \sum_{l=1}^{q_{k-1}} w_{j,l}^{(k)} z_l\right) \overset{\text{def.}}{=} \phi\langle \boldsymbol{w}_j^{(k)}, \boldsymbol{z}\rangle, \qquad \text{for } j = 1, \ldots, q_k, \tag{1.5}$$

with weights $\boldsymbol{w}^{(k)} = (\boldsymbol{w}_1^{(k)}, \ldots, \boldsymbol{w}_{q_k}^{(k)})' = (w_{1,0}^{(k)}, \ldots, w_{q_k,q_{k-1}}^{(k)})' \in \mathbb{R}^{q_k(1+q_{k-1})}$ and activation function $\phi : \mathbb{R} \to \mathbb{R}$. We give some remarks:

- The $k$-th hidden layer is obtained by (feed-forward) mapping the $q_{k-1}$ neurons $\boldsymbol{z}^{(k-1)} = (z_1^{(k-1)}, \ldots, z_{q_{k-1}}^{(k-1)})' \in \mathbb{R}^{q_{k-1}}$ to the $q_k$ neurons $\boldsymbol{z}^{(k)} = (z_1^{(k)}, \ldots, z_{q_k}^{(k)})' \in \mathbb{R}^{q_k}$, see (1.4). Thus, layer $k-1$ has $q_{k-1}$ neurons and layer $k$ has $q_k$ neurons. $q_{k-1}$ and $q_k$ are hyperparameters determining the architecture of the network. Moreover, we initialize $q_0$ to be the dimension of $\mathcal{X}^+$ with initial neurons $\boldsymbol{z}^{(0)} = (\boldsymbol{x}, v) \in \mathcal{X}^+$.

- The $k$-th hidden layer has $q_k(1+q_{k-1})$ parameters $\boldsymbol{w}^{(k)} \in \mathbb{R}^{q_k(1+q_{k-1})}$ called *weights*. They describe the scalar products $\langle \boldsymbol{w}_j^{(k)}, \boldsymbol{z} \rangle$ in the neurons, providing a reduction of dimension (in a first step) from $q_{k-1}$ to 1, for indexes $j = 1, \ldots, q_k$. The intercepts $w_{j,0}^{(k)}$ are also called *biases* in neural network literature.

- $\phi : \mathbb{R} \to \mathbb{R}$ is a (non-linear) activation function, which models the strengths of the activations in the neurons. Often, one of the following four choices is made

$$\phi(x) = \begin{cases} \frac{1}{1+e^{-x}} & \text{sigmoid activation function,} \\ \tanh(x) & \text{hyperbolic tangent activation function,} \\ \mathbb{1}_{\{x \geq 0\}} & \text{step function activation,} \\ x\mathbb{1}_{\{x \geq 0\}} & \text{rectified linear unit (ReLU) activation function.} \end{cases} \tag{1.6}$$

- For more comments we refer to Remarks 6.2 in Noll et al. [22].

A general network architecture with $K$ hidden layers for our Poisson regression problem is obtained as follows: choose hyperparameters $q_1, \ldots, q_K \in \mathbb{N}$ and initialize $q_0$ to be the dimension of the feature space $\mathcal{X}^+ \subset \mathbb{R}^{q_0}$ (which provides the *input layer*). The network regression function is defined by the *composition*

$$\mu : \mathcal{X}^+ \to \mathbb{R}_+, \qquad (\boldsymbol{x}, v) \mapsto \mu(\boldsymbol{x}, v) = \left(g \circ \boldsymbol{z}^{(K:1)}\right)(\boldsymbol{x}, v) = \left(g \circ \boldsymbol{z}^{(K)} \circ \cdots \circ \boldsymbol{z}^{(1)}\right)(\boldsymbol{x}, v),$$

with $\boldsymbol{z}^{(K:1)} = \boldsymbol{z}^{(K)} \circ \cdots \circ \boldsymbol{z}^{(1)}$ and with log-linear regression function $g : \mathbb{R}^{q_K} \to \mathbb{R}_+$ defined by

$$\boldsymbol{z}^{(K:1)} \;\mapsto\; g(\boldsymbol{z}^{(K:1)}) = \exp\left(w_0^{(K+1)} + \sum_{j=1}^{q_K} w_j^{(K+1)} z_j^{(K:1)}\right) = \exp\langle \boldsymbol{w}^{(K+1)}, \boldsymbol{z}^{(K:1)} \rangle. \tag{1.7}$$

This last layer (called *output layer*) only receives $q_{K+1} = 1$ neuron, and as activation function we choose the exponential function because claim frequencies should be strictly positive. This network architecture has *depth* $K$ and receives a *network parameter* $\theta \in \mathbb{R}^r$ of dimension $r = \sum_{k=1}^{K+1} q_k(1+q_{k-1})$ collecting all network weights $\boldsymbol{w}^{(k)}$, $k = 1, \ldots, K+1$. Two examples with $K = 1, 2$ hidden layers are given in Figure 3. These two examples have network parameters of dimensions $r = 241$ and $r = 195$, respectively.

**Remarks 1.1 (generalized linear model, GLM)** We work in a Poisson regression model, here. This model belongs to the exponential dispersion family (EDF), see Wüthrich [35]. The log-link is the canonical link function for the Poisson regression model and, in this sense, we work under the canonical link in (1.7). In the special case of depth $K = 0$, i.e. without hidden layers, (1.7) provides the Poisson generalized linear model (GLM). A GLM is also obtained by choosing a linear activation function $\phi(x) = x$ because the composition of linear functions remains a linear function, that is, the depth of the network is not a relevant feature in this linear case (only the dimension $q_k$, $0 \leq k \leq K$, of the smallest layer matters).

For our warming-up example we choose exactly the same set-up as in Section 6 of Noll et al. [22], the only difference is that we now consider input $(\boldsymbol{x}, v) \in \mathcal{X}^+$ for modeling the expected number of claims $\mu(\boldsymbol{x}, v)$, whereas in [22] we have been considering input $\boldsymbol{x} \in \mathcal{X}$ for modeling the special case of $\lambda(\boldsymbol{x})v$. In particular, this modeling includes (i) feature pre-processing as in [22], (ii)

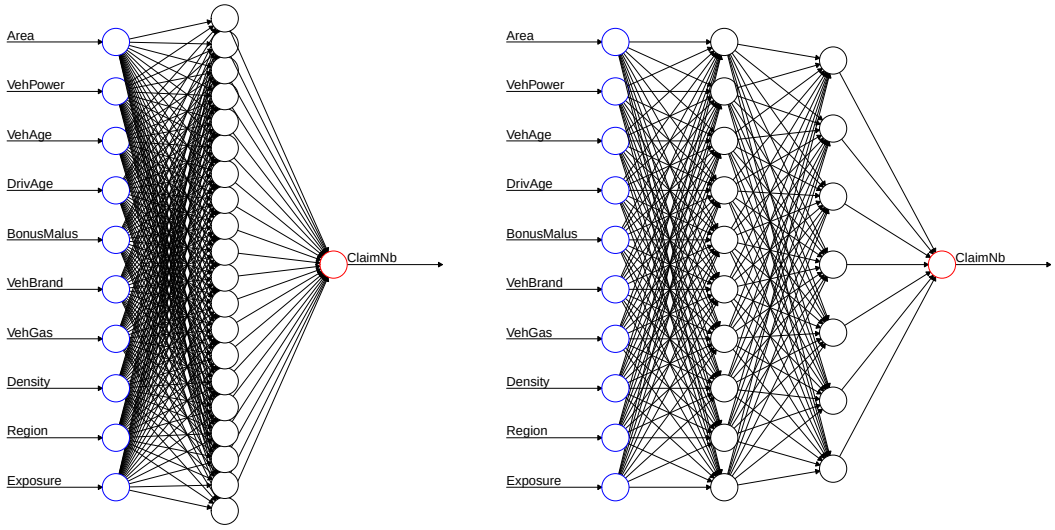


Figure 3: (lhs) (shallow) network with $K = 1$ hidden layers having $q_1 = 20$ neurons; (rhs) (deep) network with $K = 2$ hidden layers having $q_1 = 10$ and $q_2 = 7$ hidden neurons in the first and second hidden layer, respectively; input layers have dimension $q_0 = 10$ here; the input layer has blue color and the output layer has red color.

choice of a network with $K = 1$ hidden layers, (iii) choice of $q_1 = 20$ hidden neurons, and (iv) choice of hyperbolic tangent activation function $\phi(x) = \tanh(x)$. This corresponds to Figure 3 (lhs), and the network regression function reads as

$$(\boldsymbol{x}, v) \;\mapsto\; \mu(\boldsymbol{x}, v) = \exp\left( w_0^{(2)} + \sum_{j=1}^{q_1} w_j^{(2)} \tanh\left( w_{j,0}^{(1)} + \sum_{l=1}^{q_0-1} w_{j,l}^{(1)} x_l + w_{j,q_0}^{(1)} v \right) \right), \tag{1.8}$$

where the last terms $w_{j,q_0}^{(1)} v$, $j = 1, \ldots, q_1$, are the main difference to the model in [22]. These

| | in-sample loss | out-of-sample loss |
|---|---|---|
| network with $q_1 = 20$ and $(\boldsymbol{x}, v) \mapsto \lambda(\boldsymbol{x})v$: model (1.1) | 30.45048 | 31.58770 |
| network with $q_1 = 20$ and $(\boldsymbol{x}, v) \mapsto \mu(\boldsymbol{x}, v)$: model (1.2) | 29.25702 | 30.52270 |

Table 2: in-sample and out-of-samples losses of the network predictions for the two regression functions $(\boldsymbol{x}, v) \mapsto \lambda(\boldsymbol{x})v$ and $(\boldsymbol{x}, v) \mapsto \mu(\boldsymbol{x}, v)$; losses are in $10^{-2}$.

modeling choices will be discussed in much more detail in the sections below, and we are also going to discuss the implementation, below.

For the moment, we just fit this model as described in Section 6 of [22], using the R library `keras`, and we perform exactly the same in-sample and out-of-sample analysis as in [22]. The results are presented in Table 2. We observe a clear decrease in the resulting losses from the former model (1.1) to the latter model (1.2). This suggests that the time exposure $v \in (0, 1]$ should not be considered in a linear fashion, supporting actuarial literature, but the general approach with regression function $\mu$ as in (1.8) may be more appropriate. However, we emphasize once

more our disclaimer, that we do not have sufficient information about our data to do a sensible decision about the best consideration of the exposure measure.

### 1.4 Designing a network

The choice of a particular network architecture and its calibration involve many steps which we are going to discuss in detail in this tutorial. In each of these steps the modeler has to make certain decisions, and it may require that each of these decisions is revised several times in order to get the best (or more modestly a good) predictive model. The choices involve:

(a) data cleaning and data pre-processing

(b) choice of loss function (objective function) and performance measure for model calibration;

(c) number of hidden layers $K$;

(d) number of neurons $q_1, \ldots, q_K$ in the hidden layers;

(e) choice of activation function $\phi$;

(f) optimization algorithm used for calibration which may include further choices of

  (i) initialization of algorithm,

  (ii) random (mini-)batches of data,

  (iii) stopping, number of iterations, number of epochs, etc.,

  (iv) parameters like learning rates, momentum parameters, etc.;

(g) normalization layers, dropout rates;

(h) regularization like LASSO or ridge regression, etc.

These choices correspond to the modeling cycle that is typically performed in statistical applications, the final validation step is not mentioned above.

## 2 Data cleaning and data pre-processing: item (a)

In general, data cleaning and data pre-processing is the most important task in statistical modeling, and surely it is the most time consuming one. Often more than 80% of the total time is spent for this task. In this section we will not further discuss data cleaning, but we are going to highlight a few important points in data pre-processing that are necessary for a successful application of networks.

In network modeling the choice of the scale of the feature components may substantially influence the fitting procedure of the predictive model. Therefore, data pre-processing requires careful consideration. We treat unordered categorical (nominal) feature components and continuous (or ordinal) feature components separately. Ordered categorical feature components are treated like continuous ones, where we simply replace the ordered categorical labels by integers. Binary categorical feature components are coded by 0's and 1's for the two binary labels (for binary labels we do not distinguish between ordered and unordered components). Remark that

if we choose an anti-symmetric activation function, i.e. $-\phi(x) = \phi(-x)$, we may also set binary categorical feature components to $\pm 1/2$, which may simplify initialization of optimization algorithms.

## 2.1 Unordered (nominal) categorical feature components

We need to transform (nominal) categorical feature components to numerical values. The most commonly used transformations are the so-called *dummy coding* and the *one-hot encoding*. Both methods construct binary representations for categorical labels. For dummy coding one label is chosen as reference level. Dummy coding then uses binary variables to indicate which label a particular policy possesses *if it differs* from the reference level. In our example we have two unordered categorical feature components, namely `VehBrand` and `Region`.[3] We use `VehBrand` as illustration. It has 11 different labels $\{\texttt{B1}, \texttt{B10}, \texttt{B11}, \texttt{B12}, \texttt{B13}, \texttt{B14}, \texttt{B2}, \texttt{B3}, \texttt{B4}, \texttt{B5}, \texttt{B6}\}$. We choose `B1` as reference label. Dummy coding then provides the coding scheme given in Table 3 (lhs). We observe that the 11 labels are replaced by 10-dimensional feature vectors $\boldsymbol{x}^\star$ in $\{0,1\}^{10}$, with

| label | feature components $\boldsymbol{x}^\star \in \{0,1\}^{10}$ | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| B1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| B10 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| B11 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| B12 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| B13 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 |
| B14 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 |
| B2 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 |
| B3 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| B4 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 |
| B5 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 |
| B6 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 |

| label | feature components $\boldsymbol{x}^\star \in \{0,1\}^{11}$ | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| B1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| B10 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| B11 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| B12 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| B13 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 |
| B14 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 |
| B2 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 |
| B3 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| B4 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 |
| B5 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 |
| B6 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 |

Table 3: examples of dummy coding (lhs) and one-hot encoding (rhs) for the categorical `VehBrand` labels.

components summing up to either 0 or 1 (row sums in Table 3, lhs).
In contrast to dummy coding, one-hot encoding does not choose a reference level, but uses an indicator for each label. In this way the 11 labels of `VehBrand` are replaced by the 11 unit vectors in $\mathbb{R}^{11}$, see Table 3 (rhs). The main difference between dummy coding and one-hot encoding is that the former leads to full rank design matrices, whereas the latter does not. This implies that under one-hot encoding there are identifiability issues in parametrizations. In network modeling identifiability is less important because we typically work in over-parametrized non-convex optimization problems (with multiple equally good models/parametrizations); on the other hand, identifiability in GLMs is an important feature because one typically tries to solve a convex optimization problem, where the full rank property is important to efficiently find the (unique) solution.
Remark that other coding schemes could be used for categorical feature components such as Helmert's contrast coding. In classical GLMs the choice of the coding scheme typically does

[3] We treat the feature component `Area` as ordered categorical, as indicated in Noll et al. [22].

not influence the prediction, however, interpretation of the results may change by considering a different contrast. In network modeling the choice of the coding scheme may influence the prediction: typically, we exercise an early stopping rule in network calibrations. This early stopping rule and the corresponding result may depend on any chosen modeling strategy, such as the encoding scheme of categorical feature components.

Remark that dummy coding and one-hot encoding may lead to very high-dimensional input layers in networks, and it provides sparsity in input features. Moreover, the Euclidean distance between any two labels in the one-hot encoding scheme is that same. From natural language processing (NLP) we have learned that there are more efficient ways of representing categorical feature components, namely, by embedding them into lower-dimensional spaces so that proximity in these spaces has a useful meaning in the regression task. In networks this can be achieved by so-called embedding layers. In actuarial science, these have been proposed by Richman [27] and they have been explored in a life insurance example in Richman–Wüthrich [28]. For a broader discussion of embedding layers in the context of our French MTPL example we refer to our tutorial Schelldorfer–Wüthrich [31].

### 2.2 Continuous feature components

In theory, continuous feature components do not need pre-processing if we choose a sufficiently rich network, because the network may take care of feature components living on different scales. This statement is of purely theoretical value. In practice, continuous feature components need pre-processing such that they all live on a similar scale and such that they are sufficiently equally distributed across this scale. The reason for this requirement is that the calibration algorithms mostly use gradient descent methods (GDMs). These GDMs only work properly, if all components live on a similar scale and, thus, all directions contribute equally to the gradient. Otherwise, the optimization algorithms may get trapped in saddle points or in regions where the gradients are flat (also known as vanishing gradient problem). Often, one uses $[-1,1]$ as the common scale because the/our choice of activation function is focused to that scale, see (1.6); we also refer to Section 4.5, below.

A popular transformation is the so-called *MinMaxScaler*. For this transformation we fix each continuous feature component of $\boldsymbol{x}$, say $x_l$, at a time. Denote the minimum and the maximum of the domain of $x_l$ by $m_l$ and $M_l$, respectively. The MinMaxScaler then replaces

$$x_l \;\mapsto\; x_l^\star = \frac{2(x_l - m_l)}{M_l - m_l} - 1 \;\in\; [-1,1].$$

In practice, it may happen that the minimum $m_l$ or the maximum $M_l$ is not known. In this case one chooses the corresponding minimum and/or maximum of the features in the observed data. For prediction under new features one then needs to keep the original scaling of the initially observed data, i.e. the one which has been used for model calibration.

Another popular transformation considers the empirical residuals of each continuous feature component over the entire portfolio. For this we denote by $\overline{x}_l$ and $s_l$ the empirical mean and standard deviation of the continuous feature component $x_{i,l}$ over the portfolio $i = 1, \ldots, n$. We then replace

$$x_l \;\mapsto\; x_l^\star = \frac{x_l - \overline{x}_l}{s_l}.$$

The empirical mean and standard deviation should only be based on the learning data $\mathcal{D}$, because the test data $\mathcal{T}$ should "truly" be unseen during learning; see Section 3.4 below for the definition of the learning data $\mathcal{D}$ and the test data $\mathcal{T}$. However, exactly the same scaling constants should be applied to the features in the learning *and* test data.
Remark that if we have outliers, the above transformations may lead to very concentrated transformed feature components $x_{i,l}^\star$, $i = 1, \ldots, n$, because the outliers may, for instance, dominate the maximum in the MinMaxScaler. In this case, feature components should be transformed first by a log-transformation or by a quantile transformation so that they become more equally spaced (and robust) across the real line.

**Conclusion.**
In our example we use dummy coding for the feature components `VehBrand` and `Region`. We use the MinMaxScaler for `Area` (after transforming $\{\mathrm{A}, \ldots, \mathrm{F}\}$ to $\{1, \ldots, 6\}$), `VehPower`, `VehAge` (after capping at age 20), `DrivAge` (after capping at age 90), `BonusMalus` (after capping at level 150) and `Density` (after first taking the log-transform). `VehGas` we transform to $\pm 1/2$ and the volume `Exposure` $\in (0, 1]$ we keep untransformed. The resulting feature space $\mathcal{X}^+$ has dimension $q_0 = 39$ (note that this differs from Figure 3 because we now use dummy coding for categorical feature components which turns `VehBrand` into a 10-dimensional dummy vector and `Region` into a 21-dimensional dummy vector). This exactly corresponds to the second model presented in Table 2 (if we use a shallow network with $q_1 = 20$ hidden neurons), and it has a network parameter of dimension $r = 821$.

# 3 Choice of loss function (objective function): item (b)

## 3.1 Poisson deviance loss function

For claim frequency modeling we make the Poisson model assumption (1.2). The canonical choice of an objective function under this model choice is the Poisson deviance loss. For a given estimator $\widehat{\mu}(\cdot)$ of $\mu(\cdot)$, the Poisson deviance loss on data $\mathcal{D} = \{(N_i, \boldsymbol{x}_i, v_i) : i = 1, \ldots, n\}$ is given by

$$\mathcal{L}(\mathcal{D}, \widehat{\mu}(\cdot)) \;=\; \frac{1}{n} \sum_{i=1}^{n} 2N_i \left[ \frac{\widehat{\mu}(\boldsymbol{x}_i, v_i)}{N_i} - 1 - \log\left( \frac{\widehat{\mu}(\boldsymbol{x}_i, v_i)}{N_i} \right) \right]. \tag{3.1}$$

If we consider a homogeneous expected frequency model $\mu(\boldsymbol{x}, v) \equiv \lambda v$, the minimizer $\widehat{\mu}(\boldsymbol{x}, v) = \widehat{\lambda} v$ of the deviance loss (3.1) is obtained by the MLE

$$\widehat{\lambda}^{\mathrm{MLE}} = \frac{\sum_{i=1}^n N_i}{\sum_{i=1}^n v_i}. \tag{3.2}$$

In this homogeneous model the MLE is unbiased and its uncertainty can be quantified:

$$\mathbb{E}\left[\widehat{\lambda}^{\mathrm{MLE}}\right] = \lambda \qquad \text{and} \qquad \mathrm{Var}\left(\widehat{\lambda}^{\mathrm{MLE}}\right) = \frac{\lambda}{\sum_{i=1}^n v_i}.$$

Remark that on the set of distribution functions with finite first moment, the Poisson deviance scoring (loss) function is strictly consistent for the expected value, see Definition 2.1 in Gneiting [10].

### 3.2 Square loss functions

A second option that is often considered as objective functions are square losses and weighted square losses. These are given by, respectively,

$$\mathcal{L}(\mathcal{D}, \widehat{\mu}(\cdot)) \;=\; \frac{1}{n}\sum_{i=1}^{n} \left(N_i - \widehat{\mu}(\boldsymbol{x}_i, v_i)\right)^2,$$

and

$$\mathcal{L}(\mathcal{D}, \widehat{\mu}(\cdot)) \;=\; \frac{1}{n}\sum_{i=1}^{n} \frac{1}{\widehat{\mu}(\boldsymbol{x}_i, v_i)} \left(N_i - \widehat{\mu}(\boldsymbol{x}_i, v_i)\right)^2.$$

These choices are motivated by the properties, under model assumption (1.2),

$$\mathbb{E}[N_i] = \mu(\boldsymbol{x}_i, v_i) \qquad \text{and} \qquad \mathrm{Var}(N_i) = \mu(\boldsymbol{x}_i, v_i).$$

In general, we do not recommend these latter two loss functions: the square loss does not consider the underlying volumes appropriately, and the weighted square loss is not robust because the estimated frequency $\widehat{\mu}(\cdot)$ appears in the denominator of the loss function, this is particularly problematic in the low frequency case.

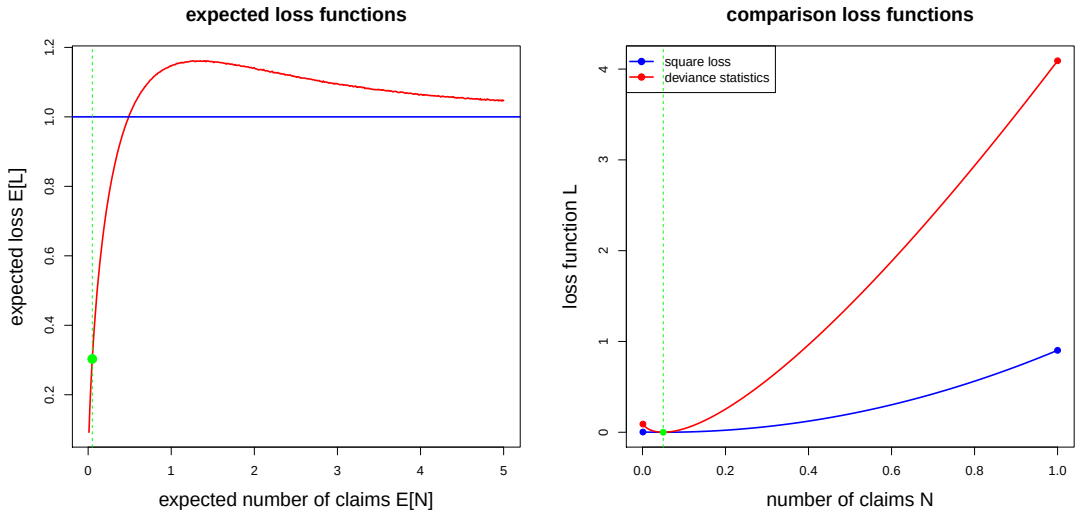


Figure 4: (lhs) expected deviance loss $\mathbb{E}[N] \mapsto 2\mathbb{E}[\mu(\boldsymbol{x}, v) - N - N\log(\mu(\boldsymbol{x}, v)/N)]$ (red color) and expected weighted square loss $\mathbb{E}[N] \mapsto \mathbb{E}[\varepsilon^2]$ (blue color) both as a function of $\mathbb{E}[N] = \mu(\boldsymbol{x}, v)$; (rhs) deviance loss and square loss as a function of the observed number of claims $N \in \{0, 1\}$ (red and blue dots) for $\mathbb{E}[N] = \mu(\boldsymbol{x}, v) = 5\%$.

In Figure 4 (lhs) we plot in red color the expected Poisson deviance loss

$$2\mathbb{E}\left[\mu(\boldsymbol{x}, v) - N - N\log(\mu(\boldsymbol{x}, v)/N)\right] = -2\mu(\boldsymbol{x}, v)\log\mu(\boldsymbol{x}, v) + 2\mathbb{E}[N\log N], \tag{3.3}$$

as a function of the expected number of claims $\mathbb{E}[N] = \mu(\boldsymbol{x}, v)$ of a Poisson random variable $N$. This expected deviance loss is around $30.3 \cdot 10^{-2}$ for an expected value of $\mathbb{E}[N] = 5\%$ (see green

illustration in Figure 4 (lhs)). This order of magnitude is in line with Table 2. The blue line in Figure 4 (lhs) illustrates the expected loss of the weighted square loss function given by

$$\mathbb{E}[\varepsilon^2] = \mathbb{E}\left[(N-\mu(\boldsymbol{x},v))^2/\mu(\boldsymbol{x},v)\right] = 1,$$

which of course is equal to 1 under the corresponding Poisson assumption.[4]
In Figure 4 (rhs) we plot the deviance loss and the weighted square loss function

$$N \mapsto 2\left[\mu(\boldsymbol{x},v) - N - N\log\left(\frac{\mu(\boldsymbol{x},v)}{N}\right)\right] \qquad \text{and} \qquad N \mapsto \varepsilon^2 = \frac{(N-\mu(\boldsymbol{x},v))^2}{\mu(\boldsymbol{x},v)},$$

for a given expected number of claims of $\mathbb{E}[N] = \mu(\boldsymbol{x},v) = 5\%$. Note that $N \in \{0,1,2,\ldots\}$ can only take integer values (whereas the red and blue lines in Figure 4 (rhs) illustrate $N \in [0,1]$). The green line in Figure 4 (rhs) gives the expected number of claims. A realization $N=0$ only contributes a small amount to the loss, and $N=1$ contributes hugely to the loss (more pronounced for the deviance loss). On the other hand, the sensitivity in a slight change of $\mu(\boldsymbol{x},v)$ has a comparably small influence, as can be seen from Figure 4 (rhs). This shows that the loss functions will largely be dominated by pure randomness in the realizations of $N$, and slight modifications in the model $\mu(\boldsymbol{x},v)$ can only hardly be detected. This is a common problem in low frequency problems (and also relates to the class imbalance problem in machine learning making low frequency prediction problems a very difficult task, see also next section).

**Remark.** In view of (3.3) we can minimize the expected deviance loss w.r.t. to the unknown parameter $\mu$ of $N$, i.e. we may consider

$$\widehat{\mu}^* = \arg\min_{\widehat{\mu}} \mathcal{R}(\mu|\widehat{\mu}) = \arg\min_{\widehat{\mu}} 2\mathbb{E}\left[\widehat{\mu} - N - N\log(\widehat{\mu}/N)\right].$$

In Gneiting [10] this minimizer is called the optimal point forecast for $N$ under the Poisson deviance scoring function. This minimizer is given by $\widehat{\mu}^* = \mathbb{E}[N]$ which implies that this scoring function is (strictly) consistent for the expected value, see Theorem 2.2 in Gneiting [10].

### 3.3 Binary loss functions

Often, claim frequencies are very small in insurance. In Table 4 we provide the policies with the

| number of claims $N_i$ | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 8 | 9 | 11 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| number of policies | 643'953 | 32'178 | 1'784 | 82 | 7 | 2 | 1 | 1 | 1 | 3 | 1 |
| total exposures $v_i$ | 336'616 | 20'671 | 1'153 | 53 | 3 | 1 | 0.3 | 0.4 | 0.1 | 1.1 | 0.3 |

Table 4: split of the portfolio w.r.t. number of claims.

corresponding numbers of claims. We observe that more than 90% of the policies do not suffer a claim. For this reason, one often speaks about a class imbalance because the outcome $N_i = 0$ is by far the most common one. Since the event $\{N_i > 1\}$ is even much less likely than the

[4] Remark that Figure 4 (lhs) exactly illustrates the expected dispersion estimates (deviance (red) and Pearson's (blue)) of the Poisson model, see Section 7.3.3 in [34].

event $\{N_i = 1\}$, one is tempted to replace the Poisson problem by a binomial problem (binary classification problem). This may come at the *loss* of *some* information. We replace $N_i$ by

$$Y_i = \mathbb{1}_{\{N_i \geq 1\}}. \tag{3.4}$$

$Y_i$ has a binomial distribution under model assumption (1.2) with success probability

$$p(\boldsymbol{x}_i, v_i) = \mathbb{P}[Y_i = 1] = 1 - \mathbb{P}[Y_i = 0] = 1 - \mathbb{P}[N_i = 0] = 1 - \exp(-\mu(\boldsymbol{x}_i, v_i)).$$

We can now apply binary classification to estimate $\widehat{p}(\boldsymbol{x}_i, v_i)$ which in turn provides estimator

$$\widehat{\mu}(\boldsymbol{x}_i, v_i) = -\log(1 - \widehat{p}(\boldsymbol{x}_i, v_i)). \tag{3.5}$$

The binomial model proposes the deviance loss function

$$\begin{aligned}\mathcal{L}(\mathcal{D}, \widehat{p}(\cdot)) &= -\frac{2}{n}\sum_{i=1}^{n} Y_i \log(\widehat{p}(\boldsymbol{x}_i, v_i)) + (1 - Y_i)\log(1 - \widehat{p}(\boldsymbol{x}_i, v_i)) \\ &= -\frac{2}{n}\sum_{i=1}^{n} Y_i \log\left(\frac{\widehat{p}(\boldsymbol{x}_i, v_i)}{1 - \widehat{p}(\boldsymbol{x}_i, v_i)}\right) + \log(1 - \widehat{p}(\boldsymbol{x}_i, v_i)).\end{aligned}$$

Observe that the first term under the sum on the second line is the logit transform of $\widehat{p}(\boldsymbol{x}_i, v_i)$. The first line gives the cross-entropy or Kullback-Leibler loss if we calculate its expected value w.r.t. $Y_i$, i.e. the Kullback-Leibler loss of $\widehat{p}(\boldsymbol{x}_i, v_i)$ w.r.t. $p(\boldsymbol{x}_i, v_i)$ is given by

$$\mathcal{R}(p|\widehat{p}) = \frac{1}{2}\mathbb{E}_Y[\mathcal{L}(\mathcal{D}, \widehat{p}(\cdot))] = -\frac{1}{n}\sum_{i=1}^{n} p(\boldsymbol{x}_i, v_i)\log(\widehat{p}(\boldsymbol{x}_i, v_i)) + (1 - p(\boldsymbol{x}_i, v_i))\log(1 - \widehat{p}(\boldsymbol{x}_i, v_i)),$$

where the expected value operator $\mathbb{E}_Y$ is applied to $Y_i$. The square loss function in the binomial model reads as

$$\mathcal{L}(\mathcal{D}, \widehat{p}(\cdot)) = \frac{1}{n}\sum_{i=1}^{n}(\widehat{p}(\boldsymbol{x}_i, v_i) - Y_i)^2 = \frac{1}{n}\sum_{i=1}^{n} Y_i(1 - \widehat{p}(\boldsymbol{x}_i, v_i))^2 + (1 - Y_i)\widehat{p}(\boldsymbol{x}_i, v_i)^2,$$

where for the last identity we use $Y_i \in \{0, 1\}$. This provides expected square loss w.r.t. $Y_i$

$$\mathcal{R}(p|\widehat{p}) = \mathbb{E}_Y[\mathcal{L}(\mathcal{D}, \widehat{p}(\cdot))] = \frac{1}{n}\sum_{i=1}^{n} p(\boldsymbol{x}_i, v_i)(1 - \widehat{p}(\boldsymbol{x}_i, v_i))^2 + (1 - p(\boldsymbol{x}_i, v_i))\widehat{p}(\boldsymbol{x}_i, v_i)^2.$$

If we minimize these expected losses, say for one risk only, we obtain

$$\mathcal{I}(p) = \min_{\widehat{p}} \mathcal{R}(p|\widehat{p}) = \begin{cases} -p\log p - (1-p)\log(1-p) & \text{entropy impurity function,} \\ p(1-p) & \text{Gini impurity function.} \end{cases}$$

Thus, the deviance loss and the square loss are directly related to the entropy and the Gini impurity functions, respectively, we also refer to Figure 6.7 in [37] on binary classification trees. Moreover, these two loss functions have minimizer $\widehat{p}^* = p = \mathbb{E}_Y[Y]$, i.e. they are (strictly) consistent for the expected value.

### 3.4 Conclusion of Sections 3.1-3.3 on the choice of the loss function

The detour via binary classification for an imbalanced Poisson prediction problem may have another disadvantage besides a potential loss of information due to (3.4). Namely, it may induce a bias because, using Jensen's inequality, we obtain from identity (3.5)

$$\mathbb{E}\left[\widehat{\mu}(\boldsymbol{x}_i, v_i)\right] \geq -\log\left(1 - \mathbb{E}\left[\widehat{p}(\boldsymbol{x}_i, v_i)\right]\right).$$

Thus, if we have an unbiased estimate for $\widehat{p}(\boldsymbol{x}_i, v_i)$ it will result in a biased estimate for $\widehat{\mu}(\boldsymbol{x}_i, v_i)$. For these reasons we work with the Poisson deviance loss (3.1) as objective function. Model calibration is then done by making this deviance loss function small w.r.t. the estimator $\widehat{\mu}(\cdot)$ on the given data $\mathcal{D}$ (*learning data*). Since the resulting *in-sample loss* (3.1) is prone to over-fitting, we typically evaluate the quality of the fit by calculating an *out-of-sample loss* on *test data* $\mathcal{T} = \{(N_t, \boldsymbol{x}_t, v_t) : t = 1, \ldots, n_{\mathcal{T}}\}$ that is disjoint from $\mathcal{D}$:

$$\mathcal{L}(\mathcal{T}, \widehat{\mu}(\cdot)) \;=\; \frac{1}{n_{\mathcal{T}}} \sum_{t=1}^{n_{\mathcal{T}}} 2N_t \left[\frac{\widehat{\mu}(\boldsymbol{x}_t, v_t)}{N_t} - 1 - \log\left(\frac{\widehat{\mu}(\boldsymbol{x}_t, v_t)}{N_t}\right)\right]. \tag{3.6}$$

In machine learning parlance we say that we analyze how the model fitted on data $\mathcal{D}$ *generalizes* to before unseen data $\mathcal{T}$. In all our analysis we use exactly the two data sets constructed in Section 2 of Noll et al. [22]. The learning data $\mathcal{D}$ has $n = 610'212$ policies and the test data $\mathcal{T}$ has $n_{\mathcal{T}} = 67'801$ policies, we also refer to Table 3 in Noll et al. [22].[5] A first illustrative example has already been provided in Table 2.

## 4 Optimization algorithms: item (f)

In the remainder of this manuscript we consider different network models (network architectures). We fit these models to the learning data $\mathcal{D}$ by making the in-sample losses $\mathcal{L}(\mathcal{D}, \widehat{\mu}(\cdot))$, given in (3.1), small. We assess the quality of these fits (generalization capability) by considering the corresponding out-of-sample losses $\mathcal{L}(\mathcal{T}, \widehat{\mu}(\cdot))$, given in (3.6), on the test data $\mathcal{T}$.
A first naïve approach would aim at minimizing $\mathcal{L}(\mathcal{D}, \widehat{\mu}(\cdot))$ in $\widehat{\mu}(\cdot)$, this would provide the MLE for the corresponding Poisson network regression model. This approach is naïve because, typically, networks are over-parametrized and the MLE would heavily over-fit to the data $\mathcal{D}$. This would lead to a poor out-of-sample performance on $\mathcal{T}$, because an over-fitted model does not generalize to other data as it also mimics the random/noisy part in $\mathcal{D}$. Therefore, in network calibrations, we are *not* interested in finding the MLE, but we would like to find a sufficiently good parametrization, which also has a good out-of-sample performance (generalization property). Having this said, it is clear that typically in network calibrations there is a lot of redundancy[6] which results in many competing predictive models of similar quality. We briefly explain this based on the example given in Figure 5.

[5] The partition has been generated under R version 3.5.0 (2018-04-23). In the upgrade from R version 3.5.x to R version 3.6.x the function `sample` which is used to generate this partition has been changed. For reproducability under the new version one should set `RNGversion(''3.5.0'')`.

[6] Recall that we try to fit a regression model to "noisy" observations which may lead to over-fitting if the regression function follows these noisy observations too closely. This is different from network approximations to (given) *deterministic functions* where we do not have the problem (and notion) of over-fitting, and where we try to fit the deterministic function as accurately as possible.

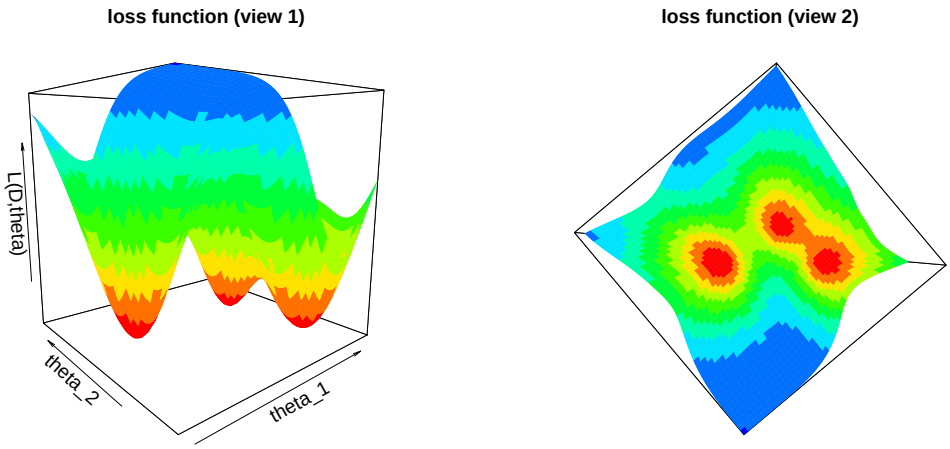


Figure 5: surface of the loss function $\theta \mapsto \mathcal{L}(\mathcal{D}, \theta)$ as a function of a two-dimensional network parameter $\theta = (\theta_1, \theta_2)'$: the two plots provide the loss surface from two different angles.

In Figure 5 we plot the surface of a loss function (objective function) $\theta \mapsto \mathcal{L}(\mathcal{D}, \theta)$ as a function of a two-dimensional parameter $\theta = (\theta_1, \theta_2)'$. The two graphs illustrate the same surface from two different angles. This objective function is assumed to have 3 different local minima (red color in plots), i.e. three extremal points for the choice of the parameter $\theta$. If we assume that the red and yellow colors indicate parameter choices $\theta$ that over-fit to the data $\mathcal{D}$ (very small in-sample loss), we would choose a network parameter $\theta$ in the green part of the surface. Having continuity in $\theta$ immediately tells us that there are infinitely many competing models with a similar performance (green area in the plots). Thus, choosing a sufficiently good parametrization means that we choose a network parameter $\theta$ providing a loss in the green area of Figure 5, and there does not exist a unique best choice, because we project a high-dimensional selection problem to the real line (via the objective function), and on the way there a lot of information is getting lost. This is best visible by looking at the models on individual policy level: on the global portfolio level the values of the objective function can be identical, nevertheless, there can be substantial differences on individual policy level. A similar consideration holds true if we compare different network architectures.

## 4.1 Stochastic gradient descent method

**Terminology: shallow and deep networks.**
Networks with one hidden layer $K = 1$ are called *shallow networks*, networks with multiple hidden layers $K > 1$ are called *deep networks*. A shallow and a deep network example are provided in Figure 3.

For illustration we choose a fixed network architecture consisting of a shallow network having $q_1 = 20$ hidden neurons in the single hidden layer, see Figure 3 (lhs). As activation function we choose the hyperbolic tangent. These choices provide exactly the regression function introduced

in (1.8). The choice of the network architecture is often called "selection of hyperparameters". For the moment, we assume that these hyperparameters are given, and we discuss model calibration for such a given network architecture. The choice of the hyperparameters is discussed in detail in Section 6, below.

The plain-vanilla method for network calibration is the gradient descent method (GDM), additionally using back-propagation for an efficient evaluation of the gradients. In GDMs, we study the in-sample loss $\mathcal{L}(\mathcal{D}, \mu(\cdot))$ as objective function in the network parameter $\theta$, that is, we consider

$$\theta \;\mapsto\; \mathcal{L}(\mathcal{D}, \mu_\theta(\cdot)), \tag{4.1}$$

where $\mu(\cdot) = \mu_\theta(\cdot)$ is the network regression function (1.8), and where the network parameter $\theta$ collects all the weights $\boldsymbol{w}^{(k)}$, $k = 1, \ldots, K+1$, see Section 1.3. The GDM looks iteratively for the direction of the maximal *local* decrease of the loss function (4.1) which is given by the negative gradient of $\mathcal{L}(\mathcal{D}, \mu_\theta(\cdot))$ w.r.t. $\theta$, i.e. for the Poisson deviance loss function we have

$$-\nabla_\theta \mathcal{L}(\mathcal{D}, \mu_\theta(\cdot)) \;=\; -\frac{1}{n}\sum_{i=1}^{n} 2\left[\mu_\theta(\boldsymbol{x}_i, v_i) - N_i\right] \; \nabla_\theta \log(\mu_\theta(\boldsymbol{x}_i, v_i)).$$

A small step $\varrho > 0$ (called *learning rate*) into the direction of the negative gradient, that is, an update $\theta \mapsto \widetilde{\theta} = \theta - \varrho \, \nabla_\theta \mathcal{L}(\mathcal{D}, \mu_\theta(\cdot))$, will lead to a (maximal local) decrease in loss of size

$$\mathcal{L}(\mathcal{D}, \mu_{\widetilde{\theta}}(\cdot)) \;=\; \mathcal{L}(\mathcal{D}, \mu_\theta(\cdot)) - \varrho \, \|\nabla_\theta \mathcal{L}(\mathcal{D}, \mu_\theta(\cdot))\|^2 + o(\varrho), \qquad \text{as } \varrho \downarrow 0.$$

This is the locally optimal move from $\theta$ to $\widetilde{\theta}$ in the network parameter. Back-propagation is then used to efficiently calculate these (negative) gradients, for more theory and explanation on the back-propagation method we refer to Nielsen [21]. There are different versions of this GDM that explore optimal learning rates $\varrho > 0$, momentum-based improvements, Nesterov acceleration, etc. All this variants of the GDM aim at speeding up the convergence of the algorithm by not only considering the last locally optimal move. We briefly mention some of them which are available in the library Keras,[7] for a more detailed description we refer to Sections 8.3 and 8.5 in Goodfellow et al. [11].

Listing 2: optimizer 'sgd'

```
optimizer_sgd(lr = 0.01, momentum = 0, decay = 0, nesterov = FALSE,
                                                   clipnorm = NULL, clipvalue = NULL)
```

**Predefined gradient descent methods.**

- The stochastic gradient descent method, called 'sgd',[8] can be fine-tuned for the speed of convergence by using optimal learning rates, momentum-based improvements, the Nesterov acceleration and optimal batches, see Listing 2; 'stochastic' gradient means that in contrast to (steepest) gradient descent, we explore locally optimally steps on random sub-samples

[7] Keras is a user-friendly API to TensorFlow, see https://tensorflow.rstudio.com/keras/; many results are sensitive in the versions chosen, we use Keras version 2.2.4, TensorFlow version 1.10.

[8] The letter 's' in 'sgd' stands for the stochastic choice of subsamples (batches) of a given size in the GDM.

(mini-batches) of the learning data $\mathcal{D}$, this is mainly motivated by computational reasons coming from big data;

- 'adagrad' chooses learning rates that differ in all directions of the gradient and that consider the directional sizes of the gradients ('ada' stands for adapted);
- 'adadelta' is a modified version of 'adagrad' that overcomes some deficiencies of the latter, for instance, the sensitivity to hyperparameters;
- 'rmsprop' is another method to overcome the deficiencies of 'adagrad' ('rmsprop' stands for root mean square propagation);
- 'adam' stands for adaptive moment estimation, similar to 'adagrad' it searches for directionally optimal learning rates based on the momentum induced by past gradients measured by an $\ell^2$-norm;
- 'adamax' considers optimal learning rates as 'adam' but based on the $\ell^\infty$-norm;
- 'nadam' is a Nesterov accelerated version of 'adam'.

Listing 3: R script for fitting networks in Keras

```
1  library(keras)
2
3  seed <- 100
4  set.seed(seed)
5  use_session_with_seed(seed)
6
7  model <- keras_model_sequential()
8  model %>%
9    layer_dense(units = q1, activation = 'tanh', input_shape = c(ncol(Xlearn))) %>%
10   layer_dense(units = 1, activation = 'exponential')
11
12 summary(model)
13
14 model %>% compile(
15   loss = 'poisson',
16   optimizer ='sgd'
17 )
18
19 fit <- model %>% fit(Xlearn, learn$ClaimNb, epochs=100, batch_size=10000)
```

In order to perform network calibration we use the R interface to Keras. The corresponding code is provided in Listing 3. On lines 3-7 we initialize our `model` using a TensorFlow backend.[9] On lines 8-10 we define a (fully-connected, feed-forward) shallow network with $q_1$ hidden neurons and hyperbolic tangent activation function, the output layer has exponential activation function, see also (1.8). Since the dimension of the feature space is $q_0 = 39$ we receive a $r = 821$-dimensional network parameter $\theta$ (this can be displayed by the command on line 12). Lines 14-17 compile the `model`, using the Poisson deviance loss function as objective function, finally on line 19 the `model` is fitted. For this fitting we only use the learning data $\mathcal{D}$ (encoded in the design matrix `Xlearn` and the responses `learn$ClaimNb`). `epochs` provides the number of times the entire

[9] see https://keras.io/backend/

learning data is run through the gradient descent algorithm. Since, typically, it is too costly to handle the entire learning data at once, the data is partitioned (randomly) into batches of size `batch_size` (stochastic gradient descent). Note that this partitioning of the data is of particular interest if we work with big data because it allows us to explore the data sequentially.

| optimizer | epochs | batch size | run time | in-sample loss | out-of-sample loss |
|---|---|---|---|---|---|
| 'sgd' | 200 | 10'000 | 95s | 31.36754 | 32.36219 |
| 'adagrad' | 200 | 10'000 | 92s | 30.41779 | 31.42018 |
| 'adadelta' | 200 | 10'000 | 92s | 30.16726 | 31.17942 |
| 'rmsprop' | 200 | 10'000 | 100s | 29.91239 | 30.85548 |
| 'adam' | 200 | 10'000 | 99s | 29.80492 | 30.85164 |
| 'adamax' | 200 | 10'000 | 92s | 30.30699 | 31.34296 |
| 'nadam' | 200 | 10'000 | 93s | 29.53307 | 30.65416 |

Table 5: results of the GDM illustrated in Listing 3 for different optimizers, always using the same initial parameter for $\theta$; shallow network with $q_1 = 20$; losses are in $10^{-2}$.

We run the GDM provided in Listing 3 for the different optimizers introduced above. For each optimizer we use the same initial parameter for $\theta$, the same batch size and the same number of epochs. The results are given in Table 5. We note that the improved versions of the GDM, like 'nadam' provide clearly better convergence results than the (plain-vanilla) 'sgd' of Listing 2. This is also supported by Figure 6 that shows the decrease of loss during the gradient descent

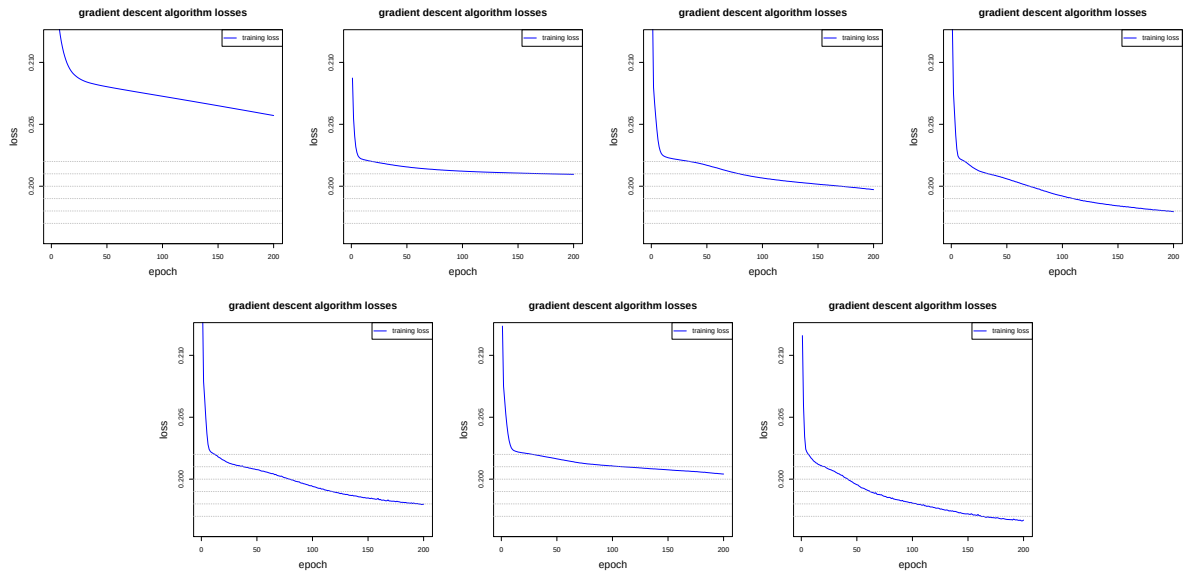


Figure 6: illustration of the GDM (losses over the different epochs) using the different optimizers (top) 'sgd', 'adagrad', 'adadelta', 'rmsprop', (bottom) 'adam', adamax' and 'nadam', see also Table 5; the $y$-scale is the same in all graphs.

algorithm for each optimizer separately. We have received consistent results under other choices of initial parameters. Since fine-tuning the 'sgd' for learning rates, etc., is too time-consuming we continue with the pre-specified optimizer 'nadam' which outperforms the others in the analysis

of Table 5. From this table we see that 200 epochs on batches of size 10'000 take roughly 100 seconds on our learning data using a personal laptop Intel(R) Core(TM) i7-8550U CPU @ 1.80GHz 1.99GHz with 16GB RAM (which has a comparably modest computational power). We also observe that the results are not competitive, yet, with the ones in Table 2, in fact, for the latter table we have run the algorithm for 1000 epochs (on the same batch size).

**Remark.** Note that in Table 5 we provide the Poisson deviance losses which are of magnitude $30 \cdot 10^{-2}$. The $y$-axis in Figure 6 is on a different scale, because the standard version in Keras of the Poisson deviance loss function drops all terms that do not involve the network parameter and it does not scale with constant 2, i.e. the Poisson deviance loss (3.1) is replaced by objective function

$$\mathcal{L}^{\text{Keras}}(\mathcal{D}, \widehat{\mu}(\cdot)) \;=\; \frac{1}{n}\sum_{i=1}^{n} \widehat{\mu}(\boldsymbol{x}_i, v_i) - N_i \log \widehat{\mu}(\boldsymbol{x}_i, v_i).$$

## 4.2 Comparison of network calibrations

We analyze the network calibrations obtained from the different optimizers presented in Table 5. They have all been starting from the same initial value, but they result in different in-sample and out-of-sample losses, this can also be seen from Figure 6.
Our next aim is to compare the resulting network parameters $\theta \in \mathbb{R}^r$ from the different optimizers. To do so, we first need to transform them because, in general, network parametrizations are not uniquely identifiable. If we have an anti-symmetric activation function $-\phi(x) = \phi(-x)$ we can perform the following sign switch operations in regression function (1.8):

$$\begin{aligned}\log \mu(\boldsymbol{x}, v) \;&=\; w_0^{(2)} + \sum_{j=1}^{q_1} w_j^{(2)} \phi\left(w_{j,0}^{(1)} + \sum_{l=1}^{q_0-1} w_{j,l}^{(1)} x_l + w_{j,q_0}^{(1)} v\right) \qquad (4.2)\\ &=\; w_0^{(2)} + \sum_{j\neq k} w_j^{(2)} \phi\left(w_{j,0}^{(1)} + \sum_{l=1}^{q_0-1} w_{j,l}^{(1)} x_l + w_{j,q_0}^{(1)} v\right) - w_k^{(2)} \phi\left(-w_{k,0}^{(1)} - \sum_{l=1}^{q_0-1} w_{k,l}^{(1)} x_l - w_{k,q_0}^{(1)} v\right).\end{aligned}$$

Thus, the two network parameters

$$\begin{aligned}\theta \;&=\; (w_{1,0}^{(1)}, \ldots, w_{k,l}^{(1)}, \ldots, w_{q_1,q_0}^{(1)}, w_0^{(2)}, \ldots, w_k^{(2)}, \ldots, w_{q_1}^{(2)})' \qquad \text{and}\\ \theta^- \;&=\; (w_{1,0}^{(1)}, \ldots, -w_{k,l}^{(1)}, \ldots, w_{q_1,q_0}^{(1)}, w_0^{(2)}, \ldots, -w_k^{(2)}, \ldots, w_{q_1}^{(2)})'\end{aligned}$$

give the same prediction (where we switch all signs that belong to index $k$). Secondly, enumeration of the hidden neurons may be permuted, still providing the same prediction. We solve this identifiability issue for anti-symmetric activation functions by considering a fundamental domain as introduced by Rüger–Ossen [30]. For a general network parameter $\theta$, its fundamental version is constructed by the algorithm presented in Rüger–Ossen [30], after Theorem 2: We consider the weights $\boldsymbol{w}^{(1)}$ from the input $(\boldsymbol{x}, v)$ to the first hidden layer $\boldsymbol{z}^{(1)}(\boldsymbol{x}, v)$ and we apply a sign switch operation (similar to (4.2)) so that all intercepts $w_{1,0}^{(1)}, \ldots, w_{q_1,0}^{(1)}$ are positive while letting the regression function $(\boldsymbol{x}, v) \mapsto \mu(\boldsymbol{x}, v)$ being unchanged. Then, we apply a permutation operation to the indexes $j = 1, \ldots, q_1$ so that we arrive at (an order statistics)

$$0 < w_{1,0}^{(1)} < \ldots < w_{q_1,0}^{(1)}, \qquad (4.3)$$

for unchanged regression function $(\boldsymbol{x}, v) \mapsto \mu(\boldsymbol{x}, v)$. Note that this requires that all intercepts are non-zero and different from each other (which we assume for the moment). Then, we move iteratively through the hidden layers $k = 2, \ldots, K$, see Section 1.3. That is, we apply the above sign switch operation and the above permutation so that the regression function $(\boldsymbol{x}, v) \mapsto \mu(\boldsymbol{x}, v)$ does not change and such that for all hidden layers $k = 2, \ldots, K$ we have ordered intercepts

$$0 < w_{1,0}^{(k)} < \ldots < w_{q_k,0}^{(k)}.$$

Thus, every network parameter $\theta \in \mathbb{R}^r$ (satisfying the previous properties) has a unique representative (obtained by the algorithm above) in the fundamental domain

$$\boldsymbol{\Theta}^+ = \left\{ \theta \in \mathbb{R}^r;\ 0 < w_{1,0}^{(k)} < \ldots < w_{q_k,0}^{(k)}, \quad \text{for all } k = 1, \ldots, K \right\} \ \subset\ \mathbb{R}^r. \tag{4.4}$$

There may still exist some redundancies in special cases, for instance, if the outgoing weights of a given neuron are equal to 0. However, as mentioned in Rüger–Ossen [30], Section 2.2, these symmetries are of zero Lebesgue measure (for hyperbolic tangent activation), and will therefore be neglected for our purposes. Moreover, working on (reasonable) real observations will also imply that intercepts are different from 0 and different from each other. Thus, w.l.o.g., we may and will assume (after transformation) that the calibrated network parameter $\theta$ lies in the fundamental domain $\boldsymbol{\Theta}^+$.

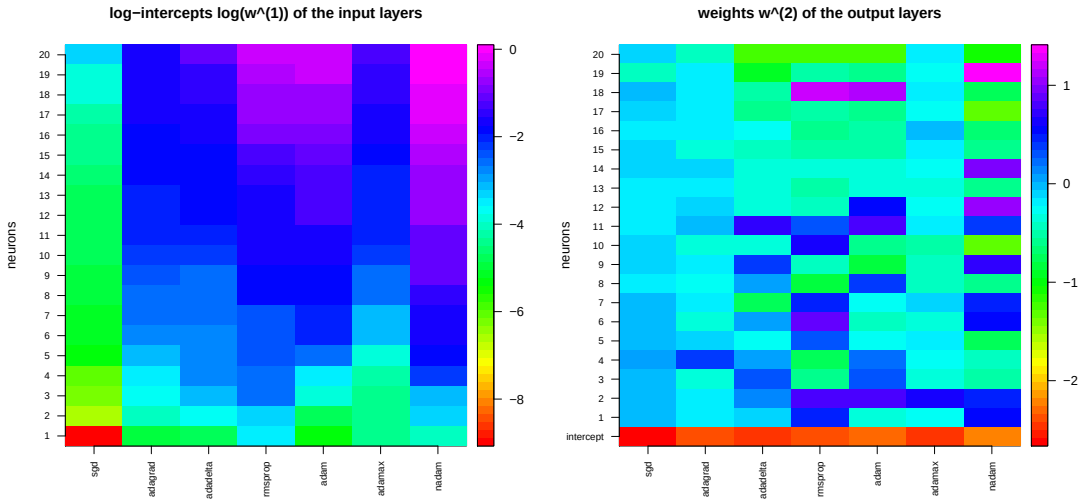


Figure 7: comparison of the calibrations obtained by the optimizers 'sgd', 'adagrad', 'adadelta', 'rmsprop', 'adam', adamax' and 'nadam': (lhs) intercepts $0 < w_{1,0}^{(1)} < \ldots < w_{q_1,0}^{(1)}$ of the inputs (on the log-scale); (rhs) output weights $\boldsymbol{w}^{(2)} = (w_j^{(2)})_{j=0,\ldots,q_1}$.

We then transform all network parameters obtained in Table 5 such that they lie in the fundamental domain $\boldsymbol{\Theta}^+$. In Figure 7 (lhs) we illustrated the intercepts of the input weights $(w_{j,0}^{(1)})_{j=1,\ldots,q_1}$ which are positive and ordered by construction, see (4.3). Note that in Figure 7 (lhs) we plot these intercepts on the log-scale. A short inspection of the figure proves that there are substantial differences between the calibrations. In general, the better calibrations like 'rmsprop', 'adam' and 'nadam' take more extreme parameter values, whereas the calibration of

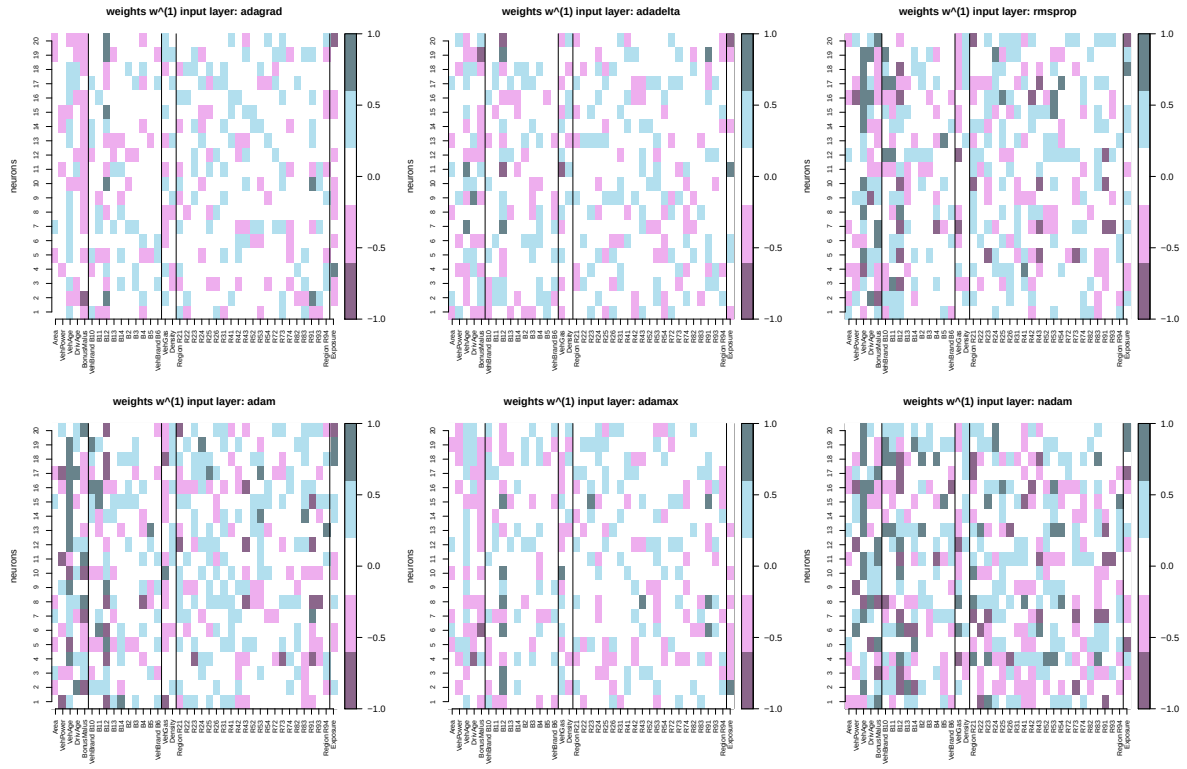


Figure 8: comparison of the calibrations obtained by the optimizers 'adagrad', 'adadelta', 'rmsprop', 'adam', adamax' and 'nadam': input weights $(w^{(1)}_{j,l})_{j=1,\dots,q_1;l=1,\dots,q_0}$.

'sgd' seems to be quite flat. This indicates that the former optimizers lead to faster convergence than plain-vanilla stochastic gradient descent methods, see also Figure 6.
A similar picture is obtained for the output weights $\boldsymbol{w}^{(2)} = (w^{(2)}_j)_{j=0,\dots,q_1}$, see Figure 7 (rhs). The intercept $w^{(2)}_0$ is of magnitude $-2.5$ but the remaining weights take more extreme values for the better calibrations, see $w^{(2)}_{19}$ for 'nadam' and $w^{(2)}_{18}$ for 'rmsprop' and 'adam', respectively. Figure 8 illustrates the input weights $(w^{(1)}_{j,l})_{j=1,\dots,q_1,l=1,\dots,q_0}$. The `Exposure` variable $v$ corresponds to the last columns $(w^{(1)}_{j,q_0})_{j=1,\dots,q_1}$ in the plots of Figure 8. These exposures take extreme weights in 'nadam', 'rmsprop' and 'adam' exactly in the neurons where the output weights $w^{(2)}_j$ have large values. This illustrates the importance of the exposure variables in our predictive model.
Figures 7 and 8 allow us to start interpreting the data and network calibrations. If we focus on the 'nadam' calibration, we see a couple of extreme colors in the input weights $w^{(1)}_{j,l}$, for instance, for $l = 3$ corresponding to `VehAge`, see column 3 of Figure 8 (bottom-right). This tells us something about the importance of certain variables. We can also study interactions of `VehAge` with other variables by studying rows in Figure 8 (bottom-right), corresponding to the neurons $j = 1, \dots, q_1$. Moreover, Figure 7 (rhs) tells us how these neuron activations are transmitted to the output.

### 4.3 Epochs, batches and computational time

In this subsection we analyze the computational time and the predictive performance obtained by choosing different batch sizes in the GDM, see line 15 of Listing 3. `epochs` indicates how many times we go through the entire learning data $\mathcal{D}$, and `batch_size` indicates the size of the subsamples considered in each GDM step. Thus, if the batch size is equal to the number of observations $n$ we do exactly one GDM step in one epoch, if the batch size is equal to 1 then we do $n$ GDM steps in one epoch until we have seen the entire learning data $\mathcal{D}$. Note that smaller batches are needed for big data because it is not feasible to simultaneously calculate the gradient on all data efficiently if we have many observations. Therefore, we partition the entire data at random into (mini-) batches in the application of the GDM.

| optimizer | epochs | batch size | GDM steps per epoch | run time | av. time per GDM step | in-sample loss | out-of-sample loss |
|---|---|---|---|---|---|---|---|
| 'nadam' | 100 | 610'212 | 1 | 51s | 0.5114s | 31.26300 | 32.22074 |
| 'nadam' | 100 | 122'043 | 5 | 48s | 0.0959s | 30.43280 | 31.42829 |
| 'nadam' | 100 | 61'022 | 10 | 46s | 0.0455s | 30.24270 | 31.27183 |
| 'nadam' | 100 | 12'205 | 50 | 45s | 0.0091s | 29.73265 | 30.79400 |
| 'nadam' | 100 | 6'103 | 100 | 48s | 0.0048s | 29.58644 | 30.68083 |
| 'nadam' | 100 | 1'221 | 500 | 85s | 0.0017s | 29.44668 | 30.57108 |
| 'nadam' | 100 | 611 | 1'000 | 134s | 0.0013s | 29.45842 | 30.63919 |
| 'nadam' | 100 | 123 | 5'000 | 520s | 0.0010s | 29.44405 | 30.63011 |

Table 6: batch sizes, run times, GDM steps and losses; shallow network with $q_1 = 20$, see also Figures 9 and 10 (lhs); losses are in $10^{-2}$.

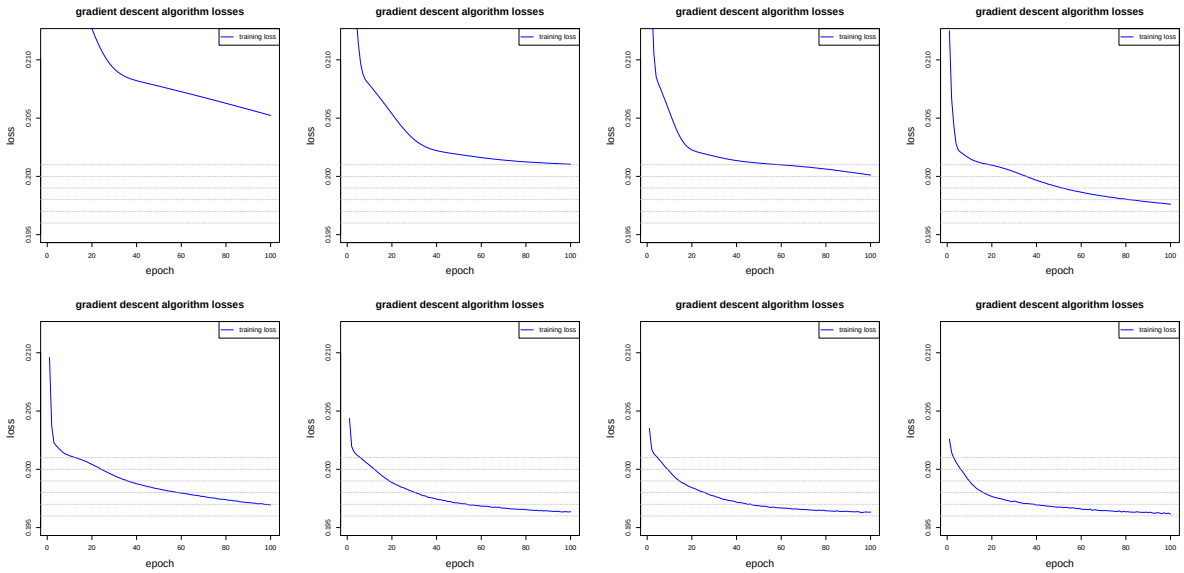


Figure 9: illustration of the GDM (losses over the different epochs) using the batch sizes of Table 6 and 100 epochs; the $y$-scale is the same in all graphs.

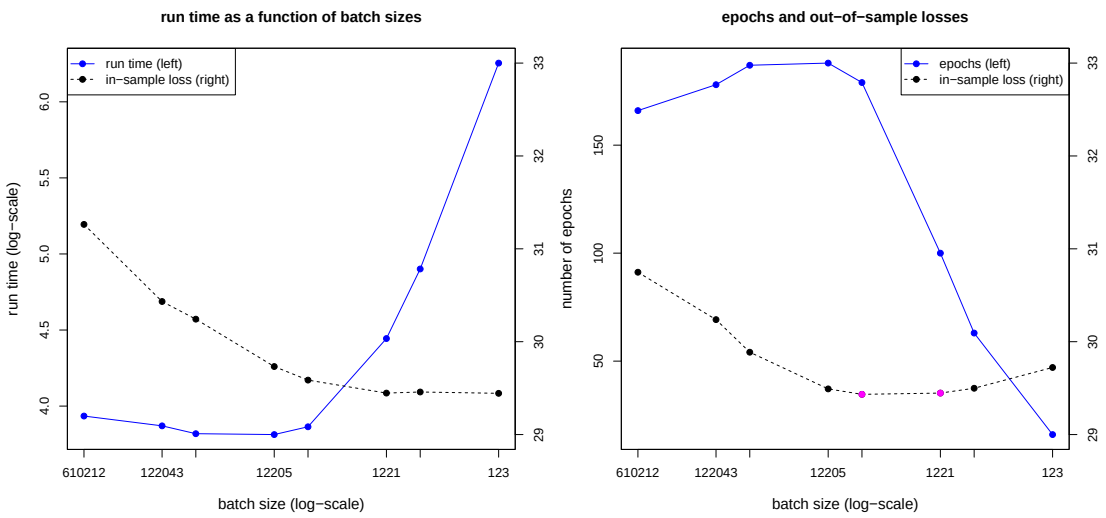


Figure 10: (lhs) illustration of run time and out-of-sample losses of Table 6; (rhs) illustration of epochs and out-of-sample losses of Table 7 for constant run time; losses are in $10^{-2}$.

In Table 6 and Figures 9-10 (lhs) we compare the results for different batch sizes (using the optimizer 'nadam' and always having the same initial parameter for $\theta$). For the maximal batch size $n = 610'212$ we can do exactly one GDM step in one epoch, for batch size 123 we can do 5'000 GDM steps in one epoch. For the maximal batch size we need to calculate the gradient on the entire data $\mathcal{D}$ which takes 0.5114s, for batch size 123 we calculate the gradient on 123 policies taking in average 0.0010s. Thus, the latter is, of course, much faster but on the other hand we need to calculate 5'000 gradients to run through the entire data (in an epoch). This results in 5.2042s per epoch, thus, 10 times longer than for the maximal batch size, see Figure 10 (lhs), blue curve. That is, for the total run time we obtain a trade-off between the batch size and the number of GDM steps we need to perform in order to screen the whole data in an epoch. In our setup, the minimal run time of 45s (for 100 epochs) is achieved for a batch size of 12'205.

These run times should be contrasted with the quality of the fits, in all set-ups of Table 6 we screen the entire data 100 times (number of epochs), the corresponding loss developments over the 100 epochs in each set-up are illustrated in Figure 9. The best out-of-sample performance for 100 epochs is achieved by a batch size of 1'221 which corresponds to 500 GDM steps per epoch (and a total run time of 85s), see also Figure 10 (lhs). Here, we have a trade-off between the number of GDM steps (smaller batch sizes) and the law of large numbers (bigger batch sizes). If the batch size is too small, then we may consider too many batches that are not well-balanced, and hence we move too often into a direction that is not sufficiently relevant for the entire data. The art of calibration then is to fine-tune batch size, run time and performance.

In Table 7 and Figures 11-10 (rhs) we provide a similar analysis as in the previous table, but this time we try to keep the total run time constant (roughly 90s) by adjusting the number of epochs accordingly. From the table and the figures we see that the optimal batch size for our network architecture (in terms of run time versus out-of-sample loss) is roughly 6'000. Note that this is similar to the batch size of 10'000 used in the analysis of Table 5. For a homogeneous portfolio

| optimizer | epochs | batch size | GDM steps per epoch | run time | av. time per GDM step | in-sample loss | out-of-sample loss |
|---|---|---|---|---|---|---|---|
| 'nadam' | 166 | 610'212 | 1 | 85s | 0.5148s | 30.74804 | 31.69301 |
| 'nadam' | 178 | 122'043 | 5 | 86s | 0.0968s | 30.23769 | 31.25951 |
| 'nadam' | 187 | 61'022 | 10 | 86s | 0.0458s | 29.88677 | 30.98261 |
| 'nadam' | 188 | 12'205 | 50 | 85s | 0.0091s | 29.49177 | 30.61715 |
| 'nadam' | 179 | 6'103 | 100 | 90s | 0.0051s | 29.43319 | 30.56317 |
| 'nadam' | 100 | 1'221 | 500 | 89s | 0.0018s | 29.44668 | 30.57108 |
| 'nadam' | 63 | 611 | 1'000 | 90s | 0.0014s | 29.49870 | 30.63575 |
| 'nadam' | 16 | 123 | 5'000 | 86s | 0.0011s | 29.72239 | 30.73217 |

Table 7: batch sizes, run times, GDM steps and losses; shallow network with $q_1 = 20$, see also Figures 11 and 10 (rhs); losses are in $10^{-2}$.

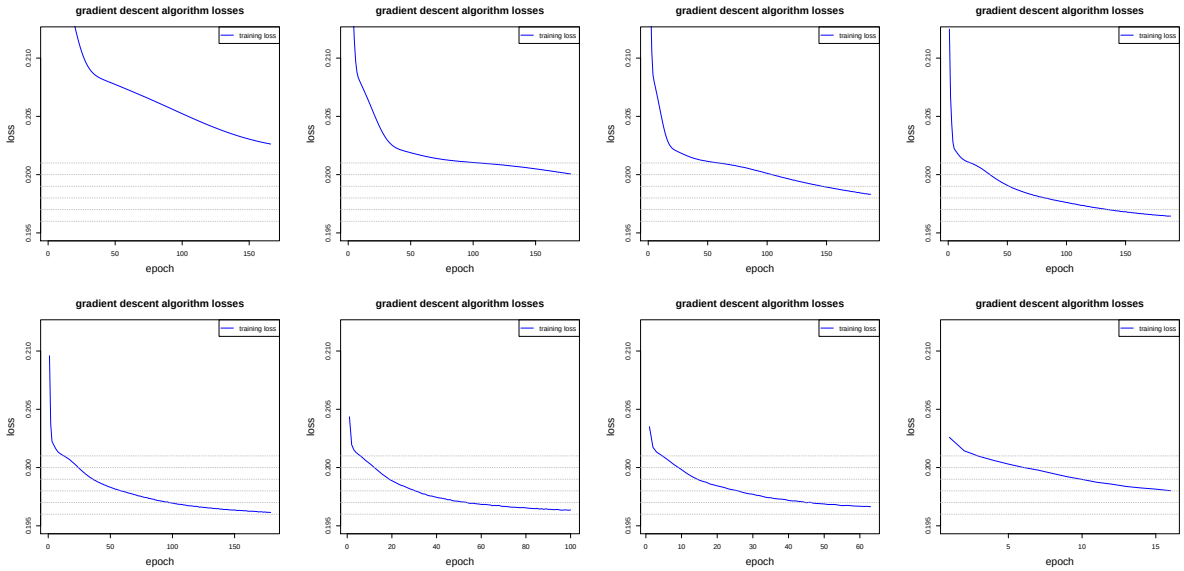


Figure 11: illustration of the GDM (losses over the different epochs) using the batch sizes and number of epochs of Table 7; the $y$-scale is the same in all graphs.

with $N_i \overset{\text{i.i.d.}}{\sim} \text{Poi}(\mu = 5\%)$ we obtain for batch size 6'000 confidence bounds of two standard deviations given by

$$\mu \pm 2 \cdot \sqrt{\frac{\mu}{6'000}} \;=\; 5\% \pm 0.58\%, \tag{4.5}$$

i.e. a precision of roughly 10% relative to the parameter $\mu$ to be estimated. This precision seems to be good in many situations, but of course, in general, this will depend on the complexity of the network architecture, on the heterogeneity of the underlying portfolio and on the level of $\mu$ (class imbalances in 0 will require higher batch sizes).

## 4.4 Stratified batches and under/over-sampling

### 4.4.1 Stratified batches

On line 15 of Listing 3 we specify the `batch_size` and the number of `epochs`: for the stochastic gradient descent steps we partition (at random) all observations $\mathcal{D}$ into batches of size `batch_size`. In one `epoch` we consider all batches once, and hence each case $(N_i, \boldsymbol{x}_i, v_i)$ in the observations is seen exactly once in an epoch. The partitioning of the data $\mathcal{D}$ into batches is

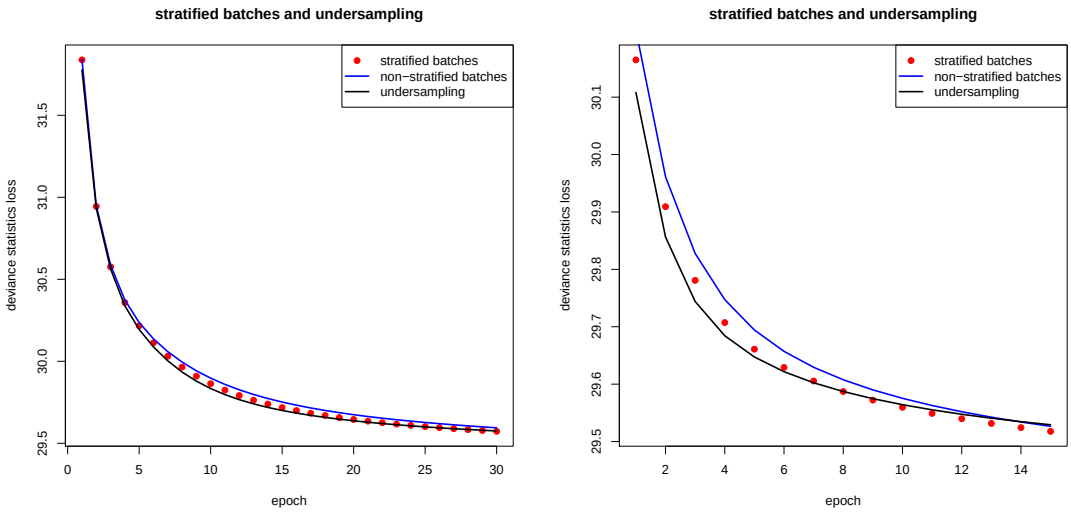


Figure 12: stratified batches and under/over-sampling: (lhs) `batch_size` $= 10'000$ with 30 epochs, and (rhs) `batch_size` $= 1'000$ with 15 epochs; for under/over-sampling we use factor 2.

done at random, and it may happen that several potential outliers lie in the same batch. This may imply that some steps of the 'sgd' algorithm deteriorate too much into a wrong direction (because the chosen batch is not a typical observation). This happens especially if the chosen batch size is small and the expected frequency is low (class imbalance problem), the latter providing rates of convergence of magnitude $\sqrt{\mu/\texttt{batch\_size}}$, see also (4.5). In statistics and

| batch type | epochs | batch size | GDM steps per epoch | run time | in-sample loss | out-of-sample loss |
|---|---|---|---|---|---|---|
| non-stratified | 30 | 10'000 | 62 | 88s | 29.59458 | 30.76872 |
| stratified | 30 | 10'000 | 62 | 88s | 29.57311 | 30.75321 |
| under/over-sampling | 30 | 10'000 | 65 | 92s | 29.57528 | 30.76261 |
| non-stratified | 15 | 1'000 | 611 | 53s | 29.52651 | 30.69125 |
| stratified | 15 | 1'000 | 611 | 54s | 29.51779 | 30.69621 |
| under/over-sampling | 15 | 1'000 | 641 | 56s | 29.52924 | 30.71172 |

Table 8: momentum-based stochastic gradient descent 'sgd' method for stratified batches and with under/over-sampling; for under/over-sampling we use factor 2; shallow network with $q_1 = 20$; losses are in $10^{-2}$.

in machine learning (especially in cross-validation) this difficulty is taken care of by choosing

a stratified partition of the data $\mathcal{D}$, see also Section 1.4.2-1.4.5 in [37]. In our case, the partition may be chosen such that the cases $(N_i, \boldsymbol{x}_i, v_i)$ with $N_i = 0$ are equally distributed among the batches, and likewise for the observations $N_i > 0$ among the batches. This is exactly the idea behind considering a *stratified* partition of the data $\mathcal{D}$. Often this leads to better rates of convergence (because batches resemble more typical observations). In Figure 12 we compare the stratified case (red dots) to the non-stratified version (blue line) for batches of sizes 10'000 (lhs) and 1'000 (rhs), respectively. In both cases we see that the stratified version has better convergence properties. This is also illustrated in the in-sample losses provided in Table 8.[10]

### 4.4.2 Under/over-sampling

A second method to improve rates of convergence in low frequency problems is to consider so-called under-sampling (or over-sampling). A batch of size, say, 1'000 should roughly have 100 cases $(N_i, \boldsymbol{x}_i, v_i)$ with $N_i > 0$ and 900 cases with $N_i = 0$ for an empirical frequency of 10% (for volumes $v \equiv 1$). The idea behind under/over-sampling is to take too many cases with $N_i > 0$ into a batch, but compensating this over-weighting of claims by increasing the weight of the cases with $N_i = 0$. For instance, if we double the cases with $N_i > 0$ in a batch, then we only receive roughly 800 cases with $N_i = 0$ in that batch, and we compensate this *under-sampling* of zero-claims by attaching $2\mu(\boldsymbol{x}_i, v_i)$ to the latter cases.[11] This idea is, of course, related to importance sampling. This under/over-sampling with a factor 2 is illustrated in Figure 12 (black line) and in Table 8. In particular, in the early stage of the calibration we see a faster convergence using under/over-sampling, but for fine-tuning one should switch to the observations on the original scale. Doubling the positive cases $N_i > 0$ also implies that we have to perform more GDM steps per epoch, see Table 8.

**Conclusions.** Since the resulting differences in Table 8 are rather small, we will not further follow up stratified batches and under/over-sampling. Moreover, we are not aware of how stratified batches and under/over-sampling can (easily) be implemented in Keras. For this reason, we have used our own R code to calculate Table 8.

## 4.5 Initialization of gradient descent method

As described in Section 4.1, a main difficulty (for a given network architecture) is to find a reasonably good network parameter $\theta \in \mathbb{R}^r$. If we use the GDM, convergence properties of the algorithm will depend on a good choice of the initial value $\theta \in \mathbb{R}^r$. Some optimizers, like the ones in Keras, try internally to choose good initial values (the default initializer in Keras is "glorot_uniform", for details see Glorot-Bengio [9]).
We remind of Section 2 on feature pre-processing: for the activation functions $\phi$ introduced in (1.6), the change in activation mainly takes place in the neighborhood of the origin (except for ReLU). For this reason, all feature components were pre-processed to this domain (because the GDM is only sensitive around the origin w.r.t. the chosen activation functions). Far off the origin, the gradients will be almost zero and the GDM algorithm will not work (or convergence will be

[10] Note that the run times in Tables 5 and 8 differ because for the former we have used the R interface to Keras back-end version and the latter is a self-coded programme in R because we are not aware of a stratified version of the stochastic gradient descent algorithm in Keras.

[11] This factor 2 has the interpretation of an offset in Poisson regression modeling.

very slow). This implies that also initial network weights $\theta$ for the algorithm should be chosen such that the scalar products $\langle \boldsymbol{w}_j^{(k)}, \boldsymbol{z}^{(k-1)} \rangle$ are in the neighborhood of zero, see (1.5). This will ensure that we do not face the so-called *vanishing gradient problem*. Moreover, the initial network parameter should be randomized to make sure that it does not have unwanted symmetries. Symmetries in initializations may imply that the GDM gets trapped in saddle points, this is similar to the considerations in (4.2). In deep networks with many hidden layers, one often inserts normalization layers to avoid the vanishing gradient problem. This normalization layers map the hidden neurons back to a reasonable scale around the origin. Normalization layers are part of the network architecture and will be further discussed in Section 6, below.

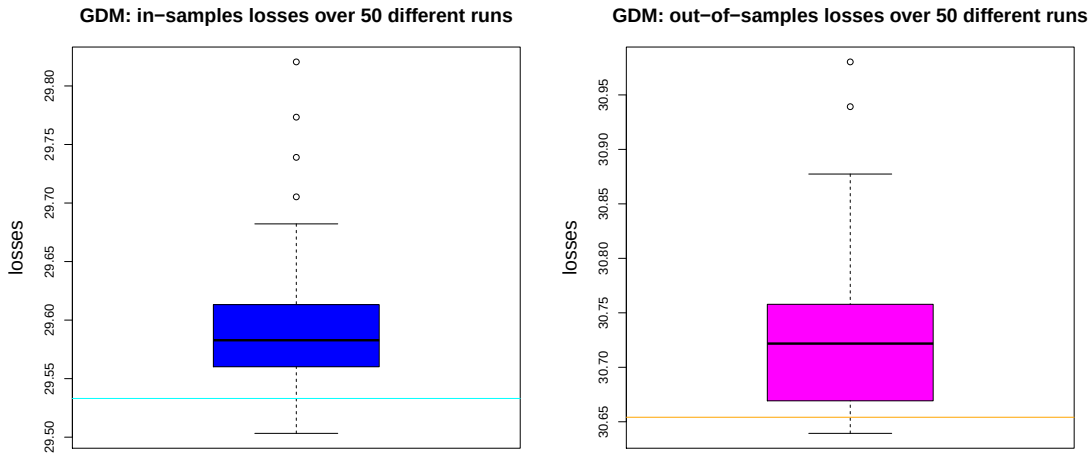


Figure 13: 50 different neural network calibrations using identical set-ups except for initial seeds: (lhs) in-sample losses and (rhs) out-of-sample losses; the horizontal lines show the 'nadam' calibration given in Table 5.

Nevertheless, even if we use latest state-of-the-art concepts, there are still some things that should attract our attention. If we use, say, the default "glorot_uniform" initializer, this choice still requires setting a seed for sampling the initial configuration as well as for sampling the random mini-batches. For each different seed choice we receive a different network calibration, we again refer to Figure 5. Our goal is to understand the volatility obtained in the different networks if we run 50 different calibrations in identical situations, only changing the seed of the initial configuration. We choose exactly the same set-up is in the 'nadam' calibration of Table 5. The results are illustrated in Figure 13. From this plot we conclude that there are quite some differences between different seeds, and it is worth to explore different initial configurations. The horizontal lines show the calibration obtained in Table 5, which says that, by chance, we have received a rather good one, and it could also be worse (though still better than for most other optimizers).

**Embedding/nesting and the homogeneous model.** Another way of initializing a network is to embed (nest) a classical actuarial model into the network. This can, for instance, be done

by using a skip connection. Using this approach we can initialize the network such that the initial calibration of the GDM exactly provides the classical actuarial model, and the network is boosting this classical actuarial model. This idea is described in more detail in our tutorial Schelldorfer–Wüthrich [31]. It helps to improve speed of convergence, and it also helps to detect missing structure in the classical actuarial model.
A simple way to bring the initial network onto the right price level is to embed the homogeneous model into the neural network. This can be achieved by setting the output weights $w_j^{(K+1)}$ of the neurons $1 \leq j \leq q_K$ equal to zero, and by initializing the output intercept $w_0^{(K+1)}$ to the homogeneous model. This can be obtained by replacing lines 8-10 of Listing 3 by the code given in Listing 4, where `lambda` denotes the homogeneous MLE on $\mathcal{D}$.

Listing 4: initialization with the homogeneous model

```
model %>%
  layer_dense(units = q1, activation = 'tanh', input_shape = c(nrow(Xlearn))) %>%
  layer_dense(units = 1, activation = 'exponential',
              weights=list(array(0, dim=c(q1,1)), array(log(lambda), dim=c(1))))
```

In general, we always use the standard initialization provided by "glorot_uniform" for all weights $w_j^{(k)}$ in the hidden layers $1 \leq k \leq K$, and the output weights are initialized according to Listing 4. The GDM then directly aims at improving the homogeneous model.

# 5 Over-fitting, dropout and regularization: items (g)-(h)

We now come to the probably most important question in neural network applications: how can we prevent a neural network calibration from over-fitting to the learning data?

## 5.1 Over-fitting and early stopping

In this section we discuss potential over-fitting of network regression functions to the learning data $\mathcal{D}$. As described in Section 4, most network architectures are over-parametrized in the sense that they allow for a huge modeling flexibility.[12] This implies that if we run the GDM for too long it will result in over-fitting to the learning data $\mathcal{D}$. This over-fitting implies that we will have a small in-sample loss but a poor out-of-sample performance (a big generalization error). We explore this on a neural network architecture of depth $K = 3$ with $(q_1, q_2, q_3) = (20, 15, 10)$ hidden neurons and hyperbolic tangent activation function, see Listing 5.
This network has 1'286 network parameters. We fit this network using the 'nadam' optimizer over 2'000 epochs on mini-batches of size 5'000. In Figure 14 (lhs) we show the decrease in in-sample loss $\mathcal{L}(\mathcal{D}, \widehat{\mu}(\cdot))$ of the GDM on the learning data $\mathcal{D}$ in blue color, and the resulting out-of-sample losses $\mathcal{L}(\mathcal{T}, \widehat{\mu}(\cdot))$ for each epoch on the test data $\mathcal{T}$ are shown in magenta color. We see a typical picture of over-fitting here: the in-sample loss is monotonically decreasing, but the out-of-sample loss increases in later epochs. Thus, in view of Figure 14 (lhs), the GDM should be *early stopped* after roughly 150 to 200 epochs, because in later epochs the GDM learns the noisy part in the learning data $\mathcal{D}$, and not real model structure.

[12] In Section 6.1 we discuss universality theorems which state that (noisy) observations can be approximated arbitrarily well if we allow for arbitrarily many hidden neurons (and if the noisy observations are "non-conflicting").

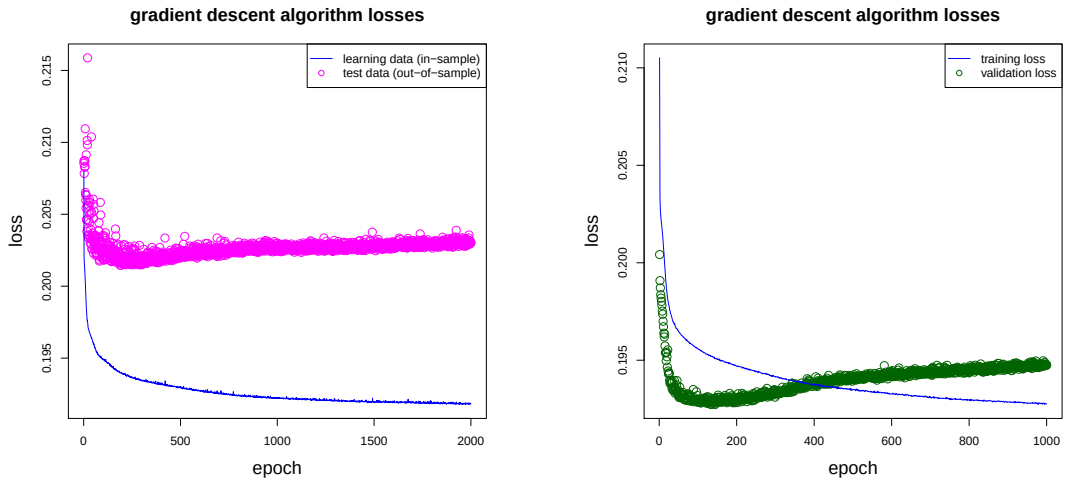


Figure 14: (lhs) over-fitting: decrease in in-sample loss $\mathcal{L}(\mathcal{D}, \widehat{\mu}(\cdot))$ (blue) and resulting out-of-sample loss $\mathcal{L}(\mathcal{T}, \widehat{\mu}(\cdot))$ (magenta); (rhs) over-fitting: determining the optimal calibration network using a callback of the model with minimal validation loss.

Early stopping is more art than science, it is usually done by partitioning the learning data $\mathcal{D}$ into a training set and a validation set. The training set is used for fitting the model and the validation set is used for (out-of-sample) validation. We emphasize that we choose the validation set disjointly from the test data $\mathcal{T}$ because the early stopping rule should not have seen the test data $\mathcal{T}$, as this latter data may still be used later for the choice of the optimal model (if, for instance, we need to decide between networks and boosting machines).
In Figure 14 (rhs) we split the learning data 8:2 into training set (blue color) and validation set (green color). We fit the network on the training set and we out-of-sample validate it on the validation set. We observe that the model starts to over-fit to the learning data after roughly 150 epochs, since the validation loss (green color) starts to increase thereafter. Installing a callback we retrieve the model with the lowest validation loss; the corresponding code is shown in Listing 5, and Table 9 provides the results.

| epochs | batch size | in-sample loss on $\mathcal{D}$ | out-of-sample loss on $\mathcal{T}$ |
|---|---|---|---|
| 1000 epochs | 5'000 | 28.80785 | 30.62573 |
| early stopped | 5'000 | 29.10088 | 30.28564 |

Table 9: fitting statistics of the early stopped GDM using a callback for the model with the lowest validation loss, see Listing 5 and Figure 14 (rhs).

From Table 9 we observe that early stopping is necessary, the upper row considers the model received in Figure 14 (rhs) after 1000 epochs, the lower row has been extracted from the model in Figure 14 (rhs) that has the smallest validation loss (after roughly 150 epochs). This early stopping provides a clearly better out-of-sample performance (generalization to the test data $\mathcal{T}$ which is disjoint from the validation data).

Listing 5: callback of best parameters in Keras

```
set.seed(seed)
use_session_with_seed(seed)

design  <- layer_input(shape = c(ncol(Xlearn)), dtype = 'float32', name = 'design')

output = design %>%
        layer_dense(units=20, activation='tanh', name='layer1') %>%
        layer_dense(units=15, activation='tanh', name='layer2') %>%
        layer_dense(units=10, activation='tanh', name='layer3') %>%
        layer_dense(units=1,  activation='exponential', name='output')

model <- keras_model(inputs = c(design), outputs = c(output))
model %>% compile(loss = 'poisson', optimizer = 'nadam')

CBs <- callback_model_checkpoint("path0", monitor = "val_loss",
                                 save_best_only = TRUE, save_weights_only = TRUE)

fit <- model %>% fit(list(Xlearn), learn$ClaimNb, epochs=1000, batch_size=5000,
                                   validation_split=.2, callbacks=CBs)

load_model_weights_hdf5(model, "path0")
```

Of course, also all these results involve seeds for the initial network parameter, the choice of the random mini-batches, but also the split between training and validation set. These seeds influence the solution in a similar way as has been seen in Figure 13. Moreover, the validation set should have a reasonable size and structure, so that it can mimic generalization losses. Often either splits 8:2 or 9:1 are done, the latter requires bigger data sets, we remind of (4.5).

### 5.2 Regularization

In this section we present a more sophisticated method to prevent from over-fitting. A common idea in statistics is to introduce a regularization term (penalty function). This regularization term prevents regression functions from taking too wild shapes because more extreme network parameters receive a higher penalty term. Choose $p \geq 1$ and regularization parameter $\eta > 0$. We defined the regularized in-sample (Poisson deviance) loss by

$$\mathcal{L}(\mathcal{D}, \widehat{\mu}(\cdot); \eta) \;=\; \frac{1}{n} \sum_{i=1}^{n} 2N_i \left[ \frac{\widehat{\mu}(\boldsymbol{x}_i, v_i)}{N_i} - 1 - \log\left( \frac{\widehat{\mu}(\boldsymbol{x}_i, v_i)}{N_i} \right) \right] + \eta \, \|\theta_-\|_p^p \,, \tag{5.1}$$

we also refer to (3.1), and where $\theta_-$ denotes a subset of all network parameters $\theta$. Typically, we do not regularize all components of $\theta$, for instance, often intercepts $w_{j,0}^{(k)}$ are excluded from regularization. Thus, we penalize the in-sample loss for network parameters $\theta$ that have a bigger $\ell^p$-norm in the selected components $\theta_-$. The bigger the regularization parameter $\eta$, the less the regression function will follow the (in-sample) observations, because for $\theta_- \equiv 0$ we have a homogeneous (regression) model in the corresponding neuron activations.
In statistics, one usually considers $p \in \{1, 2\}$: the $\ell^1$-norm gives so-called LASSO regression (least absolute shrinkage and selection operator regression), and the $\ell^2$-norm regularization is called ridge regression, see Section 3.4 in Hastie et al. [13]. An $\ell^2$-norm penalizes large values of the network parameter more than small ones, whereas the $\ell^1$-norm applies the same scale to

the entire range of the parameters.[13] There are many variants now, for instance, we can use a different regularization parameter in each layer, etc.

Listing 6: ridge regression regularization in Keras

```
output = design %>%
        layer_dense(units=20, kernel_regularizer=regularizer_l2(0.00001),
                                  activation='tanh', name='layer1') %>%
        layer_dense(units=15, kernel_regularizer=regularizer_l2(0.00001),
                                  activation='tanh', name='layer2') %>%
        layer_dense(units=10, kernel_regularizer=regularizer_l2(0.00001),
                                  activation='tanh', name='layer3') %>%
        layer_dense(units=1, activation='exponential', name='output')
```

In Listing 6 we illustrate the implementation of $\ell^2$-ridge regularization. The regularization has been defined for each hidden layer separately, and in Listing 6 we only apply regularization to the network parameters excluding intercepts and excluding the output layer. Including intercepts would require additional commands `bias_regularizer`. The size of the regularization parameter $\eta$ has been chosen for all hidden layers as $10^{-5}$, see Listing 6. The performance of this set-up is studied below.

Minimizing the regularized in-sample loss (5.1) has a nice Bayesian interpretation. Observe that we have (negative) loss (and if we consider all components of $\theta$)

$$-\frac{n}{2}\mathcal{L}(\mathcal{D}, \widehat{\mu}(\cdot); \eta) \;=\; \sum_{i=1}^{n} \left(-\widehat{\mu}(\boldsymbol{x}_i, v_i) + N_i \log \widehat{\mu}(\boldsymbol{x}_i, v_i)\right) \;-\; \frac{\eta n}{2} \left\|\theta\right\|_p^p \;+\; \text{const.}$$

That is, minimizing the regularized in-sample loss provides the same result as maximizing the Bayesian posterior distribution of the network parameter $\theta$, given observations $\mathcal{D}$, under the assumption that $\theta$ has a prior distribution $\pi(\theta) \propto \exp(-\eta n \left\|\theta\right\|_p^p/2)$. For ridge regression this corresponds to a Gaussian prior distribution and for LASSO regression we have a Laplace prior distribution. The regularized optimal network parameter is the Bayesian maximal a posteriori (MAP) estimator, for given observations $\mathcal{D}$.

We now explore ridge regression on our data. The difficult part is a sensible choice of the regularization parameter $\eta > 0$. A very large choice leads, basically, to the homogeneous model, because every non-zero parameter is heavily punished (regularized); for a very small choice we will run into over-fitting. Typically, in statistics, one uses cross-validation to select a reasonable regularization parameter $\eta > 0$. In neural network applications, cross-validation is not a feasible solution because of computational times. Therefore, we just did some trial and error on a few selected values, and we come up with a choice of $\eta = 10^{-5}$ which seems to work well in our example.

We consider exactly the same set-up as in Figure 14 (lhs), i.e. we choose a network of depth $K = 3$ with $(q_1, q_2, q_3) = (20, 15, 10)$ hidden neurons and hyperbolic tangent activation function. We fit this network using the 'nadam' optimizer over 2'000 epochs on batches of size 5'000. The plot on the left-hand side of Figure 15 (lhs) is identical to the one in Figure 14 (lhs). It

[13] Ridge regression mainly leads to shrinkage of large parameter values, whereas LASSO regression also helps for parameter selection because certain parameters may be shrinkaged exactly to zero for (sufficiently) large regularization parameter, i.e. LASSO regression may lead to sparsity, we refer to Hastie et al. [14] and to Section 4.3.2 in [37] for a more extended treatment of LASSO regression models.

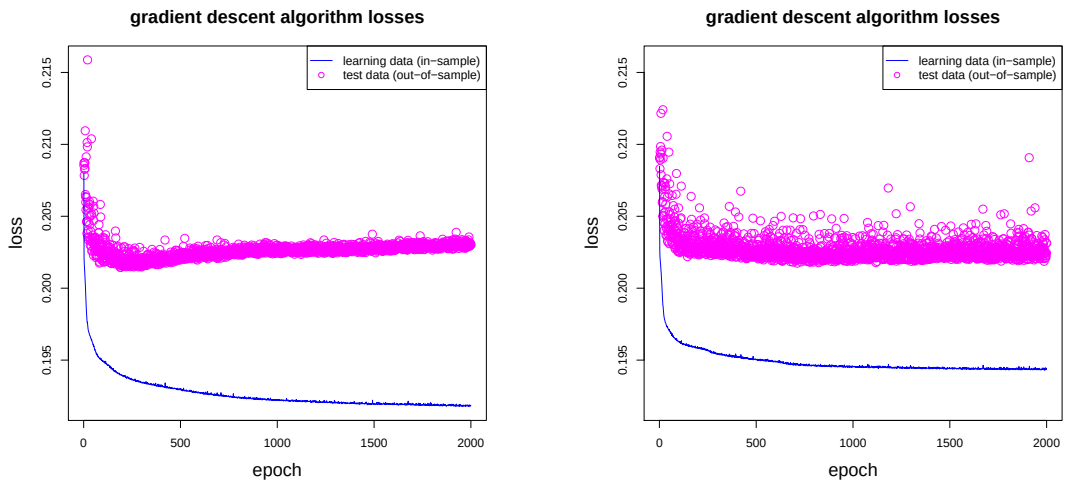


Figure 15: (lhs) over-fitting: decrease in in-sample loss $\mathcal{L}(\mathcal{D}, \widehat{\mu}(\cdot))$ (blue) and resulting out-of-sample loss $\mathcal{L}(\mathcal{T}, \widehat{\mu}(\cdot))$ (magenta); (rhs) ridge regularized version.

shows over-fitting if we run the GDM over 2'000 epochs. Figure 15 (rhs) uses exactly the same set-up, but we add ridge regularization to the 3 hidden layers according to Listing 6. We observe that under this regularization layout we hardly have over-fitting, because the out-of-sample loss remains stable after having reached a certain minimum. The resulting model is provided in Table 10. The ridge regularized model is similarly good as the early stopped in Table 9.

| regularization/dropout | in-sample loss | out-of-sample loss |
|---|---|---|
| un-penalized GDM version | 28.54443 | 30.55742 |
| ridge regression with $\eta = 10^{-5}$ | 28.88793 | 30.29690 |
| dropout rate $p = 1\%$ | 28.70364 | 30.67337 |
| dropout rate $p = 2\%$ | 28.71591 | 30.51971 |
| dropout rate $p = 5\%$ | 28.92088 | 30.39499 |
| dropout rate $p = 10\%$ | 29.17424 | 30.47817 |

Table 10: over-fitting, regularization and dropout; deep network with $K = 3$ and $(q_1, q_2, q_3) = (20, 15, 10)$; losses are in $10^{-2}$.

Remark that the analysis in this section is not fully sound because Figure 15 should not show any out-of-sample losses on the test data, and appropriate regularization parameters $\eta > 0$ should either be chosen with cross-validation or with splitting the learning data $\mathcal{D}$ into training set and validation set, see Section 5.1. Since we will not use this regularization approach below, we do not further explore these points, but we just leave Figure 15 as it stands giving a proof of concept indicating that regularization according to (5.1) may serve its purpose if appropriately applied.

Our final remark is that application of LASSO regularization may lead to sparsity in network connections, turning the fully connected network into a sparsely connected network. This may be another interesting approach to be followed up. We refrain here from doing so.

### 5.3 Dropout

Another way to prevent from over-fitting is to introduce dropout rates for individual neurons. That is, we choose a fixed so-called dropout rate which is a probability $p \in [0, 1)$. In each step of the GDM every neuron (in a specified layer) is removed (independently from the others) with probability $p$. Listing 7 illustrates the implementation of dropouts: on lines 3,5,7 we specify the

Listing 7: R script for a shallow network with dropout in Keras

```
1 output = design %>%
2         layer_dense(units=20, activation='tanh', name='layer1') %>%
3            layer_dropout (rate = 0.05) % >%
4         layer_dense(units=15, activation='tanh', name='layer2') %>%
5            layer_dropout (rate = 0.05) % >%
6         layer_dense(units=10, activation='tanh', name='layer3') %>%
7            layer_dropout (rate = 0.05) % >%
8         layer_dense(units=1,  activation='exponential', name='output')
```

dropout layers that drops any of the neurons independently from each other with probability $p = 5\%$ in each learning step. These dropouts prevent from over-fitting and make the calibration more robust because individual neurons cannot be over-trained to a specific task, but the whole composite of neurons has the provide a good prediction, even in the case of some of them dropping out. We present the results for dropout probabilities $p \in \{1\%, 2\%, 5\%, 10\%\}$ in Table 10, and Figure 16 illustrates the corresponding decay of in- and out-of-sample losses.

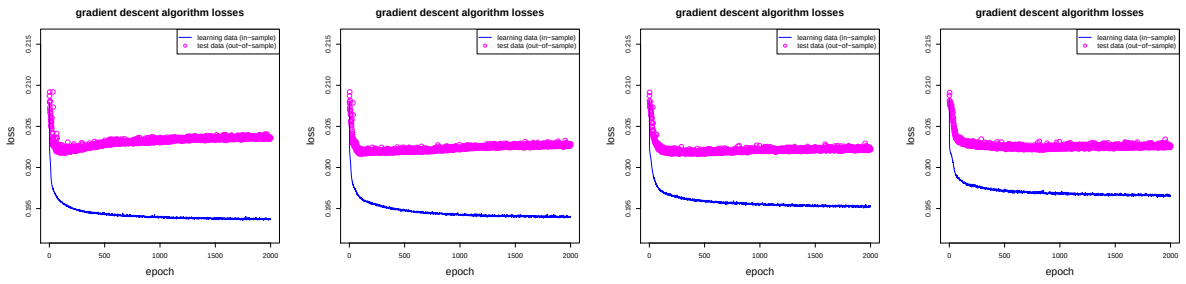


Figure 16: illustration of fitting performance using dropouts with dropout probabilities $p = 1\%, 2\%, 5\%, 10\%$ (from left to right).

Each calibration uses exactly the same network architecture and it starts from exactly the same initial network parameter. From Figure 16 we conclude that with increasing dropout rate, the potential for over-fitting is getting smaller: note that for $p = 10\%$ the out-of-sample loss does not have a local minimum anymore. On the other hand, if the dropout rate $p$ is too large, we receive less competitive models because the models move close(r) to the homogeneous one. In essence, the same remarks apply as to LASSO and ridge regularization in Section 5.2, namely, optimal dropout rates should be determined by cross-validation (which often is too time consuming). Moreover, in a rigorous analysis we should not show the out-of-sample performance in Figure 16, but we should rather split the learning data $\mathcal{D}$ to training and validation set. However, we provide Figure 16 in the current form because it is useful to get the right intuition about dropouts.

We conclude with another interesting derivation presented in Section 18.6 Efron–Hastie [7], which shows that dropout can under certain assumptions be understood as ridge regularization. This derivation also shows how the dropout rate is related to the regularization parameter.

### 5.4 Comparison of the results of Table 10

In this section we compare the results of the different network models received in Table 10 (we skip the dropout model with $p = 2\%$ in order to not have too many models to be compared). Of course, such a comparison is not easy because our networks are quite complex objects. The aim in this section is therefore to understand the neuron activations $\boldsymbol{z}_i^{(3:1)} = \boldsymbol{z}^{(3:1)}(\boldsymbol{x}_i, v_i) = \left(\boldsymbol{z}^{(3)} \circ \boldsymbol{z}^{(2)} \circ \boldsymbol{z}^{(1)}\right)(\boldsymbol{x}_i, v_i) \in (-1,1)^{q_3}$, $i = 1, \ldots, n$, in the last hidden layer of our deep network of depth $K = 3$. Using the canonical link of the Poisson regression model, we get responses for this network, see (1.7),

$$(\boldsymbol{x}_i, v_i) \;\mapsto \mu(\boldsymbol{x}_i, v_i) = \exp\langle \boldsymbol{w}^{(4)}, \boldsymbol{z}_i^{(3:1)} \rangle. \tag{5.2}$$

As emphasized in Richman [27] and Wüthrich [35], we can interpret $(\boldsymbol{z}_i^{(3:1)})_{i=1,\ldots,n}$ as *learned representations* because it reflects a pre-processing of the feature information $(\boldsymbol{x}_i, v_i)_{i=1,\ldots,n}$. With these pre-process features we perform a classical Poisson GLM, which is exactly reflected by the right-hand side of regression function (5.2), we also refer to Remarks 1.1.

In Figure 17 we analyze the learned representations $\boldsymbol{z}_i^{(3:1)} \in (-1,1)^{q_3}$, $i = 1, \ldots, n$, in the last hidden layer ($q_3 = 10$) received from the different calibrations given in Table 10. The first plot on the top row provides the singular values of a principal component analysis (PCA). We therefore construct the design matrix $Z = (\boldsymbol{z}_1^{(3:1)}, \ldots, \boldsymbol{z}_n^{(3:1)})' \in \mathbb{R}^{n \times q_3}$. A singular value decomposition (SVD) is received by considering

$$Z = U \Lambda V,$$

with orthogonal matrix $U \in \mathbb{R}^{n \times q_3}$, orthogonal matrix $V \in \mathbb{R}^{q_3 \times q_3}$ and diagonal matrix $\Lambda = \operatorname{diag}(\lambda_1, \ldots, \lambda_{q_3})$ having singular values $\lambda_1 \geq \ldots \geq \lambda_{q_3} \geq 0$, see Section 14.5 in Hastie et al. [13] for the SVD. Typically, small singular values indicate that the dimensionality of the problem can be reduced because there are some directions which do not sufficiently contribute to the explanation of the design matrix $Z$ (on a linear scale). From Figure 17 we see, as expected, that the un-penalized version and the dropout version with $p = 1\%$ are similar because this small dropout rate does play a significant role. We also observe that these two models have the biggest singular values which illustrates that all $q_3 = 10$ neurons are needed, and each one plays its own individual role. If we increase the dropout rate to $p = 5\%, 10\%$, later singular values start to be smaller, indicating that there are higher correlations between the neuron activations. This illustrates that if one neuron drops out another neuron (being similar) will take over the role of the former one. A similar but more extreme picture is obtained by ridge regression. The ridge regression picture even suggests that 7 hidden neurons would be sufficient in the last hidden layer, because three singular values are close to zero.

The remaining plots in Figure 17 show Spearman's rho correlations of the activated neurons $\boldsymbol{z}_i^{(3:1)}$, $i = 1, \ldots, n$, for the different calibrations provided in Table 10. These correlation plots support the SVD results.

Dropout prevents from extreme behavior. Note that even if these activated neurons show high correlations they are needed. We focus on dropout rate $p = 10\%$ which corresponds to Figure 17 (2nd row, rhs). We observe high correlations between the first three neuron activations

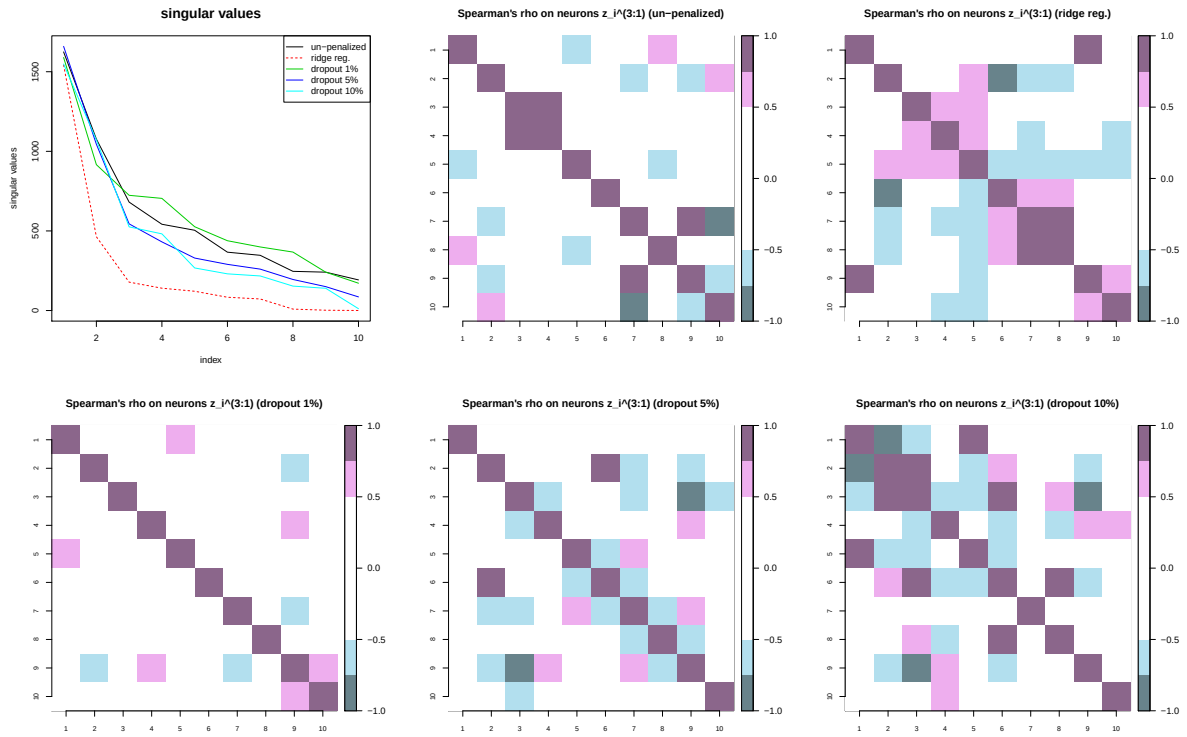


Figure 17: comparison of the calibrations obtained by the models of Table 10: (1st row, lhs) singular values $\lambda_1 \geq \ldots \geq \lambda_{q_3} \geq 0$ of a PCA of the learned representations $\boldsymbol{z}_i^{(3:1)}$, $i = 1, \ldots, n$, for the different calibrations; other plots provide Spearman's rho correlations of the activated neurons $\boldsymbol{z}_i^{(3:1)}$, $i = 1, \ldots, n$, for the un-penalized and the ridge regularized models (1st row), and the dropout models for $p = 1\%, 5\%, 10\%$ (2nd row).

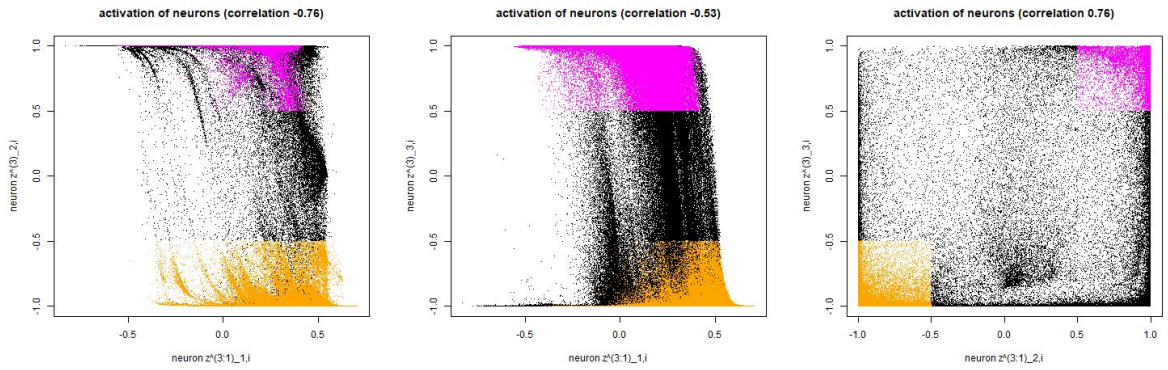


Figure 18: neuron activations $z_{j,i}^{(3:1)}$ for $j = 1, \ldots, 3$ and $i = 1, \ldots, n$ of the dropout network with $p = 10\%$: orange color shows neurons with $\max\{z_{2,i}^{(3:1)}, z_{3,i}^{(3:1)}\} \leq -0.5$, magenta color shows neurons with $\min\{z_{2,i}^{(3:1)}, z_{3,i}^{(3:1)}\} \geq 0.5$.

$(z_{1,i}^{(3:1)}, z_{2,i}^{(3:1)}, z_{3,i}^{(3:1)})$, $i = 1, \ldots, n$. Figure 18 shows the scatter plot of these highly correlated neurons $(z_{1,i}^{(3:1)}, z_{2,i}^{(3:1)}, z_{3,i}^{(3:1)})_{i=1,\ldots,n}$. Though we have high correlations, they are still far from being perfectly correlated. In fact, from Figure 18 we see that certain neurons try to partition the feature space in a certain way in order to have similar insurance policies close to each other under the learned representations $\boldsymbol{z}_i^{(3:1)} = \boldsymbol{z}^{(3:1)}(\boldsymbol{x}_i, v_i)$, $i = 1, \ldots, n$.

### 5.5 Short summary on regularization

We have presented several methods in this section to prevent from over-fitting. Typically, callbacks based on a split of the learning data to training set and validation set are preferred. Ridge regularization and dropouts are less popular because fine-tuning of regularization parameters and dropout rates may be difficult because of run times. In the remainder of these notes we will use callbacks on validation sets to prevent from over-fitting. Therefore, we typically focus on plots similar to Figure 14 (rhs), and we will use blue color for the training set and green color for the validation set.

## 6 Choice of the network architecture: items (c)-(e)

In the previous sections we have been working with fixed network architectures. In Sections 1.3 and 4 we were working with a shallow network having $q_1 = 20$ hidden neurons, and a deep network with $K = 3$ hidden layers having $(q_1, q_2, q_3) = (20, 15, 10)$ hidden neurons was used in Section 5. In the present section we discuss these choices.

### 6.1 Mathematical results: universality theorems and complexity

We start by analyzing the question of the "right" network architecture from a theoretical point of view. In Section 1.3 we have introduced a network architecture that may consist of $K \in \mathbb{N}$ hidden layers each having $q_k \in \mathbb{N}$ hidden neurons, for $k = 1, \ldots, K$. An example with $K = 2$ hidden layers and $(q_1, q_2) = (10, 7)$ hidden neurons is illustrated in Figure 3 (rhs).
A main theoretical question is: what is the minimal required network complexity so to be able to model a fairly general class of regression functions? This question is related to Hilbert's 13th problem, and it was Kolmogorov [17] and his student Arnold [1] who gave a first answer to this kind of question.[14] These early results, however, have not been restricted to a single type of activation function. It was Cybenko [5] and Hornik et al. [15] who proved a *universality theorem* for the sigmoid activation function, saying that *shallow* networks are dense in the set of compactly supported continuous functions (either in supremum or in $L^p$-norm, $p \geq 1$). Thus, shallow networks with arbitrarily many hidden neurons are sufficient for approximating any compactly supported continuous function arbitrarily well.
Similar universality theorems (for non-polynomial activation functions) have been proved by Leshno et al. [18],[15] Park–Sandberg [23, 24], Petrushev [25] and Isenbeck–Rüschendorf [16]. There are two types of proofs for these universality theorems, on the one hand there are algebraic methods of Stone–Weierstrass type and on the other hand Wiener–Tauberian denseness type

[14] see https://en.wikipedia.org/wiki/Kolmogorov-Arnold_representation_theorem

[15] In fact, Leshno et al. [18] state that universality holds if and only if one has a non-polynomial activation function.

proofs. A second stream of literature is Barron [2], Yukich et al. [39], Makavoz [19] and Döhler–Rüschendorf [6], these authors consider approximation results. There is more literature on applications to functional estimations using complexity regularization.
In view of this literature it seems sufficient to work with shallow networks. Why may we want to consider more complex network architectures? We start by discussing networks with one hidden layer. Assume for the moment that $\phi$ is the step function activation and $\mathcal{X}^+ = \mathbb{R}^{q_0}$. In this case we receive neurons in the single hidden layer given by

$$z_j^{(1)}((\boldsymbol{x}, v)) = \phi\langle \boldsymbol{w}_j^{(1)}, (\boldsymbol{x}, v)\rangle = \mathbb{1}_{\left\{\sum_{l=1}^{q_0-1} w_{j,l}^{(1)} x_l + w_{j,q_0}^{(1)} v \geq -w_{j,0}^{(1)}\right\}} \in \{0,1\}, \qquad \text{for } j = 1, \ldots, q_1.$$

Thus, neuron $z_j^{(1)}$ partitions the feature space $\mathcal{X}^+$ using hyperplane

$$H_j^{(1)} = \left\{ \boldsymbol{z} \in \mathbb{R}^{q_0};\ \sum_{l=1}^{q_0} w_{j,l}^{(1)} z_l = -w_{j,0}^{(1)} \right\}, \qquad \text{for } j = 1, \ldots, q_1.$$

This implies that a shallow network with $q_1$ hidden neurons provides a partition of the feature space $\mathcal{X}^+ = \mathbb{R}^{q_0}$ given by the hyperplanes $H_1^{(1)}, \ldots, H_{q_1}^{(1)}$ (here we use the step function activation). Zaslavsky [40] has proved that this partition can have a maximal complexity of

$$\#(q_1, q_0) = \sum_{j=0}^{\min\{q_0, q_1\}} \binom{q_1}{j} \quad \text{disjoint sets.} \tag{6.1}$$

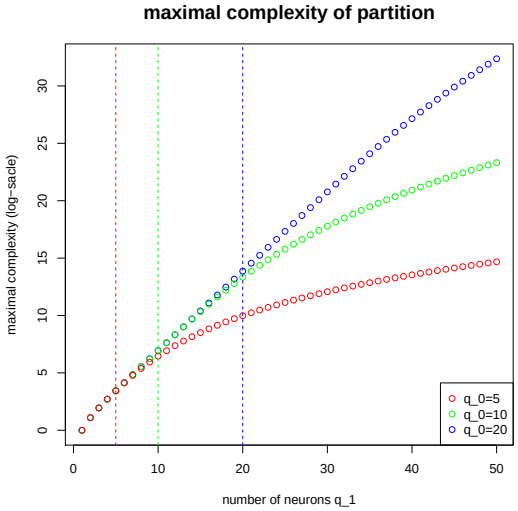


Figure 19: maximal complexity $\#(q_1, q_0)$ of partition (on log-scale) for $q_0 = 5, 10, 20$ (red, green, blue) and $q_1 = 1, \ldots, 50$ neurons (on the $x$-axis).

In Figure 19 we plot the maximal complexity $\#(q_1, q_0)$ which can be achieved by a shallow network with $q_1 = 1, \ldots, 50$ hidden neurons and having feature space $\mathcal{X}^+ = \mathbb{R}^{q_0}$ of dimensions $q_0 = 5, 10, 20$ (red, green blue). Note that the graph is on the log-scale ($y$-axis). We observe an exponential growth of the maximal complexity for $q_1 \leq q_0$, and a slow down to a polynomial growth after $q_0$ (illustrated by the vertical doted lines in Figure 19). If we have a feature space

of dimension $q_0 = 10$ and if we choose a shallow network with $q_1 = 20$ hidden neurons (and step function activation) we obtain a maximal complexity of $\#(q_1 = 20, q_0 = 10) = 616'665$. Thus, we obtain a fairly complex (multivariate) step function to approximate the unknown regression function $\mu : \mathcal{X}^+ \to \mathbb{R}_+$. At the first sight, this seems sufficient for the data of Listing 1.[16]
For deep networks with more than one hidden layer, i.e. $K \geq 2$, the situation is more complicated. Montúfar et al. [20] provide a lower bound for the complexity that can be obtained by such a deep network. Assume that we have the ReLU activation function and that $q_k \geq q_0$ for all $k = 1, \ldots, K$. Theorem 4 in Montúfar et al. [20] states that the maximal complexity is bounded below by

$$\#(q_K, \ldots, q_0) \;\geq\; \left( \prod_{k=1}^{K-1} \left\lfloor \frac{q_k}{q_0} \right\rfloor^{q_0} \right) \; \sum_{j=0}^{q_0} \binom{q_K}{j} \qquad \text{(disjoint) linear regions.} \tag{6.2}$$

**Example 6.1 (Shallow vs. deep networks: complexity)**

We provide an example for the resulting complexity: choose $q_0 = 2$ and $q_k = 4$ for $k \geq 1$. From (6.2) we obtain

$$\#(q_K, \ldots, q_0) \;\geq\; 4^{K-1} \left( \binom{4}{0} + \binom{4}{1} + \binom{4}{2} \right) = \frac{11}{4}\, 4^K. \tag{6.3}$$

If we consider a shallow network with the same number of hidden neurons we obtain (6.1)

$$\#(Kq_1, q_0) = \binom{Kq_1}{0} + \binom{Kq_1}{1} + \binom{Kq_1}{2} = 8K^2 + 2K + 1. \tag{6.4}$$

From this we see that the complexity of the deep network grows faster (exponentially) than the complexity of a shallow network (polynomially) with the same number of hidden neurons if we go deep instead of wide, see (6.3) and (6.4). The deep network has $r = 20K - 3$ parameters and the shallow network $r = 16K + 1$ parameters, thus, the number of parameters grows linearly in both cases but at different speeds. This finishes the example. ∎

The previous example shows that the complexity of the resulting regression function may grow faster for deep networks than for shallow networks if the number of hidden neurons exceeds the dimension $q_0$ of the feature space. For this reason, it is often recommended to work with deep networks for more complex regression functions. Shallow networks typically have a lower approximation capacity than deep ones, and the size of the hidden layer in the shallow network often grows very fast in width in order to get good approximations. The following example shows that a rather simple regression function can be reconstructed easily by a deep network but not by a shallow one.

**Example 6.2 (Shallow vs. deep networks: partitions)**

We present a simple example that can easily be reconstructed by a deep network, but not by a shallow one. Choose a two-dimensional feature space $\mathcal{X} = [0, 1]^2$ and define the regression function $\lambda : \mathcal{X} \to \mathbb{R}_+$ by

$$\boldsymbol{x} \;\mapsto\; \lambda(\boldsymbol{x}) = 1 + \mathbb{1}_{\{x_2 \geq 1/2\}} + \mathbb{1}_{\{x_1 \geq 1/2,\; x_2 \geq 1/2\}} \;\in\; \{1, 2, 3\}. \tag{6.5}$$

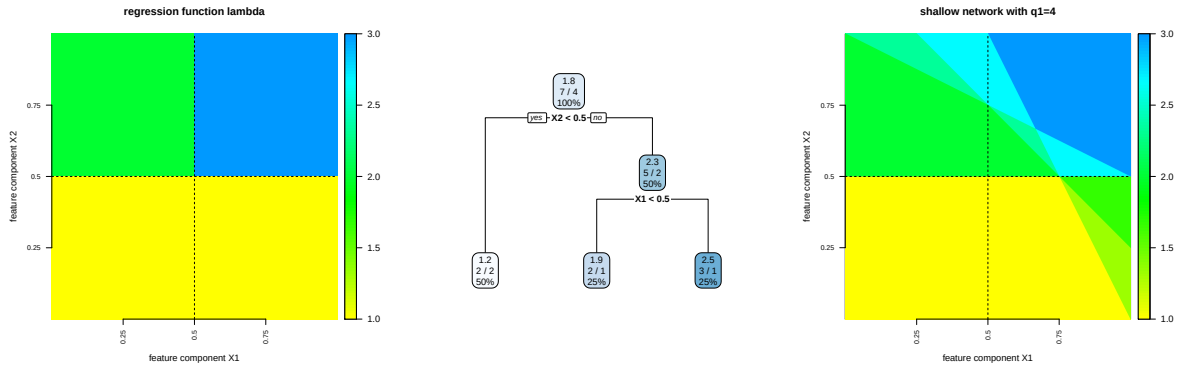


Figure 20: (lhs) regression function (6.5); (middle) regression tree with 2 splits and 3 leaves; (rhs) shallow network approximation with $q_1 = 4$ hidden neurons.

This regression function is illustrated in Figure 20 (lhs).
This regression function (6.5) can easily be modeled by a regression tree[17] with 2 splits and 3 leaves, see Figure 20 (middle). Namely, we need a first split for $\mathbb{1}_{\{x_2 \geq 1/2\}}$ and a second one for $\mathbb{1}_{\{x_1 \geq 1/2\}}$ on the set $\{x_2 \geq 1/2\}$. That is, we require an interaction of the feature components $x_1$ and $x_2$ which can easily be constructed by a regression tree.
If we choose a shallow network with $q_1 = 4$ hidden neurons and step function activation we can construct the regression function

$$\boldsymbol{x} \;\mapsto\; \widehat{\lambda}(\boldsymbol{x}) = w_0^{(2)} + \sum_{j=1}^{4} w_j^{(2)} \mathbb{1}_{\{\langle \boldsymbol{w}_j^{(1)}, \boldsymbol{x} \rangle \geq 0\}}.$$

Figure 20 (rhs) provides such a regression function $\widehat{\lambda}$ for $w_0^{(2)} = w_1^{(2)} = 1$, $w_2^{(2)} = w_3^{(2)} = w_4^{(2)} = 1/3$, $\langle \boldsymbol{w}_1^{(1)}, \boldsymbol{x} \rangle = x_2 - 1/2$ (horizontal split), $\langle \boldsymbol{w}_2^{(1)}, \boldsymbol{x} \rangle = x_1 + x_2 - 5/4$ (split with slope $-1$), $\langle \boldsymbol{w}_3^{(1)}, \boldsymbol{x} \rangle = 2x_1 + x_2 - 2$ (split with slope $-2$), and $\langle \boldsymbol{w}_4^{(1)}, \boldsymbol{x} \rangle = x_1 + 2x_2 - 2$ (split with slope $-1/2$). We observe that this shallow network cannot perfectly replicate the true regression function (6.5), and more hidden neurons are needed for getting a better approximation (which is possible due to the universality theorems, but we may need $q_1 \to \infty$).
We rewrite (6.5) choosing step function activations: for $\boldsymbol{x} \in \mathcal{X}$ we define the first hidden layer

$$\boldsymbol{x} \;\mapsto\; \boldsymbol{z}^{(1)}(\boldsymbol{x}) = \Big( z_1^{(1)}(\boldsymbol{x}), z_2^{(1)}(\boldsymbol{x}) \Big)' = \big( \mathbb{1}_{\{x_1 \geq 1/2\}}, \mathbb{1}_{\{x_2 \geq 1/2\}} \big)' \in \{0,1\}^2.$$

This provides

$$\begin{aligned} \lambda(\boldsymbol{x}) &= 1 + \mathbb{1}_{\{x_2 \geq 1/2\}} + \mathbb{1}_{\{x_1 \geq 1/2\}} \mathbb{1}_{\{x_2 \geq 1/2\}} = 1 + z_2^{(1)}(\boldsymbol{x}) + z_1^{(1)}(\boldsymbol{x})\; z_2^{(1)}(\boldsymbol{x}) \\ &= 1 + \mathbb{1}_{\{z_2^{(1)}(\boldsymbol{x}) \geq 1/2\}} + \mathbb{1}_{\{z_1^{(1)}(\boldsymbol{x}) + z_2^{(1)}(\boldsymbol{x}) \geq 3/2\}} = 1 + z_1^{(2)}\Big(\boldsymbol{z}^{(1)}(\boldsymbol{x})\Big) + z_2^{(2)}\Big(\boldsymbol{z}^{(1)}(\boldsymbol{x})\Big), \end{aligned} \tag{6.6}$$

with neurons in the second hidden layer given by

$$\boldsymbol{z} \;\mapsto\; \boldsymbol{z}^{(2)}(\boldsymbol{z}) = \Big( z_1^{(2)}(\boldsymbol{z}), z_2^{(2)}(\boldsymbol{z}) \Big)' = \big( \mathbb{1}_{\{z_2 \geq 1/2\}}, \mathbb{1}_{\{z_1 + z_2 \geq 3/2\}} \big)', \qquad \text{for } \boldsymbol{z} \in [0,1]^2.$$

[16] Note that our situation is fairly more sophisticated because of categorical feature components.
[17] For an introduction to regression trees we refer to Section 4 of Noll et al. [22].

Thus, we obtain deep network regression function

$$\boldsymbol{x} \;\mapsto\; \lambda(\boldsymbol{x}) = \left\langle \boldsymbol{w}^{(3)}, \boldsymbol{z}^{(2:1)}(\boldsymbol{x}) \right\rangle = \left\langle \boldsymbol{w}^{(3)}, (\boldsymbol{z}^{(2)} \circ \boldsymbol{z}^{(1)})(\boldsymbol{x}) \right\rangle,$$

with output weights $\boldsymbol{w}^{(3)} = (1,1,1)' \in \mathbb{R}^3$. We conclude that a deep network with $K = 2$ hidden layers and $q_1 = q_2 = 2$ hidden neurons, i.e. with totally 4 hidden neurons, can perfectly replicate example (6.5). ∎

**Conclusions.**

- The findings of the previous example are that if we have different types of interactions involving different numbers of feature components, we often receive better modeling properties with deep networks. In the previous example, the deep network is used to model a multiplicative interaction (6.6), whereas shallow networks are most appropriate for additive structure. This gives us some intuition about the functioning of layers, as a rule of thumb we may state that going wide in layers acts like having bigger summations, and going deep rather like having bigger products. Thus, the width is related to the complexity of individual feature components, and the depth is related to the complexity of interactions of feature components.

- There is an increasing number of research that tries to analyze such convergence rate questions, for instance, Shaham et al. [32] prove that under certain assumptions on the feature space a sparsely connected neural network of depth 4 is sufficient to appropriately approximate nice regression functions. An interesting reference is Grohs et al. [12] called "deep neural network approximation theory". This paper studies the approximation capacity of different neural network architectures which builds on our intuition above. Basically, these authors argue that increasing shallow networks is like *superimposing* more basis functions, and such superpositions lead to polynomial convergence rates. Going deep in networks means *composing* more basis functions, and such compositions lead to exponential convergence rates in their modeling set-up [12] (based on ReLU activations). We believe that it is worth to study and develop an approximation theory for compositions of basis functions; this has not been on the core agenda of mathematical research.

## 6.2 Shallow vs. deep networks

In this section we study different network architectures. In particular, we compare shallow networks with $q_1 = 10, 20, 30, 40, 50$ hidden neurons to deep networks with multiple hidden layers $K = 2, 3, 4$. In all models we choose the same batch size of 5'000 samples and we use the same optimizer 'nadam'. Since all architectures have different network parameters (of different dimensions $r$) we cannot use identical initial configurations for the network parameters. Therefore, we just use the standard initialization provided in Keras with output layer initialized with the homogeneous model, see Listing 4. We perform a thorough training and validation analysis using 20% of the data as validation data, and we retrieve the model with the lowest validation loss using a callback. The results are given in Table 11, and we are going to analyze and discuss them in the next two subsections.

| architecture | parameters | epochs | run time | in-sample loss | out-of-sample loss |
|---|---|---|---|---|---|
| shallow network $q_1 = 10$ | 411 | 2'000 | 824s | 29.36776 | 30.46487 |
| shallow network $q_1 = 20$ | 821 | 2'000 | 830s | 29.23560 | 30.46158 |
| shallow network $q_1 = 30$ | 1'231 | 2'000 | 952s | 29.14599 | 30.57247 |
| shallow network $q_1 = 40$ | 1'641 | 2'000 | 955s | 29.00995 | 30.71580 |
| shallow network $q_1 = 50$ | 2'051 | 2'000 | 1'264s | 29.28273 | 30.67552 |
| deep net $(q_1, q_2) = (10, 10)$ | 521 | 2'000 | 879s | 29.14120 | 30.38606 |
| deep net $(q_1, q_2) = (10, 20)$ | 611 | 2'000 | 940s | 29.05296 | 30.32940 |
| deep net $(q_1, q_2) = (20, 10)$ | 1'021 | 2'000 | 1'007s | 29.20648 | 30.41486 |
| deep net $(q_1, q_2) = (20, 20)$ | 1'241 | 2'000 | 1'137s | 29.18965 | 30.38943 |
| $(q_1, q_2, q_3) = (10, 10, 10)$ | 631 | 2'000 | 981s | 29.20254 | 30.32576 |
| $(q_1, q_2, q_3) = (10, 10, 20)$ | 751 | 2'000 | 1'070s | 28.94057 | 30.34968 |
| $(q_1, q_2, q_3) = (10, 15, 10)$ | 736 | 2'000 | 1'027s | 28.96739 | 30.26629 |
| $(q_1, q_2, q_3) = (10, 15, 20)$ | 906 | 2'000 | 1'124s | 28.98431 | 30.30432 |
| $(q_1, q_2, q_3) = (10, 20, 10)$ | 841 | 2'000 | 1'177s | 29.24748 | 30.43185 |
| $(q_1, q_2, q_3) = (10, 20, 20)$ | 1'061 | 2'000 | 1'154s | 28.95609 | 30.35747 |
| $(q_1, q_2, q_3) = (20, 10, 10)$ | 1'131 | 2'000 | 1'072s | 29.22954 | 30.42635 |
| $(q_1, q_2, q_3) = (20, 10, 20)$ | 1'251 | 2'000 | 1'231s | 28.98987 | 30.37648 |
| $(q_1, q_2, q_3) = (20, 15, 10)$ | 1'286 | 2'000 | 1'207s | 29.10088 | 30.28564 |
| $(q_1, q_2, q_3) = (20, 15, 20)$ | 1'456 | 2'000 | 1'368s | 29.18128 | 30.34904 |
| $(q_1, q_2, q_3) = (20, 20, 10)$ | 1'441 | 2'000 | 1'204s | 29.13374 | 30.41227 |
| $(q_1, q_2, q_3) = (20, 20, 20)$ | 1'661 | 2'000 | 1'268s | 29.13334 | 30.39423 |
| $(q_1, q_2, q_3, q_4) = (20, 15, 10, 10)$ | 1'396 | 2'000 | 1'286s | 29.18722 | 30.35759 |
| $(q_1, q_2, q_3, q_4) = (20, 15, 15, 10)$ | 1'526 | 2'000 | 1'489s | 28.98548 | 30.39928 |
| $(q_1, q_2, q_3, q_4) = (20, 20, 10, 10)$ | 1'551 | 2'000 | 1'343s | 29.17139 | 30.39881 |
| $(q_1, q_2, q_3, q_4) = (20, 20, 15, 10)$ | 1'706 | 2'000 | 1'383s | 29.17308 | 30.38237 |

Table 11: comparison of different network architectures; losses are in $10^{-2}$ and we have retrieved the model with the lowest validation loss using a callback.

### 6.2.1 Shallow networks

We start by analyzing the shallow networks of Table 11. First we observe that run times increase in the number of parameters of the shallow networks (for identical numbers of epochs). The resulting out-of-sample losses are quite similar between the different architectures, and they are not fully competitive with deep networks. Figure 21 shows the decrease in losses during gradient

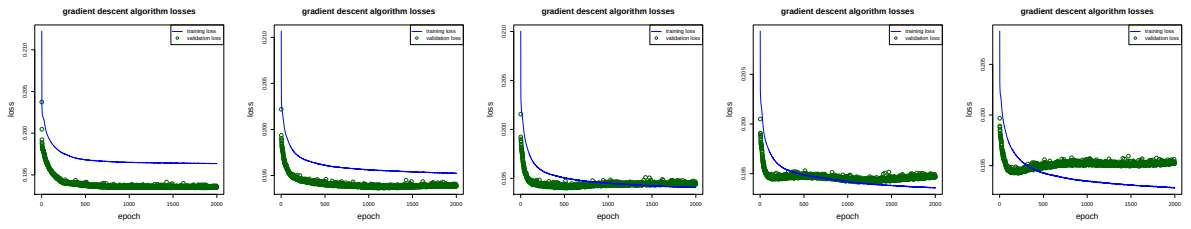


Figure 21: decreases of losses of the shallow networks of Table 11 with $q_1 = 10, 20, 30, 40, 50$ (from left to right); the $y$-scales are different in the plots.

descent fitting, in blue color we have training losses and in green color validation losses. The shallow architecture with $q_1 = 10$ does not suffer from over-fitting, and the architectures with $q_1 \geq 20$ show over-fitting, therefore, the networks with the lowest validation losses are retrieved using a callback for these bigger networks. Interestingly, the graph for $q_1 = 40$ shows two different local minimas, which indicates that this GDM may explore two different calibrations. Another point worth noting is that more complex models lead to faster convergence, for $q_1 = 50$ early stopping is exercised after roughly 200 epochs which corresponds to a run time of 130s. This faster convergence is explained by the fact that larger networks simultaneously fine-tune in more directions (degrees of freedom), however, the resulting regression model is not necessarily better because this may more easily deteriorate in over-fitting.

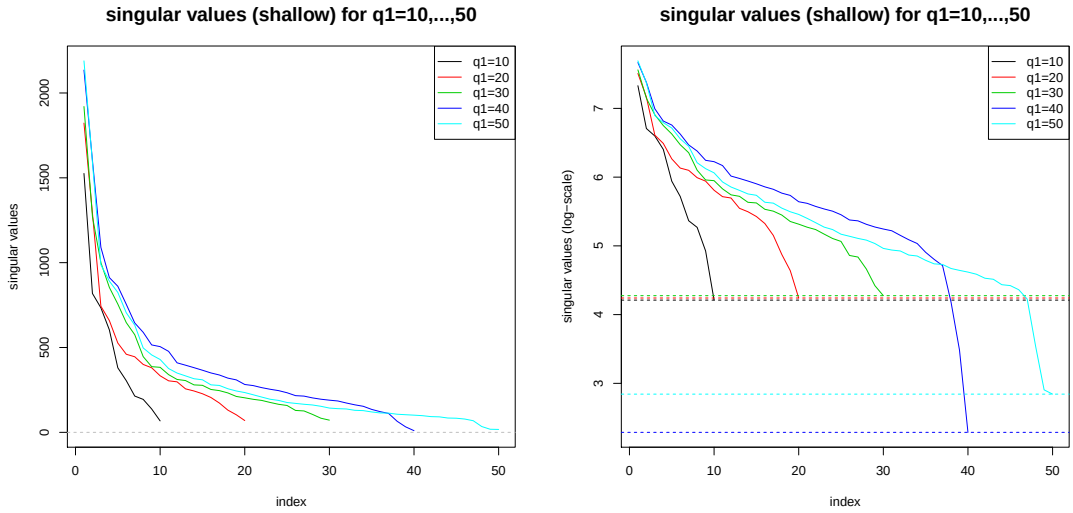


Figure 22: singular values $\lambda_1^{(q_1)} \geq \ldots \geq \lambda_{q_1}^{(q_1)} \geq 0$ of the neuron activations $\boldsymbol{z}^{(1)}(\boldsymbol{x}_i)$, $i = 1, \ldots, n$, of a shallow network with $q_1 = 10, \ldots, 50$ hidden neurons: (lhs) original scale, (rhs) log-scale.

In Figure 22 we provide the (in-sample) singular values $\lambda_1^{(q_1)} \geq \ldots \geq \lambda_{q_1}^{(q_1)} \geq 0$ of PCAs of the neuron activations $(\boldsymbol{z}^{(1)}(\boldsymbol{x}_1), \ldots, \boldsymbol{z}^{(1)}(\boldsymbol{x}_n))' \in \mathbb{R}^{n \times q_1}$ of these shallow networks for $q_1 = 10, \ldots, 50$. The first observation is that the smallest singular values $\lambda_{q_1}^{(q_1)}$ are clearly smaller for $q_1 = 40, 50$ than for $q_1 = 10, 20, 30$, see Figure 22 (rhs) where the horizontal dotted lines illustrate the smallest singular values. We interpret this as follows: the more hidden neurons, the more fine-tuning of in-sample losses is done, and the less relevant new directions are added to the PCA. The other observation is that we have $\lambda_j^{(10)} \leq \lambda_j^{(20)} \leq \lambda_j^{(30)} \leq \lambda_j^{(40)} \leq \lambda_j^{(50)}$ for many fixed $j$'s (where defined). This may be interpreted as follows: the more hidden neurons, the more individual tasks they perform, and the more (orthogonal) heterogeneity in the hidden neurons is induced.

In Table 6 of Noll et al. [22] we have concluded (based on cross-validation) that the optimal number of leaves in a regression tree for this claim frequency modeling problem is 33. On the one hand, shallow networks are more flexible,[18] and on the other hand they have some deficiencies, as shown in Example 6.2. Therefore, it is sometimes a good choice to take slightly less hidden

[18] Regression trees restrict to *standardized* binary splits which are rectangular in nature, shallow networks also allow for other angles.

neurons than the optimal number of leaves. In our case this would mean that a good choice is $q_1 < 33$. Observe that for such choices the smallest singular values $\lambda_{q_1}^{(q_1)}$, $q_1 \leq 30$, in Figure 22 are almost identical. We also highlight that the networks $q_1 = 40, 50$ have more less significant singular values below the green-red-black dotted line, which again illustrates that roughly 30 hidden neurons should be sufficient.
As for a lower bound for $q_1$ we need to study the minimal required complexity of the regression problem considered. Note that if we choose a single hidden neuron in a shallow network ($q_1 = 1$), then the hidden layer compresses all information to one dimension, i.e.,

$$(\boldsymbol{x}, v) \in \mathcal{X} \;\mapsto\; \boldsymbol{z}^{(1)}(\boldsymbol{x}, v) = \phi\langle \boldsymbol{w}_1^{(1)}, (\boldsymbol{x}, v)\rangle \in \mathbb{R},$$

and we are exactly in a generalized linear model (GLM). Any two policies $i$ and $i'$ with feature differences $(\boldsymbol{x}_i - \boldsymbol{x}_{i'}, v_i - v_{i'})$ being orthogonal to $\boldsymbol{w}_1^{(1)}$ have the same neuron activations $\boldsymbol{z}^{(1)}(\boldsymbol{x}_i, v_i) = \boldsymbol{z}^{(1)}(\boldsymbol{x}_{i'}, v_{i'})$. Thus, on hyperplanes orthogonal to $\boldsymbol{w}_1^{(1)}$ we receive identical neuron activations. This shows that if the number $q_1$ is too small, then too much information gets lost in the (first) hidden layer. Often a good choice is $q_1 > q_0$,[19] which is also a crucial assumption in complexity consideration (6.2). For our problem, we believe that $q_1 = 10$ is too small, though the out-of-sample figures are comparable to bigger networks. We come to this conclusion because we cannot see any sign of over-fitting in Figure 21 (lhs).

**Conclusion.**
We choose dimension $q_1 = 20$ or 30 for our shallow network modeling approach. We remind of the (important) Example 6.2 which shows that shallow networks may have some deficiencies in modeling interactions in feature components. For this reason we turn our attention to deep networks next.

### 6.2.2 Deep networks

If we consider deep networks with $K = 2$ hidden layers and $q_1, q_2 \in \{10, 20\}$ hidden neurons, we obtain results that are better than the shallow network ones. Note that all configurations lead to lower out-of-sample losses than their shallow counterparts. This says that we have interactions between the feature components that cannot easily be captured by shallow networks. As can be seen from Figure 23 we also receive fast convergence for more complex networks, the network $(q_1, q_2) = (20, 20)$ can be fitted in roughly 200 epochs, which corresponds to a run time of 120s. Comparing the 4 deep networks considered we conclude that they have a comparable out-of-sample performance, i.e. the explicit choice of the network architecture is less important, once they are sufficiently flexible and choices of different seeds of the gradient descent algorithm may have a bigger influence on the resulting model than the choice of the architecture. Therefore, fine-tuning architectures is not really a sensible thing to do, once we allow for sufficient degrees of freedom.
Considering $K = 3$ hidden layers, we receive further improvements. This indicates that it is worth exploring interactions in deeper networks. Also run times improve in these bigger networks; as can be seen from Figure 24, at most 500 epochs are sufficient which reduces run

[19] For this advice we account dimension 1 for a categorical feature component, and not the resulting dimension of the corresponding dummy coding. If the marginal complexities are much bigger, i.e., if we obtain high effective degrees of freedom in a generalized additive model (GAM) analysis, then we should choose a bigger $q_1$.

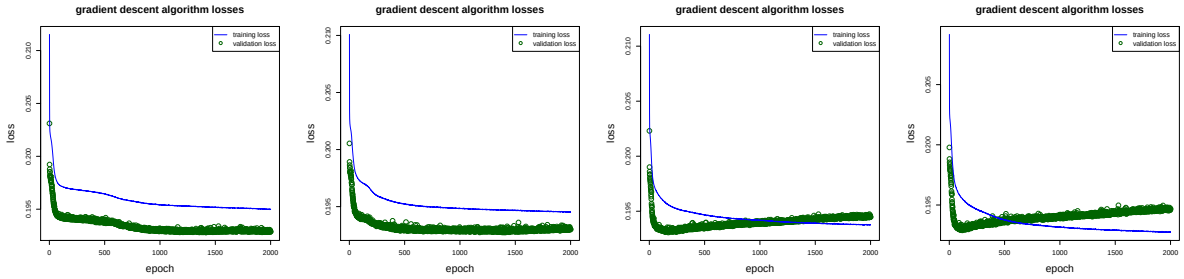


Figure 23: decreases of losses of the deep networks of Table 11 with 2 hidden layers and $(q_1, q_2) = (10, 10), (10, 20), (20, 10), (20, 20)$ (from left to right); the $y$-scales are different in the plots.

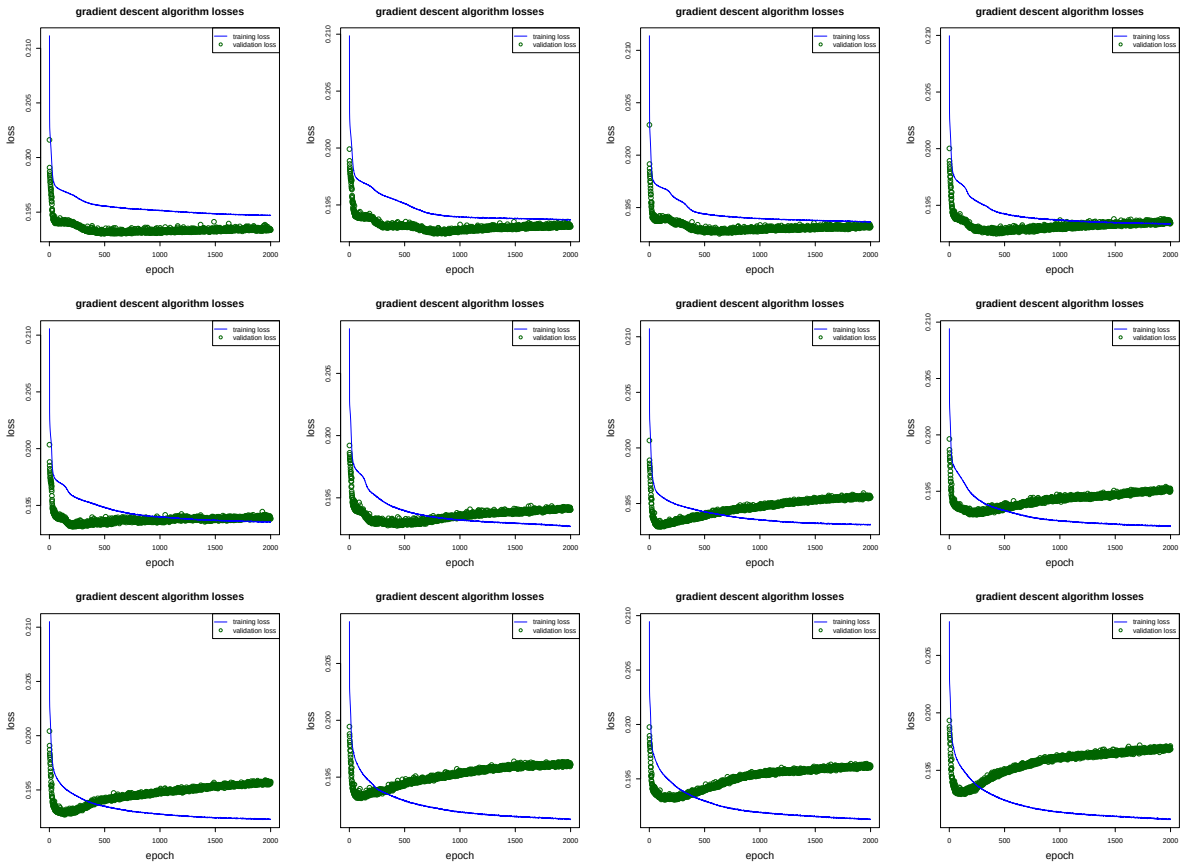


Figure 24: decreases of losses of the deep networks of Table 11 with $K = 3$ hidden layers (top row) $(q_1, q_2, q_3) = (10, 10, 10), (10, 10, 20), (10, 15, 10), (10, 15, 20)$, (middle row) $(q_1, q_2, q_3) = (10, 20, 10), (10, 20, 20), (20, 10, 10), (20, 10, 20)$, (bottom row) $(q_1, q_2, q_3) = (20, 15, 10), (20, 15, 20), (20, 20, 10), (20, 20, 20)$; the $y$-scales are different in the plots.

times to roughly 300s. Network $(q_1, q_2, q_3) = (20, 15, 10)$ is even fitted in less than 200 epochs, resulting in a run time of 120s. This is exactly the model we have already met in Table 9. As a general strategy, we believe that the first hidden layer should be sufficiently large, otherwise already in the first compression to the first hidden layer too much information gets lost. In this

sense the choice $(q_1, q_2, q_3) = (20, 15, 10)$ might be a good strategy, as it successively compresses information. The network $(q_1, q_2, q_3) = (20, 10, 20)$ can be seen as a bottleneck network which acts similarly to a (non-linear) PCA, we refer to our tutorial [26].

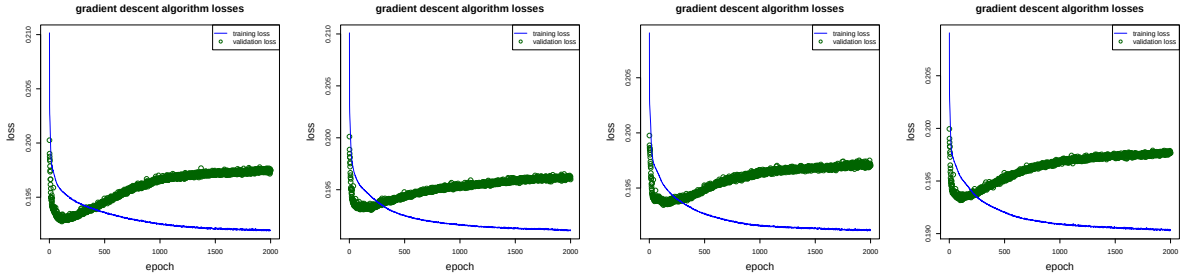


Figure 25: decreases of losses of the deep networks of Table 11 with $K = 4$ hidden layers $(q_1, q_2, q_3, q_4) = (20, 15, 10, 10), (20, 15, 15, 10), (20, 20, 10, 10), (20, 20, 15, 10)$; the $y$-scales are different in the plots.

Finally, we consider $K = 4$ hidden layers, see Table 11. We observe that the four models considered are almost indistinguishable on out-of-sample losses, which just says that all these models have the minimal required complexity to receive good predictions. On the other hand, they are not quite as accurate as the best networks with $K = 3$ hidden layers. This may indicate that three hidden layers are sufficient to capture interactions in the features for the present regression task. Also in this case we receive fast convergence, see Figure 25.

**Conclusion.**
We conclude that in theory shallow networks are sufficient, but in practice deep networks have a better fitting performance for our regression problem. In particular, three hidden layers lead to accurate models, and also to fast convergence. Moreover, trying to find "the best" network architecture is not a sensible thing to do because typically we have many comparably good models. In fact, good performance in gradient descent fitting requires to have some redundancy in the architecture. We could still sparsify this redundancy by applying LASSO regularization.

### 6.3 Activation functions

Next we discuss the choice of activation function $\phi$. Typical choices of activation functions have been introduced in (1.6). Our first remark is that the sigmoid activation function $\varsigma(x) = (1 + e^{-x})^{-1}$ can be obtained from the hyperbolic tangent one, using identity $\tanh(x/2) = 2\varsigma(x) - 1$. For this reason, it is sufficient in theory to work with one version of the two activation functions. In practice, however, the particular choice of the activation function may matter: calibration of deep networks may be slightly simpler if we choose the hyperbolic tangent activation because this will guarantee that all hidden neurons $z_j^{(k:1)}(\boldsymbol{x}) \in (-1, 1)$, which is the domain of the main activation of the neurons in the next (hidden) layer $k + 1$ (for intercept zero).
The step function activation $\phi(x) = \mathbb{1}_{\{x \geq 0\}}$ is useful for theoretical considerations. It has, for instance, been used in Section 6.1. From a practical point of view it is less useful because it is not differentiable and difficult to calibrate. Moreover, the discontinuity also implies that neighboring feature components may have rather different regression function responses: if the

| activation functions | parameters | epochs | run time | in-sample loss | out-of-sample loss |
|---|---|---|---|---|---|
| hyperbolic tangent | 1'286 | 500 | 294s | 29.10088 | 30.28564 |
| ReLU | 1'286 | 500 | 299s | 29.18429 | 30.45125 |
| hard sigmoid | 1'286 | 500 | 331s | 29.35628 | 30.55815 |

Table 12: deep network with $K = 3$ hidden layers $(q_1, q_2, q_3) = (20, 15, 10)$, batch size 5'000 and 'nadam' optimizer; losses are in $10^{-2}$ and we have retrieved the model with the lowest validation loss using a callback.

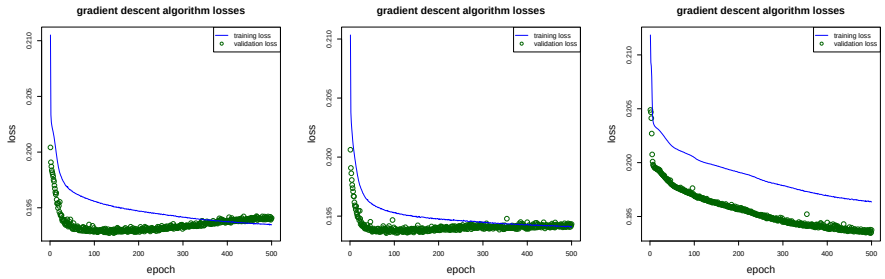


Figure 26: decreases of losses of the deep networks of Table 12 with $K = 3$ hidden layers $(q_1, q_2, q_3) = (20, 15, 10)$ and (lhs) hyperbolic tangent, (middle) ReLU, and (rhs) hard sigmoid activations; the $y$-scales are different in the plots.

step function jumps, say, between driver's ages 48 and 49, then these two driver's ages may have a rather different insurance premium. For these reasons we do not pursue with the step function activation here. Note, however, that the step function activation can be received as limit of the sigmoid activation function, i.e.

$$\lim_{b\to\infty} (1 + e^{-bx})^{-1} = \mathbb{1}_{\{x\geq 0\}}, \qquad \text{for all } x \in \mathbb{R} \setminus \{0\}.$$

Another activation that is sometimes used is the so-called hard sigmoid activation function. It is defined by

$$\phi(x) \;=\; \frac{(x \wedge 2.5) \vee (-2.5)}{5} + \frac{1}{2} \;=\; \begin{cases} 0 & \text{if } x \leq -2.5, \\ x/5 + 1/2 & \text{if } -2.5 < x < 2.5, \\ 1 & \text{if } x \geq 2.5. \end{cases}$$

This is a linear interpolation between $-2.5$ and 2.5 (as an approximation to the sigmoid activation function). Finally, we consider the ReLU activation function $\phi(x) = x\mathbb{1}_{\{x\geq 0\}}$ which has received considerable attention because of its good properties.

In Table 12 we present the results for the three considered activation functions. In all three cases we consider a deep network with $K = 3$ hidden layers having hidden neurons $(q_1, q_2, q_3) = (20, 15, 10)$, batch size 5'000, 500 epochs and the 'nadam' optimizer. Figure 26 shows the gradient descent performance, and we use a callback to retrieve the model with the lowest validation loss. We observe that in our case the hyperbolic tangent slightly outperforms the ReLU activation function in terms of prediction accuracy, see Table 12. On the other hand, ReLU leads to faster convergence in our set-up, see Figure 26. Bottom line there is not much good reason to clearly

favor one over the other activation function just from these results. On the other hand, the hard sigmoid activation function is not competitive in terms of run times, see Figure 26, and we will not further consider it.
We remark that the ReLU activation function often leads to sparsity in deep network activations because some neurons remain unactivated for the entire input. Such an effect may be wanted because it reduces the complexity of the regression model, but it can also be an undesired side effect because it may increase the difficulty in model calibration because of more vanishing gradients. Moreover, ReLU may lead to arbitrarily large activations in neurons because it is an unbounded activation function, this may be an unwanted effect because it may need re-scaling of activations to the main domain around the origin. This is exactly the topic of the next section that considers normalization layers.

### 6.4 Normalization layers

Calibration of deep networks often suffers the so-called vanishing gradient problem. This is caused by the fact that the features may lie in regions where the gradient becomes almost zero because the activation function is flat. This happens, for instance, for $\phi(x) = \tanh(x)$ if $x$ is very large (and it may be an even more acute problem for the ReLU activation function). This problem becomes even more pronounced in deep networks because, using the chain rule for differentiation, one considers the product of potentially small values. To circumvent these difficulties, one often adds so-called normalization layers between the hidden layers. These normalization layers map the neurons $(z_j^{(k:1)}(\boldsymbol{x}_i))_{j=1,\ldots,q_k}$ back to be centered around the origin having unit variance. In Listing 8 we provide the corresponding code for a deep network with 3 hidden layers having $(q_1, q_2, q_3) = (20, 15, 10)$ hidden neurons (and additionally we add dropout layers with rates $p = 5\%$ in Listing 8).

Listing 8: R script for deep network with normalization layers in Keras

```
model %>%
  layer_dense(units = 20, activation = 'tanh', input_shape = c(q0)) %>%
  layer_batch_normalization() %>%
  layer_dropout(rate = 0.05) %>%
  layer_dense(units = 15, activation = 'tanh') %>%
  layer_batch_normalization() %>%
  layer_dropout(rate = 0.05) %>%
  layer_dense(units = 10, activation = 'tanh') %>%
  layer_dropout(rate = 0.05) %>%
  layer_dense(units = 1, activation = 'exponential')
```

In Table 13, lines 'normalization layers', we present the results for the hyperbolic tangent activation and the ReLU activation, and the gradient descent performance is shown in the left column of Figure 27. A first observation is that these additional network features substantially impact run times. Considering prediction accuracy we observe that our models do not substantially benefit from normalization layers, in combination with dropout rates we receive competitive models, however, run times are not really attractive. In general, we would expect that normalization layers are more effective for the ReLU activation because it is unbounded from above, whereas the hyperbolic tangent activation always leads to neurons $z_j^{(k:1)}(\boldsymbol{x}) \in (-1, 1)$.
A main conclusion is that we may get slightly better fits with normalization and dropout layers,

| $(q_1, q_2, q_3) = (20, 15, 10)$ | parameters | epochs | run time | in-sample loss | out-of-sample loss |
|---|---|---|---|---|---|
| hyperbolic tangent activation function: | | | | | |
| no normalization | 1'286 | 500 | 294s | 29.10088 | 30.28564 |
| normalization layers | 1'356 | 500 | 541s | 29.15353 | 30.55067 |
| normalization & dropout $p = 2\%$ | 1'356 | 500 | 864s | 29.08472 | 30.36194 |
| normalization & dropout $p = 5\%$ | 1'356 | 500 | 853s | 29.11898 | 30.27658 |
| normalization & dropout $p = 10\%$ | 1'356 | 500 | 880s | 29.04255 | 30.32779 |
| rectified linear unit (ReLU) activation function: | | | | | |
| no normalization | 1'286 | 500 | 299s | 29.18429 | 30.45125 |
| normalization layers | 1'356 | 500 | 573s | 29.21236 | 30.46193 |
| normalization & dropout $p = 2\%$ | 1'356 | 500 | 867s | 29.18200 | 30.46023 |
| normalization & dropout $p = 5\%$ | 1'356 | 500 | 834s | 29.27425 | 30.33068 |
| normalization & dropout $p = 10\%$ | 1'356 | 500 | 873s | 29.21958 | 30.41721 |

Table 13: comparison of different network architectures considering normalization and dropout layers; losses are in $10^{-2}$ and we have retrieved the model with the lowest validation loss using a callback.

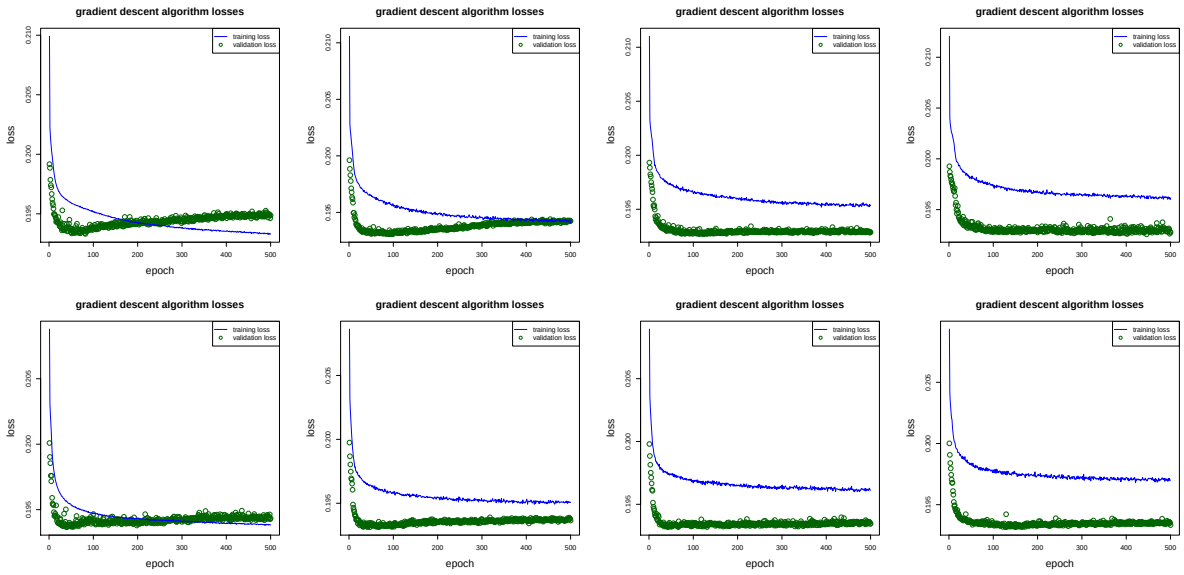


Figure 27: decreases of losses of the deep networks of Table 13 with $K = 3$ hidden layers $(q_1, q_2, q_3) = (20, 15, 10)$, layer normalizations, dropout rates $p = 0\%, 2\%, 5\%, 10\%$ (from left to right) and (top row) hyperbolic tangent activation, (bottom row) ReLU activation; the $y$-scales are different in the plots.

however, run times are not fully competitive and choices of dropout rates need additional cross-validation (or grid search), which does not make this an attractive alternative for our problem.

### 6.5 Bias regularization

There is one point which we have completely ignored so far, namely, do network calibrations meet the right portfolio price level? In all applications above we have tried to minimize Poisson deviance losses which implied that we receive accurate models and insurance prices on individual policy levels. But do these accurate prices on individual policy level also lead to the right overall price on portfolio level? The answer to this question is, unfortunately, NO! This is exactly what has been discussed in [36], typically, network calibrations have a bias and one needs to correct for these biases, otherwise the overall price is misspecified.

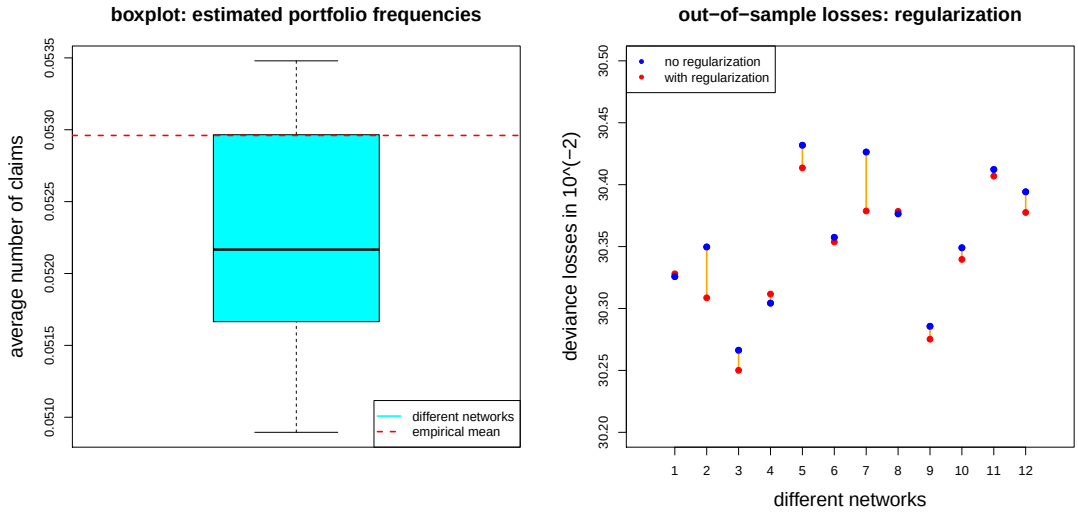


Figure 28: Bias regularization of the 12 deep networks of Table 11 with $K = 3$ hidden layers (lhs) portfolio averages $\widehat{\mu}^a$ and empirical mean $\widehat{\mu}$, (rhs) out-of-sample losses of prices without and with bias regularization.

We consider the 12 deep networks of Table 11 with $K = 3$ hidden layers, and we label the resulting regression functions by $a = 1, \ldots, 12$. Thus, we get estimated regression functions

$$(\boldsymbol{x}, v) \;\mapsto\; \widehat{\mu}^a(\boldsymbol{x}, v), \qquad \text{for } a = 1, \ldots, 12, \tag{6.7}$$

see also (1.3). For each of these 12 models we can calculate the average (in-sample) price that we charge. These average prices are given by

$$\widehat{\mu}^a = \frac{1}{n} \sum_{i=1}^{n} \widehat{\mu}^a(\boldsymbol{x}_i, v_i), \qquad \text{for } a = 1, \ldots, 12.$$

We compare these averages to the empirical mean $\widehat{\mu} = \sum_{i=1}^{n} N_i/n$. Figure 28 (lhs) shows these portfolio averages $\widehat{\mu}^a$ (cyan boxplot) and compare them to the empirical mean $\widehat{\mu} = 5.269\%$ (red dotted line). We observe that most of our estimated models under-estimate the portfolio mean, the worst being model $a = 2$ with $\widehat{\mu}^{a=2} = 5.090\%$. Of course, this misspecification asks for corrections, otherwise the portfolio will be under-priced.
A network with $K = 0$ hidden layers is a classical GLM, see Remarks 1.1. In our case this gives

us GLM regression function, see (1.7),

$$(\boldsymbol{x}, v) \;\mapsto\; \exp\left(w_0^{(1)} + \sum_{j=1}^{q_0-1} w_j^{(1)} x_j + w_{q_0}^{(1)} v\right) = \exp\langle \boldsymbol{w}^{(1)}, (\boldsymbol{x}, v)\rangle. \tag{6.8}$$

The canonical link for the Poisson distribution is the log-link. This tells us that (6.8) gives a GLM with canonical link for the Poisson case. The canonical parameter of this GLM is given by the scalar product $\langle \boldsymbol{w}^{(1)}, (\boldsymbol{x}, v)\rangle$ with regression parameter $\boldsymbol{w}^{(1)} \in \mathbb{R}^{q_0+1}$; note that this includes the intercept parameter $w_0^{(1)}$. Having a GLM with canonical link immediately implies that the resulting MLE $\widehat{\boldsymbol{w}}^{(1)}$ for $\boldsymbol{w}^{(1)}$ gives an unbiased model, see [36]. Thus, a GLM with canonical link does not suffer the same problem as our network models $\widehat{\mu}^a$, and we receive for MLE $\widehat{\boldsymbol{w}}^{(1)}$

$$\frac{1}{n}\sum_{i=1}^{n} \exp\langle \widehat{\boldsymbol{w}}^{(1)}, (\boldsymbol{x}_i, v_i)\rangle = \frac{1}{n}\sum_{i=1}^{n} N_i = \widehat{\mu}.$$

Thus, GLMs with canonical links provide the right price levels, i.e. are *unbiased*. The crucial points in the proof of this unbiasedness result are: (1) the model has an intercept variable $w_0^{(1)}$, (2) the MLE is a critical point of the Poisson deviance loss function, and (3) we use the canonical link. The fitted networks $\widehat{\mu}^a$ suffer the bias problem because the chosen network parameters are, in general, not critical points of the Poisson deviance loss function. This weakness comes from the fact that we exercise early stopping in gradient descent algorithms.
However, the correction for this deficiency is quite simple. We come back to (1.7)

$$(\boldsymbol{x}, v) \;\mapsto\; \exp\left(w_0^{(K+1)} + \sum_{j=1}^{q_K} w_j^{(K+1)} z_j^{(K:1)}(\boldsymbol{x}, v)\right) = \exp\left\langle \boldsymbol{w}^{(K+1)}, \boldsymbol{z}^{(K:1)}(\boldsymbol{x}, v)\right\rangle, \tag{6.9}$$

with activations in the last hidden layer

$$(\boldsymbol{x}, v) \;\in\; \mathbb{R}^{q_0} \;\mapsto\; \boldsymbol{z}^{(K:1)}(\boldsymbol{x}, v) = \left(\boldsymbol{z}^{(K)} \circ \cdots \circ \boldsymbol{z}^{(1)}\right)(\boldsymbol{x}, v) \;\in\; \mathbb{R}^{q_K}. \tag{6.10}$$

Observe that the right-hand sides of (6.8) and (6.9) have the same structural form, and the mapping (6.10) can be interpreted as feature pre-processing.

That is, the network layers pre-process the original features $(\boldsymbol{x}, v) \in \mathbb{R}^{q_0}$ to transformed features $\boldsymbol{z}^{(K:1)}(\boldsymbol{x}, v) \in \mathbb{R}^{q_K}$ such that they can directly be used in a GLM. In this sense, the hidden layers of the network take care of the tedious pre-processing of feature information that is typically done by the actuary so that the data is in a suitable form for GLMing. In machine learning parlance (6.10) is also called *representation learning*.

Thus, following our recipe from above, we first fit network (6.9)-(6.10), state-of-the-art, using any (fancy) version of the gradient descent algorithm, for instance, with dropouts. This then provides fitted representation learning function $(\boldsymbol{x}, v) \mapsto \widehat{\boldsymbol{z}}^{(K:1)}(\boldsymbol{x}, v)$, where the hat indicates that we have a gradient descent fitted network. This fitted network now allow us to pre-process the features of all policies as follows

$$((\boldsymbol{x}_1, v_1), \ldots, (\boldsymbol{x}_n, v_n)) \;\mapsto\; \left(\widehat{\boldsymbol{z}}_1^{(K:1)}, \ldots, \widehat{\boldsymbol{z}}_n^{(K:1)}\right) \overset{\text{def.}}{=} \left(\widehat{\boldsymbol{z}}^{(K:1)}(\boldsymbol{x}_1, v_1), \ldots, \widehat{\boldsymbol{z}}^{(K:1)}(\boldsymbol{x}_n, v_n)\right).$$

As a result, we receive modified data which is directly suitable to be used in a GLM

$$\widehat{\mathcal{D}} = \left\{ (N_i, \widehat{\boldsymbol{z}}_i^{(K:1)}) : \ i = 1, \ldots, n \right\}.$$

Based on this modified data $\widehat{\mathcal{D}}$ we refit the output weights $\boldsymbol{w}^{(K+1)}$ in (6.9) with MLE. Calculating the MLE $\widehat{\boldsymbol{w}}^{(K+1)}$ for $\boldsymbol{w}^{(K+1)}$ implies that the resulting model is unbiased

$$\frac{1}{n} \sum_{i=1}^{n} \exp\langle \widehat{\boldsymbol{w}}^{(K+1)}, \widehat{\boldsymbol{z}}_i^{(K:1)} \rangle = \frac{1}{n} \sum_{i=1}^{n} N_i = \widehat{\mu}, \tag{6.11}$$

for complete details we refer to [36].
Lines 8-9 of Listing 9 illustrate how the learned representations $\widehat{\boldsymbol{z}}_i^{(K:1)}$, $i = 1, \ldots, n$, can be extracted from a network with last hidden layer denoted by `'layer3'`, see also line 9 of Listing 5 for the original network. This is then fed into a Poisson GLM on line 12 of Listing 9 to receive the MLE $\widehat{\boldsymbol{w}}^{(K+1)}$.

Listing 9: R script for network regularization

```
1  #
2  glm.formula <- function(nb){
3     string <- "yy ~ X1"
4     if (nb>1){ for (i in 2:nb){string <- paste(string, "+X",i, sep="")} }
5     string
6      }
7  #
8  zz        <- keras_model(inputs=model$input, outputs=get_layer(model,'layer3')$output)
9  Zlearn    <- data.frame(zz %>% predict(list(Xlearn)))
10 Zlearn$yy <- learn$ClaimNb
11 #
12 glm1 <- glm(as.formula(glm.formula(ncol(Zlearn))), data=Zlearn, family=poisson())
13 fitted(glm1)
14 #
15 Ztest   <- data.frame(zz %>% predict(list(Xtest)))
16 predict(glm1, newdata=Ztest, type="response")
```

Figure 28 (rhs) shows the resulting out-of-sample losses. We note that we get an out-of-sample improvement by this additional MLE step (besides the bias regularization) for models $a = 2, 3, 5, 6, 7, 9, 10, 11, 12$, the models $a = 1, 4, 8$ suffer a small over-fitting by this additional step. The results are summarized in Table 14.

**Conclusion.** We should always apply a bias regularization step to ensure that we have an unbiased model on the portfolio level. If we work in a GLM with canonical link function, this can simply be achieved by an additional MLE step using the neuron activations in the last hidden layer $(\widehat{\boldsymbol{z}}^{(K:1)}(\boldsymbol{x}_i, v_i))_{i=1,\ldots n}$ as new covariates in the GLM. If this is not possible because, for instance, we do not use the canonical link function, then we propose to just adjust the intercept variable $w_0^{(K+1)} \in \mathbb{R}$ correspondingly.

## 6.6 Blending (network) model predictions

In the previous sections we have been analyzing different network architectures and different parametrization of networks. We have already been mentioning that as soon as a network

| deep network architectures $K=3$ | | model parameters | no regularization in-sample | no regularization out-of-sample | bias regularization in-sample | bias regularization out-of-sample |
|---|---|---|---|---|---|---|
| 1 | $(q_1,q_2,q_3)=(10,10,10)$ | 631 | 29.20254 | 30.32576 | 29.20050 | 30.32802 |
| 2 | $(q_1,q_2,q_3)=(10,10,20)$ | 751 | 28.94057 | 30.34968 | 28.92455 | 30.30864 |
| 3 | $(q_1,q_2,q_3)=(10,15,10)$ | 736 | 28.96739 | 30.26629 | 28.95891 | 30.25011 |
| 4 | $(q_1,q_2,q_3)=(10,15,20)$ | 906 | 28.98431 | 30.30432 | 28.97807 | 30.31168 |
| 5 | $(q_1,q_2,q_3)=(10,20,10)$ | 841 | 29.24748 | 30.43185 | 29.23037 | 30.41357 |
| 6 | $(q_1,q_2,q_3)=(10,20,20)$ | 1'061 | 28.95609 | 30.35747 | 28.94694 | 30.35376 |
| 7 | $(q_1,q_2,q_3)=(20,10,10)$ | 1'131 | 29.22954 | 30.42635 | 29.20972 | 30.37847 |
| 8 | $(q_1,q_2,q_3)=(20,10,20)$ | 1'251 | 28.98987 | 30.37648 | 28.98678 | 30.37847 |
| 9 | $(q_1,q_2,q_3)=(20,15,10)$ | 1'286 | 29.10088 | 30.28564 | 29.09855 | 30.27525 |
| 10 | $(q_1,q_2,q_3)=(20,15,20)$ | 1'456 | 29.18128 | 30.34904 | 29.17237 | 30.33965 |
| 11 | $(q_1,q_2,q_3)=(20,20,10)$ | 1'441 | 29.13374 | 30.41227 | 29.12836 | 30.40690 |
| 12 | $(q_1,q_2,q_3)=(20,20,20)$ | 1'661 | 29.13334 | 30.39423 | 29.12465 | 30.37756 |

Table 14: bias regularized networks of depth $K=3$ of Table 11; losses are in $10^{-2}$.

architecture has sufficient complexity we typically receive multiple equally good models, and trying to find the best network architecture is not a sensible thing to do. Even if we fix a network architecture we get multiple equally good models from different parametrization as has been seen in Figure 13, we also refer to Figure 5.

It may be an unpleasant fact that there is no unique best model, but that's how it is, and we need to learn to live with this fact as long as we do not have a theory that helps us to reduce redundancy in networks. In insurance this might be a bit troublesome because models that look equally good on portfolio level (out-of-sample loss), may still be very different on individual policy level, resulting in rather different individual insurance prices.

In the present section we take several network architectures that have found to be sufficiently good, and we are going to blend all these models into one single model. This will average out and smooth pricing, but of course such blended models are more difficult to maintain over time. For this blending we rely again on the 12 deep networks of depth $K=3$ of Table 14, we choose the bias regularized versions and we denote them by

$$(\boldsymbol{x},v) \;\mapsto\; \widehat{\widehat{\mu}}^a(\boldsymbol{x},v),$$

for $a=1,\ldots,12$. The double hat $\widehat{\widehat{\mu}}^a$ should illustrate that we consider the bias regularized versions of regression functions $\widehat{\mu}^a$, i.e. satisfying balance property (6.11).

Blending these models now simply means that we take an average prediction over all models providing regression function

$$(\boldsymbol{x},v) \;\mapsto\; \widehat{\widehat{\mu}}(\boldsymbol{x},v) \;=\; \frac{1}{12}\sum_{a=1}^{12}\widehat{\widehat{\mu}}^a(\boldsymbol{x},v). \tag{6.12}$$

Alternatively, one may also have the idea to blend the models on the canonical scale which provides regression function

$$(\boldsymbol{x},v) \;\mapsto\; \exp\left(\frac{1}{12}\sum_{a=1}^{12}\log\widehat{\widehat{\mu}}^a(\boldsymbol{x},v)\right).$$

However, we do not recommend to consider this second way of blending. Note that if the individual networks $\widehat{\widehat{\mu}}^a$ are unbiased, then Jensen's inequality will imply that averaging canonical parameters will lead to biased models. In fact, we have

$$\widehat{\widehat{\mu}}(\boldsymbol{x}, v) \;=\; \frac{1}{12}\sum_{a=1}^{12}\widehat{\widehat{\mu}}^a(\boldsymbol{x}, v) \;\geq\; \exp\left(\frac{1}{12}\sum_{a=1}^{12}\log\widehat{\widehat{\mu}}^a(\boldsymbol{x}, v)\right),$$

thus, the second version under-estimates the frequencies if the former one is unbiased. Therefore, we prefer to continue with (6.12).

| architecture | in-sample loss | out-of-sample loss |
|---|---|---|
| blended model (6.12) | 28.85961 | 30.11778 |

Table 15: blending all unbiased versions of the models given in Table 14; losses are in $10^{-2}$.

The result is presented in Table 15. We observe that we receive a substantial improvement which is beyond the predictive power of all models that we have met before! In fact, this averaging/blending seems a fairly efficient way to improve network models and to eliminate the variability introduced by different seeds and early stopping of gradient descent algorithms, the latter being motivated by the fact that we work with over-parametrized models (that may have a huge approximation capacity).

# 7 Challenging the networks with boosting

We come to the fun part now, and we are going to challenge our networks with boosting methods.

## 7.1 Boosting challenge

In (6.12) we have been averaging models. In a next (and final) step we would like to challenge this blended model by regression tree boosting. The aim of this boosting step is to see whether the regression tree boosting algorithm can still find substantial structure in the data which has not been discovered by the networks. For a general introduction to boosting we refer to our tutorial [8]. Here, we use the direct Poisson boosting machine (PBM) presented in our tutorial [22] and described in depth in the lecture notes [37].
We start from regression function $\widehat{\widehat{\mu}} : (\boldsymbol{x}, v) \mapsto \widehat{\widehat{\mu}}(\boldsymbol{x}, v)$, defined in (6.12). This provides us with a first (estimated) model

$$N_i \overset{\text{ind.}}{\sim} \text{Poi}\left(\widehat{\widehat{\mu}}(\boldsymbol{x}_i, v_i)\right), \qquad \text{for } i = 1, \ldots, n.$$

We can now try to enhance this model by considering an additional regression function $\chi : \mathcal{X}^+ \to \mathbb{R}_+$, $(\boldsymbol{x}, v) \mapsto \chi(\boldsymbol{x}, v)$ providing a new model

$$N_i \overset{\text{ind.}}{\sim} \text{Poi}\left(\chi(\boldsymbol{x}_i, v_i)\widehat{\widehat{\mu}}(\boldsymbol{x}_i, v_i)\right) \overset{\text{(d)}}{=} \text{Poi}\left(\chi(\boldsymbol{x}_i, v_i)\nu_i\right), \qquad \text{for } i = 1, \ldots, n,$$

where we consider the working weights $\nu_i = \widehat{\widehat{\mu}}(\boldsymbol{x}_i, v_i) > 0$ as (known) offsets in a Poisson regression model.

If this new regression function turns out to be identically equal to 1, i.e. $\chi(\cdot) \equiv 1$, then the first regression function $\widehat{\widehat{\mu}}(\cdot)$ is perfectly fine. Otherwise, the first regression function should be enhanced to a new regression function $(\boldsymbol{x}, v) \mapsto \widehat{\widehat{\mu}}^{+}(\boldsymbol{x}, v) = \chi(\boldsymbol{x}, v)\widehat{\widehat{\mu}}(\boldsymbol{x}, v)$, thus, this additional regression function $\chi(\cdot)$ serves at identifying missing structure in $\widehat{\widehat{\mu}}(\cdot)$.
In order to assess the quality of the (network) regression function $\widehat{\widehat{\mu}}(\cdot)$ we use a Poisson boosting machine to estimate the additional regression function $\chi(\cdot)$. For the Poisson boosting machine we refer to Section 5 of Noll et al. [22]. In order to determine an appropriate number of boosting steps we use stratified 10-fold cross-validation, that is, we partition the learning data $\mathcal{D}$ into 10 stratified subsets $\mathcal{D}_1, \ldots, \mathcal{D}_{10}$. As trigger for stratifying we use the number of claims $N_i$. The

Listing 10: R script for choosing 10 stratified subsets

```
1  set.seed(100)
2  # random ordering of learning data
3  learn$K <- runif(nrow(learn))
4  learn <- learn[order(learn$K),]
5  # order learning data according to the number of claims
6  learn <- learn[order(learn$ClaimNb, decreasing=TRUE),]
7  # choosing stratifying allocation of learning data
8  learn$K <- rep(1:10, length = nrow(learn))
9  # choose validation and training set for some k=1,...,10
10 validation_k <- learn[which(learn$K==k),]
11 train_k <- learn[which(learn$K!=k),]
```

corresponding R code is provided in Listing 10 (note that the learning data $\mathcal{D}$ is denoted by `learn`). For every $k = 1, \ldots, 10$ we receive validation set $\mathcal{D}_k$ and training set $\mathcal{D}_{-k} = \mathcal{D} \setminus \mathcal{D}_k$ on lines 10-11 of Listing 10. Based on these sets, we fit a Poisson boosting model $\widehat{\chi}^{(-k)}(\cdot)$ on the training data $\mathcal{D}_{-k}$ and we do an out-of-sample model validation on the validation set $\mathcal{D}_k$. This gives us out-of-sample validation losses for $k = 1, \ldots, 10$

$$\mathcal{L}_k = \mathcal{L}(\mathcal{D}_k, \widehat{\chi}^{(-k)}(\cdot)) \;=\; \frac{1}{|\mathcal{D}_k|} \sum_{(\boldsymbol{x}_i, v_i) \in \mathcal{D}_k} 2N_i \left[ \frac{\widehat{\chi}^{(-k)}(\boldsymbol{x}_i, v_i)\nu_i}{N_i} - 1 - \log\left( \frac{\widehat{\chi}^{(-k)}(\boldsymbol{x}_i, v_i)\nu_i}{N_i} \right) \right].$$

The stratified 10-fold cross-validation loss and the corresponding standard deviation estimate is then determined by

$$\mathrm{CV}^{(10)} = \frac{1}{10} \sum_{k=1}^{10} \mathcal{L}_k \qquad \text{and} \qquad \sigma_{\mathrm{CV}}^{(10)} = \sqrt{\frac{1}{9} \sum_{k=1}^{10} \left( \mathcal{L}_k - \mathrm{CV}^{(10)} \right)^2}.$$

The aim is to see whether the Poisson boosting machine is able to lower these cross-validation losses compared to the network estimations. We do this on our example from above

We start from the blended model $\widehat{\widehat{\mu}}(\cdot)$ given in (6.12). This provides us in-sample and out-of-sample losses of 28.85961 and 30.11778, see Table 15. We then apply a Poisson boosting machine based on standard binary split (SBS) regression trees of maximal depth `J0` $= 1, 2, 3$ and for $D = 30$ boosting iterations to each training set $\mathcal{D}_{-k}$. The corresponding R code is provided in Listing 11. This gives us for each maximal tree depth `J0` $= 1, 2, 3$ and for each boosting iteration $d = 1, \ldots, D$ the resulting stratified 10-fold cross-validation losses $\mathrm{CV}^{(10)}_{\texttt{J0},d}$ and the corresponding standard deviation estimates $\sigma^{(10)}_{\mathrm{CV}_{\texttt{J0}},d}$, see lines 19-22 in Listing 11. These

Listing 11: R script for cross-validated Poisson boosting

```
D <- 30                          # number of boosting iterations
validation.loss <- array(NA, c(10+2, D+1))
learn$predBB <- learn$predNN   # initialize working weights nu_i with network predictions
#
for (k in 1:10){
   validation_k <- learn[which(learn$K==k),]
   train_k <- learn[which(learn$K!=k0),]
   validation.loss[k,1] <- loss.function(validation_k$predBB, validation_k$ClaimNb))
   for (d in 1:D){
      tree_k <- rpart(cbind(predBB,ClaimNb) ~ Area + VehPower + VehAge + DrivAge +
                           BonusMalus + VehBrand + VehGas + Density + Region + Exposure,
                           data=train_k, method="poisson", control=rpart.control(xval=1,
                           minbucket=5000, maxdepth=J0, maxsurrogate=0, cp=0.00001))
      train_k$predBB <- predict(tree_k) * train_k$predBB
      validation_k$predBB <- predict(tree_k) * validation_k$predBB
      validation.loss[k,d+1] <- loss.function(validation_k$predBB, validation_k$ClaimNb))
                        }}
#
for (d in 1:(D+1)){
   validation.loss[K+1,d] <- mean(validation.loss[1:K,d])
   validation.loss[K+2,d] <- sd(validation.loss[1:K,d])
                        }
```

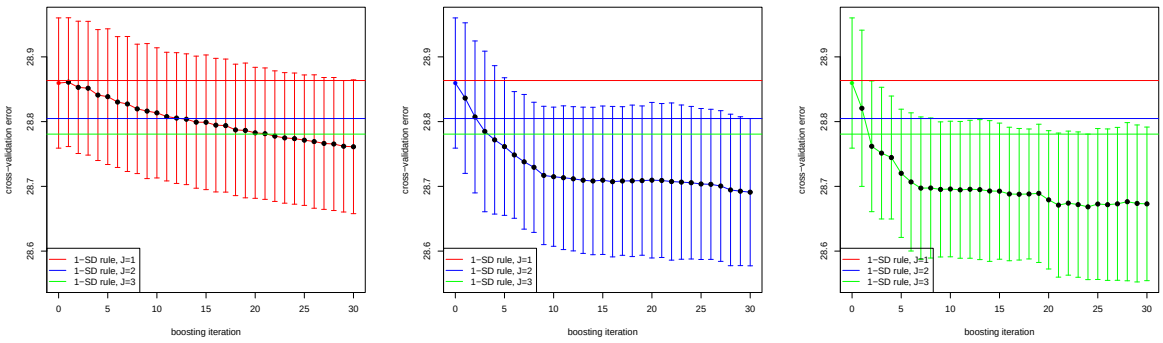


Figure 29: boosting challenge of the blended model $\widehat{\widehat{\mu}}(\cdot)$ given in (6.12): stratified 10-fold cross-validation losses $\mathrm{CV}^{(10)}_{\mathtt{J0},d} \pm \sigma^{(10)}_{\mathrm{CV}_{\mathtt{J0}},d}$ for iterations $d = 1, \ldots, D = 30$ for (lhs) maximal tree depth $\mathtt{J0} = 1$; (middle) maximal tree depth $\mathtt{J0} = 2$; and (rhs) maximal tree depth $\mathtt{J0} = 3$.

results are presented in Figure 30 for maximal tree depth $\mathtt{J0} = 1$ (lhs, red), maximal tree depth $\mathtt{J0} = 2$ (middle, blue), and maximal tree depth $\mathtt{J0} = 3$ (rhs, green). The horizontal lines show the 1-SD model selection rule for the necessary number of boosting steps.
For a maximal tree depth of $\mathtt{J0} = 1$ we see that all cross-validation losses $\mathrm{CV}^{(10)}_{1,d}$, $d = 1, \ldots, D$, (black dots) lie below the 1-SD rule line (horizontal red line in Figure 30), therefore, a boosting improvement with $\mathtt{J0} = 1$ is not justified. Note that a tree of depth 1 only allows for multiplicative interactions (under the log-link function). Such interactions can usually be found by (shallow) networks, because multiplicative interactions are additive on the canonical scale. The latter reflecting superimposing neurons in a hidden network layer.
For maximal tree depths $\mathtt{J0} = 2$ and 3 we receive roughly the same 1-SD rule lines (horizontal

blue and green lines in Figure 30). For `J0` $= 2$ this justifies 2 boosting iterations and for `J0` $= 3$ this gives 1 boosting iteration because $\mathrm{CV}^{(10)}_{2,d=2}$ and $\mathrm{CV}^{(10)}_{3,d=1}$ (black dots) lie above the 1-SD selection rule lines.
We now perform these boosting steps for `J0` $= 2$ and 3 on the entire learning data $\mathcal{D}$ and analyze them on the corresponding in-sample and out-of-sample losses. These results are presented in Table 16. In both cases we see an improvement of the loss figures, and the initial network

| | in-sample loss | out-of-sample loss |
|---|---|---|
| blended model (6.12) | 28.85961 | 30.11778 |
| 1st boosting step depth `J0` $= 2$ | 28.83204 | 30.07700 |
| 2nd boosting step depth `J0` $= 2$ | 28.81765 | 30.06170 |
| 1st boosting step depth `J0` $= 3$ | 28.81675 | 30.08798 |

Table 16: boosting steps with Poisson regression trees of depth `J0` $= 2, 3$; losses are in $10^{-2}$.

estimator $\widehat{\widehat{\mu}}(\cdot)$ should be enhanced by the resulting boosting regression estimators $\chi(\cdot)$, resulting in the new regression function $\widehat{\widehat{\mu}}^{+}(\cdot) = \chi(\cdot)\widehat{\widehat{\mu}}(\cdot)$. This provides out-of-sample losses of 30.06170 and 30.08798 for `J0` $= 2$ and `J0` $= 3$, respectively. Thus, we receive an improvement which says that the blended network model may (still) not be optimal.

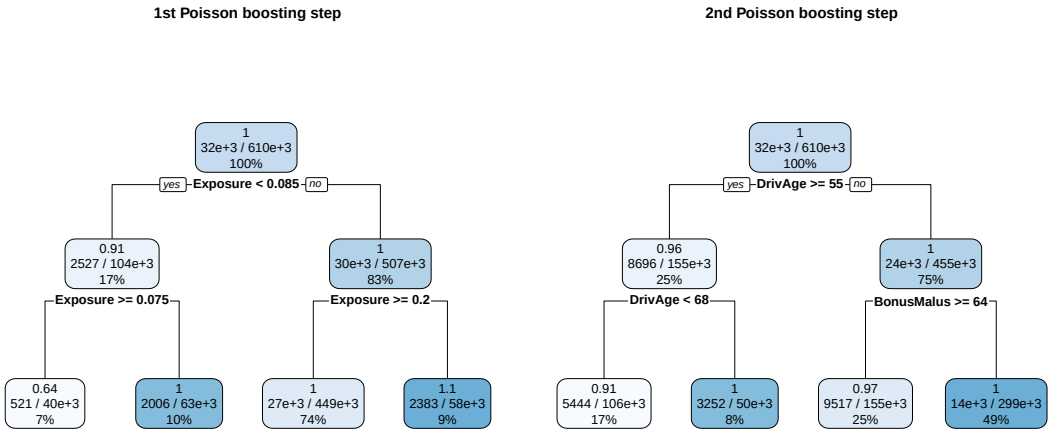


Figure 30: boosting challenge of the blended model $\widehat{\widehat{\mu}}(\cdot)$ given in (6.12): (lhs) 1st iteration of boosting algorithm; (rhs) 2nd iteration of boosting algorithm for maximal tree depth `J0` $= 2$ (note that these regression trees have been plotted with the R function `rpart.plot` which applies rounding to the labels).

In Figure 30 we analyze the two boosting steps that lead to the out-of-sample loss of 30.06170 for `J0` $= 2$. First boosting step in Figure 30 (lhs): The first 3 splits are done w.r.t. feature component `Exposure`, i.e. our blended network does still allow for some improvement in considering the exposure variable. Note that this is a marginal distribution improvement because (1) it does not involve interactions, and (2) the log-link implies a multiplicative regression model. This first

improvement should not be over-stated from a practical point of view because it is doubtable whether we should really run the exposure through the regression function or whether it should be considered as a constant offset variable. Second boosting step in Figure 30 (rhs): The boosting step proposes an improvement w.r.t. `DrivAge` and `BonusMalus`, thus, there might be an interaction term between these two variables that may still be improved, also the marginal of `DrivAge` may still allow for improvement.

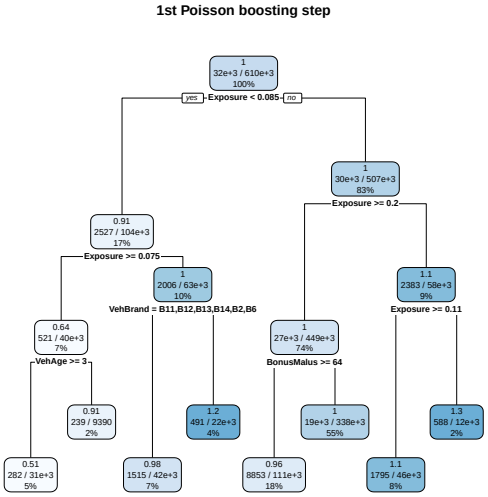


Figure 31: boosting challenge of the blended model $\widehat{\widehat{\mu}}(\cdot)$ given in (6.12): 1st iteration of boosting algorithm for maximal tree depth `J0` $= 3$ (note that these regression trees have been plotted with the R function `rpart.plot` which applies rounding to the labels).

In Figure 31 we analyze the boosting step that leads to the out-of-sample loss of 30.08798 for `J0` $= 3$. The first three splits are identical to Figure 30 (lhs). Thereafter, some interaction terms are added by considering additionally the variables `VehAge`, `VehBrand` and `BonusMalus`. However, altogether this leads to a slight over-fitting compared to the regression tree boosting of depth `J0` $= 2$. Concluding, the major improvement on the blended model $\widehat{\widehat{\mu}}(\cdot)$ is achieved by a better marginal modeling of the exposure variable.

## 7.2 Outlook: enhancement of networks

In the previous section we have provided boosting improved networks, see Table 16. We can now work with these results. However, these might be a bit unsatisfactory because we may want to fully rely on networks.[20] One way to proceed is to use the working weights $\nu_i = \widehat{\widehat{\mu}}(\boldsymbol{x}_i, v_i) > 0$ as offsets in a next network Poisson regression modeling approach. That is, we design a network $\chi : \mathcal{X}^+ \to \mathbb{R}_+$ under the model assumption

$$N_i \overset{\text{ind.}}{\sim} \text{Poi}\left(\chi(\boldsymbol{x}_i, v_i)\nu_i\right), \qquad \text{for } i = 1, \ldots, n. \tag{7.1}$$

[20] Remark that boosting with regression trees leads to discontinuities in regression functions which is sometimes unwanted. Moreover, also extrapolation is more questionable with regression trees because slopes are zero at boundaries.

The resulting regression function $\widehat{\widehat{\mu}}^{+}(\cdot) = \chi(\cdot)\widehat{\widehat{\mu}}(\cdot)$ can then be understood as a new network, which is a "network boosted network". This network boosted network has also another interpretation in machine learning parlance, namely, we fit a first (blended) network $\widehat{\widehat{\mu}}(\cdot)$ to the data. This first network is then used in a skip connection of a new network architecture. Since we do not further modify this initial $\widehat{\widehat{\mu}}(\cdot)$ network, all parameters that are related to this first network are frozen during training of the second network architecture, i.e. the skip connection is declared to be non-trainable. For more on this topic we refer to our tutorial [31]

# 8 Analysis of out-of-sample results

In this section we provide a final analysis of our results. This is done in a similar spirit as in the last section of Noll et al. [22]. We compare the Poisson boosting machine model PBM3 of Noll et al. [22], and selected models of this tutorial as highlighted in Table 17.

| method | in-sample loss | out-of-sample loss |
|---|---|---|
| boosting machine model PBM3 of Noll et al. [22], Table 11 | 30.13151 | 31.46842 |
| $(q_1, q_2, q_3) = (10, 15, 10)$, Table 14 | 28.95891 | 30.25011 |
| blended model $\widehat{\widehat{\mu}}(\cdot)$, formula (6.12) | 28.85961 | 30.11778 |
| boosted with depth `J0` $= 2$, Table 16 | 28.81765 | 30.06170 |

Table 17: comparison of selected models; losses are in $10^{-2}$.

The corresponding in- and out-of-sample losses are summarized in Table 17. We remark that this comparison is not completely fair because in model PBM3 we have not been modeling the exposures, but we have worked with model assumption (1.1). To make the comparison more fair we could also lift model PBM3 to model assumptions (1.2), and then run the boosting algorithm under these new model assumptions (which also partitions w.r.t. `Exposure`): the resulting out-of-sample loss is 30.22719, thus, on a competitive level. We will not further pursue this latter model because it has discontinuities in the exposure variable which we consider as a major disadvantage. Therefore, we refrain from further refining model PBM3. Note that the tree boosted models have the same deficiency if we include the exposure as feature component in the boosting step (which we do).
We calculate for every label the average marginal out-of-sample loss on the test data $\mathcal{T}$. These are given by

$$\frac{1}{\sum_{t=1}^{n_{\mathcal{T}}} \mathbb{1}_{\{x_{t,l}=x\}}} \sum_{t=1}^{n_{\mathcal{T}}} 2N_t \left[ \frac{\widehat{\mu}(\boldsymbol{x}_t, v_t)}{N_t} - 1 - \log\left( \frac{\widehat{\mu}(\boldsymbol{x}_t, v_t)}{N_t} \right) \right] \mathbb{1}_{\{x_{t,l}=x\}},$$

for $x$ being in the domain of the $l$-th component of $\boldsymbol{x} \in \mathcal{X}$, and $\widehat{\mu}(\cdot)$ being an estimated model. These statistics are plotted in Figure 32: Model PBM3 in cyan color, the deep network model $(q_1, q_2, q_3) = (10, 15, 10)$ in blue color, the blended network model (6.12) in red color, and the boosting improved version in orange color; moreover, the gray dotted line shows the volumes (number of policies) per label (with $y$-axis on the rhs). We start by discussing Model PBM3 (cyan) and the deep network results (blue). From Figure 32 (top, lhs) we can see that the network results are better over all `Area` codes. Big improvements in out-of-sample losses are made by the latter model for `VehAge` $= 0$ and `VehBrand` $=$ B12. In Figure 33 (lhs) we provide the same figure,

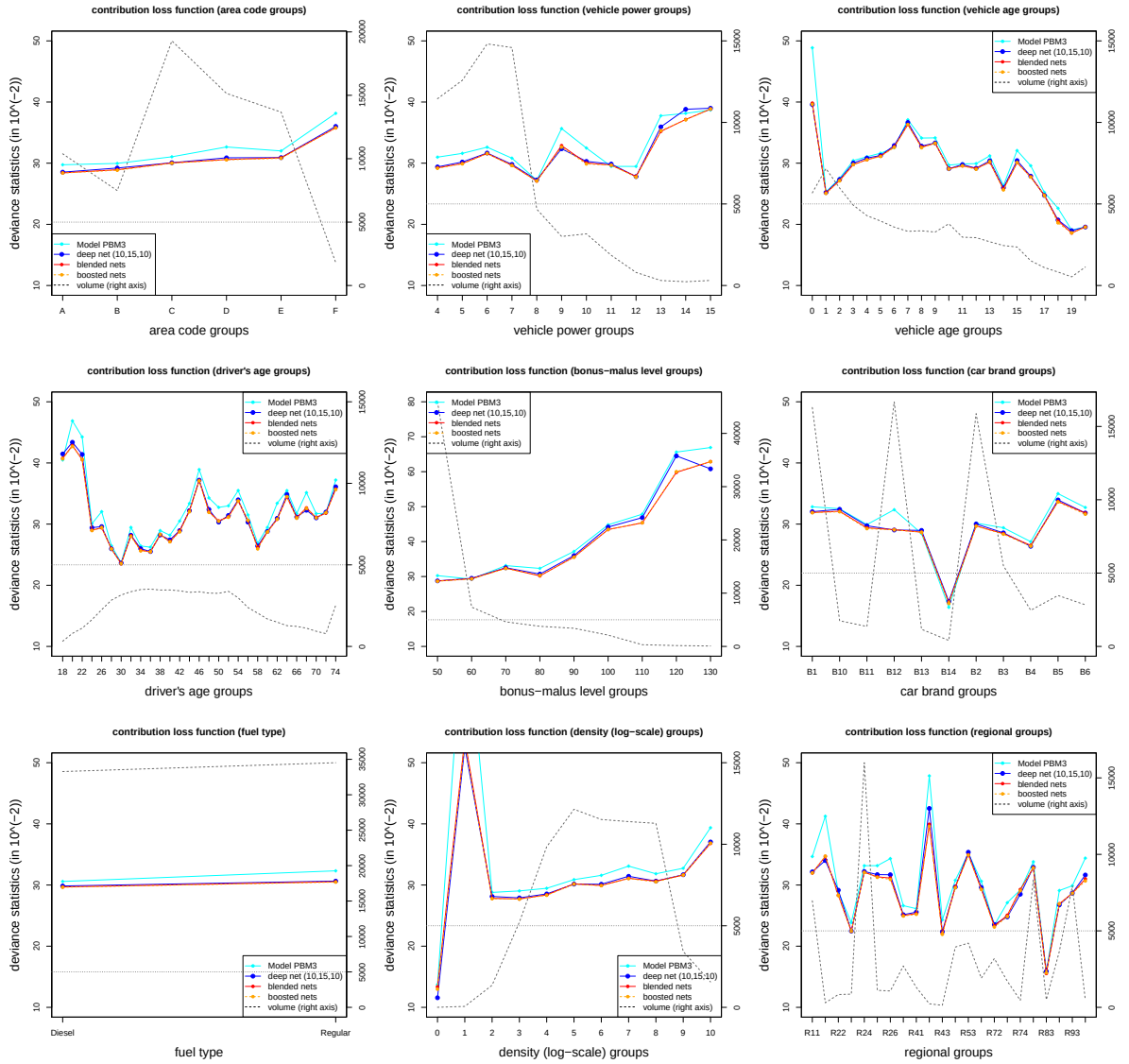


Figure 32: average marginal out-of-sample loss per label on the test data set $\mathcal{T}$: comparison of Model PBM3 (cyan), deep network model $(q_1, q_2, q_3) = (10, 15, 10)$ (blue), blended model (6.12) (red), and the boosting improved model (orange), units on the $y$-axes are in $10^{-2}$.

but as a function of the exposure. Recall that for Model PBM3 we have model assumption (1.1) (which is linear in the exposure) and for the network we have model assumption (1.2) (which allows for non-linearities in the exposure modeling). We observe major improvements in the volume modeling for `Exposure` $\in (0, 0.2]$, i.e. for short exposures, see loss statistics in Figure 33 (lhs). These short exposures also interact with `VehAge` $= 0$ and `VehBrand` $=$ B12, see Figure 2. This explains the previously mentioned improvements in out-of-sample losses on these labels. In Figure 33 (rhs) we also provide the resulting marginal frequencies for different exposures. We observe that this is clearly non-linear in the exposure and that the models $\mu(\boldsymbol{x}, v)$ are able to capture this non-linearity.

The blending (red) and boosting (orange) of the network models provides further improvements,

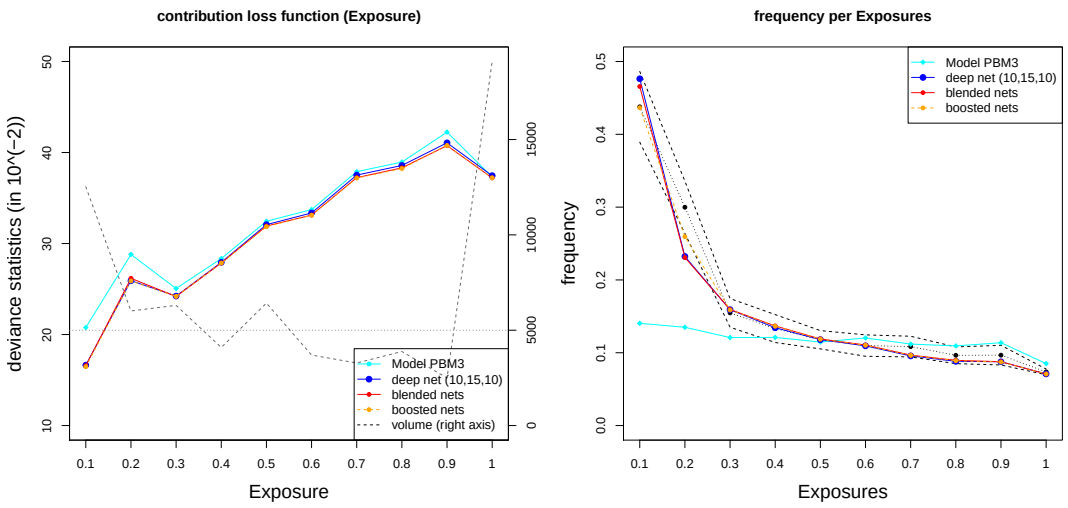


Figure 33: (lhs) average marginal out-of-sample loss per exposure level and (rhs) marginal frequency on test data set $\mathcal{T}$: comparison of Model PBM3 (cyan), deep network model $(q_1, q_2, q_3) = (10, 15, 10)$ (blue), blended model (6.12) (red), and the boosting improved model (orange); units on the $y$-axes are in $10^{-2}$ (lhs).

i.e. in general the red and orange lines are below the blue ones in Figure 32. However, improvements from blending to boosting are hardly visible in these plots. They can slightly be seen in Figure 33 (lhs) because the first 3 splits are performed on the variable `Exposure`, see Figure 30 (lhs).

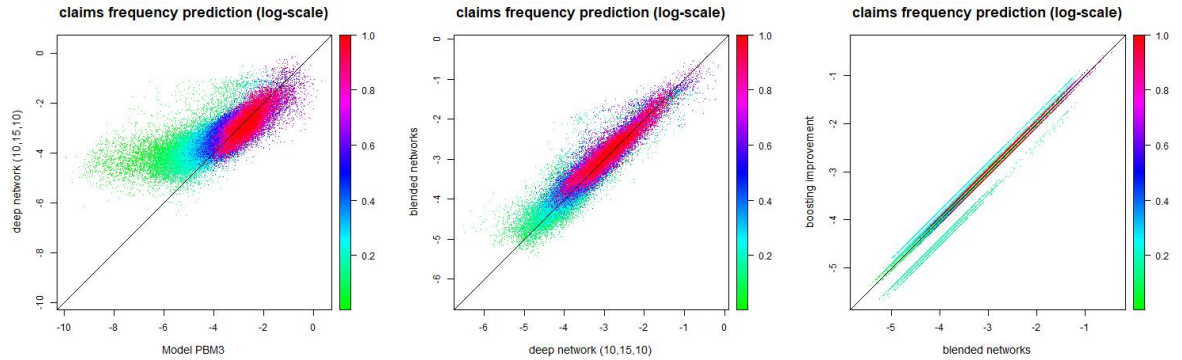


Figure 34: Out-of-sample claim frequency predictions (on log-scales) of the models considered in Table 17.

In Figure 34 we present the resulting claim frequency predictions of the 4 models considered in Table 17. The figure on the lhs compares Model PBM3 of [22] to the deep network solution with $(q_1, q_2, q_3) = (10, 15, 10)$. We observe that particularly on small volumes (green color) we receive big differences (note that the plot is on the log-scale). This (again) clearly indicates that volume modeling should not be done in a linear fashion for this data set (though a non-linear inclusion may be doubtable from a practical and interpretation point of view).

Figure 34 (middle) compares the deep network with $(q_1, q_2, q_3) = (10, 15, 10)$ hidden neurons to the blended prediction of the 12 considered network. We see some fluctuation around the diagonal line, which shows that these two models may have substantial differences on individual policy level, the latter one being the more robust one (because it is an averaged model).
Finally, we compare the boosting improved model to the blended one. These two models differ on parallel lines which comes from the fact that the tree boosting only considers totally 6 binary splits, thus, we receive a small number of clusters which are illustrated by the lines. Moreover, these splits are mainly performed on small exposures which explains the corresponding colors. This finishes our example.

# 9 Conclusions and outlook

This tutorial is considering the use of network regression models for claim frequency modeling in insurance. We have discussed all aspects of designing such a network regression model: this includes feature pre-processing, choice of loss function, choice of explicit network architecture, class imbalance problem, over-fitting, regularization, dropout, as well as model blending. Moreover, we have discussed the importance of the balance property which, in general, requires bias regularization. For the numerical analysis we have used the R interface to Keras (an API to TensorFlow). Moreover, we have provided graphical tools to analyze networks, we have discussed model improvements by boosting, and as a side product we have seen that a linear time exposure is non-optimal for our claim frequency data set.
A main deficiency of our considerations is that we have not been discussing prediction uncertainty which, in particular, should also include model uncertainty. At the moment, this is an open issue that requires further research. This will require a more systematic selection of models and model calibration.
The previous analysis has been focusing on claim frequencies in motor third party liability insurance which, by nature, is rather nice data for sufficiently large insurance portfolios. We expect much more difficulties for claim size modeling, in particular, caused by larger claims. It is another open issue to deal with such situations.
Another open point is to deal with high-dimensional feature spaces that contain many categorical feature components. In such situations one can easily run into curse of dimensionality problems that have not been present in our analysis. We believe that a more efficient way to represent categorical feature components is to use embedding layers, we refer to our tutorial [31]. Moreover, missing values will pose another challenge we may have to deal with, here maybe generative-adversarial networks (GAN) are of help because these manage to impute missing values. Finally, if there are time or causal relationships in the data there it might be more beneficial to use recurrent neural networks that can deal with such situations, we refer to our tutorial [29].

**Acknowledgment.** We would like to kindly thank Jürg Schelldorfer (Swiss Re), Ronald Richman (QED Actuaries and Consultants), Christian Lorentzen (Mobiliar), Andrea Gabrielli (ETH Zurich), John Ery (ETH Zurich), Ulrich Riegel (Munich Re) and Guangyuan Gao (Renmin, University of China) for detailed comments that have helped us to substantially improve this tutorial.

# References


[1] Arnold, V.I. (1957). On functions of three variables. *Doklady Akademii Nauk SSSR* **114/4**, 679-681.

[2] Barron, A.R. (1993). Universal approximation bounds for superpositions of sigmoidal functions. *IEEE Transactions of Information Theory* **39/3**, 930-945.

[3] CASdatasets Package Vignette (2016). Reference Manual, May 28, 2016. Version 1.0-6. Available from `http://cas.uqam.ca`.

[4] Charpentier, A. (2015). *Computational Actuarial Science with R.* CRC Press.

[5] Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems* **2**, 303-314.

[6] Döhler, S., Rüschendorf, L. (2001). An approximation result for nets in functional estimation. *Statistics and Probability Letters* **52**, 373-380.

[7] Efron, B., Hastie, T. (2016). *Computer Age Statistical Inference.* Cambridge University Press.

[8] Ferrario, A., Hämmerli, R. (2019). On boosting: theory and applications. *SSRN Manuscript* ID 3402687. Version June, 2019.

[9] Glorot, X., Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. In: Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics, *Proceedings of Machine Learning Research* **9**, 249-256.

[10] Gneiting, T. (2011). Making and evaluation point forecasts. *Journal of the American Statistical Association* **106/494**, 746-762.

[11] Goodfellow, I., Bengio, Y., Courville, A. (2016). *Deep Learning.* MIT Press, http://www.deeplearningbook.org

[12] Grohs, P., Perekrestenko, D., Elbrächter, D., Bölcskei, H. (2019). Deep neural network approximation theory. Submitted to *IEEE Transactions on Information Theory* (invited paper).

[13] Hastie, T., Tibshirani, R., Friedman, J. (2009). *The Elements of Statistical Learning. Data Mining, Inference, and Prediction.* 2nd edition. Springer Series in Statistics.

[14] Hastie, T., Tibshirani, R., Wainwright, M. (2015). *Statistical Learning with Sparsity: The Lasso and Generalizations.* CRC Press.

[15] Hornik, K., Stinchcombe, M., White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural Networks* **2**, 359-366.

[16] Isenbeck, M., Rüschendorf, L. (1992). Completeness in location families. *Probability and Mathematical Statistics* **13**, 321-343.

[17] Kolmogorov, A. (1957). On the representation of continuous functions of many variables by superposition of continuous functions of one variable and addition. *Doklady Akademii Nauk SSSR* **114/5**, 953-956.

[18] Leshno, M., Lin, V.Y., Pinkus, A., Schocken, S. (1993). Multilayer feedforward networks with a nonpolynomial activation function can approximate any function. *Neural Networks* **6/6**, 861-867.

[19] Makavoz, Y. (1996). Random approximants and neural networks. *Journal of Approximation Theory* **85**, 98-109.

[20] Montúfar, G., Pascanu, R., Cho, K., Bengio, Y. (2014). On the number of linear regions of deep neural networks. *Neural Information Processing Systems Proceedings*$^{\beta}$ **27**, 2924-2932.

[21] Nielsen, M. (2017). *Neural Networks and Deep Learning*. Online book available on http://neuralnetworksanddeeplearning.com

[22] Noll, A., Salzmann, R., Wüthrich, M.V. (2018). Case study: French motor third-party liability claims. *SSRN Manuscript* ID 3164764. Version March 4, 2020.

[23] Park, J., Sandberg, I. (1991). Universal approximation using radial-basis function networks. *Neural Computation* **3**, 246-257.

[24] Park, J., Sandberg, I. (1993). Approximation and radial-basis function networks. *Neural Computation* **5**, 305-316.

[25] Petrushev, P. (1999). Approximation by ridge functions and neural networks. *SIAM Journal on Mathematical Analysis* **30/1**, 155-189.

[26] Rentzmann, S., Wüthrich, M.V. (2019). Unsupervised learning: What is a sports car? *SSRN Manuscript* ID 3439358. Version of October 9, 2019.

[27] Richman, R. (2018). AI in actuarial science. *SSRN Manuscript* ID 3218082.

[28] Richman, R., Wüthrich, M.V. (2018). A neural network extension of the Lee-Carter model to multiple populations. *SSRN Manuscript* ID 3270877. To appear in *Annals of Actuarial Science*.

[29] Richman, R., Wüthrich, M.V. (2019). Lee and Carter go machine learning: recurrent neural networks. *SSRN Manuscript* ID 3441030, Version of August 29, 2019.

[30] Rüger, S.M., Ossen, A. (1997). The metric structure of weight space. *Neural Processing Letters* **5**, 63-72.

[31] Schelldorfer, J., Wüthrich, M.V. (2019). Nesting classical actuarial models into neural networks. *SSRN Manuscript* ID 3320525. Version of January 22, 2019.

[32] Shaham, U., Cloninger, A., Coifman, R.R. (2015). Provable approximation properties for deep neural networks. *arXiv:*1509.07385v3. Version March 28, 2016.

[33] Verbelen, R., Antonio, K., and Claeskens, G. (2018). Unraveling the predictive power of telematics data in car insurance. *Journal of the Royal Statistical Society: Series C (Applied Statistics)* **67**, 1275-1304.

[34] Wüthrich, M.V. (2013). *Non-Life Insurance: Mathematics & Statistics*. SSRN Manuscript ID 2319328. Version January 7, 2020.

[35] Wüthrich, M.V. (2019). From generalized linear models to neural networks, and back. *SSRN Manuscript* ID 3491790. Version March 2, 2020.

[36] Wüthrich, M.V. (2020). Bias regularization in neural network models for general insurance pricing. To appear in *European Actuarial Journal*.

[37] Wüthrich, M.V., Buser, C. (2016). *Data Analytics for Non-Life Insurance Pricing*. SSRN Manuscript ID 2870308. Version June 4, 2019.

[38] Wüthrich, M.V., Merz, M. (2019). Editorial: Yes, we CANN! *ASTIN Bulletin* **49/1**, 1-3.

[39] Yukich, J., Stinchcombe, M., White, H. (1995). Sup-norm approximation bounds for networks through probabilistic methods. *IEEE Transactions on Information Theory* **41/4**, 1021-1027.

[40] Zaslavsky, T. (1975). Facing up to arrangements: face-count formulas for partitions of space by hyperplanes. *Memoirs of the American Mathematical Society* **154**.